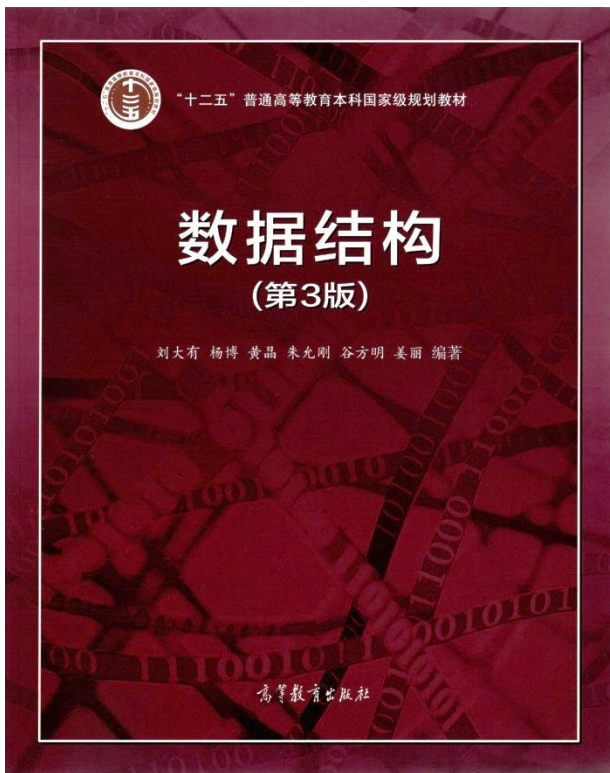


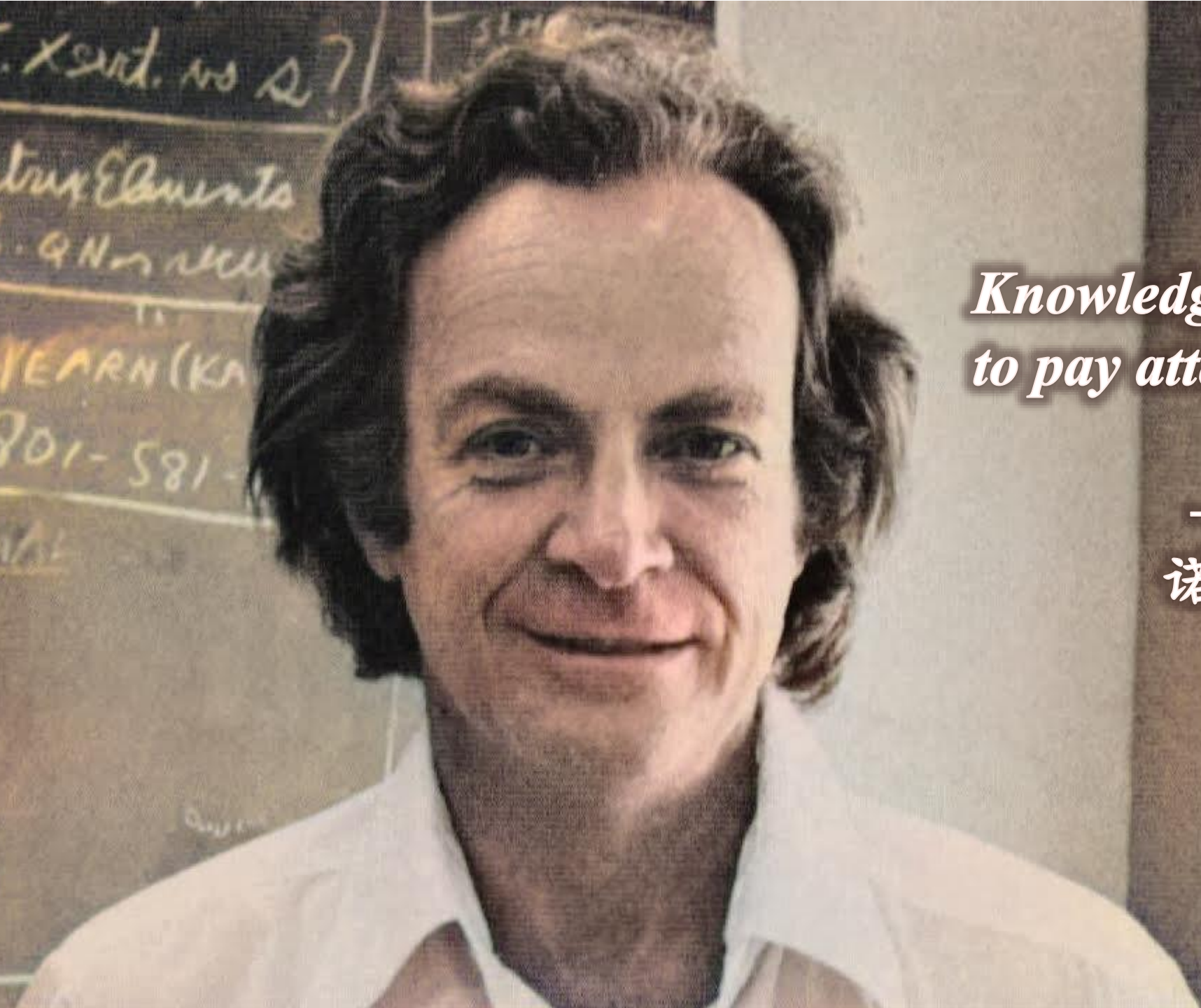


# 树和二叉树的应用

- 压缩与哈夫曼树
- 表达式树
- 并查集
- 其他应用举例

数据之法  
结构之美  
算法之道

zhuyungang@jlu.edu.cn

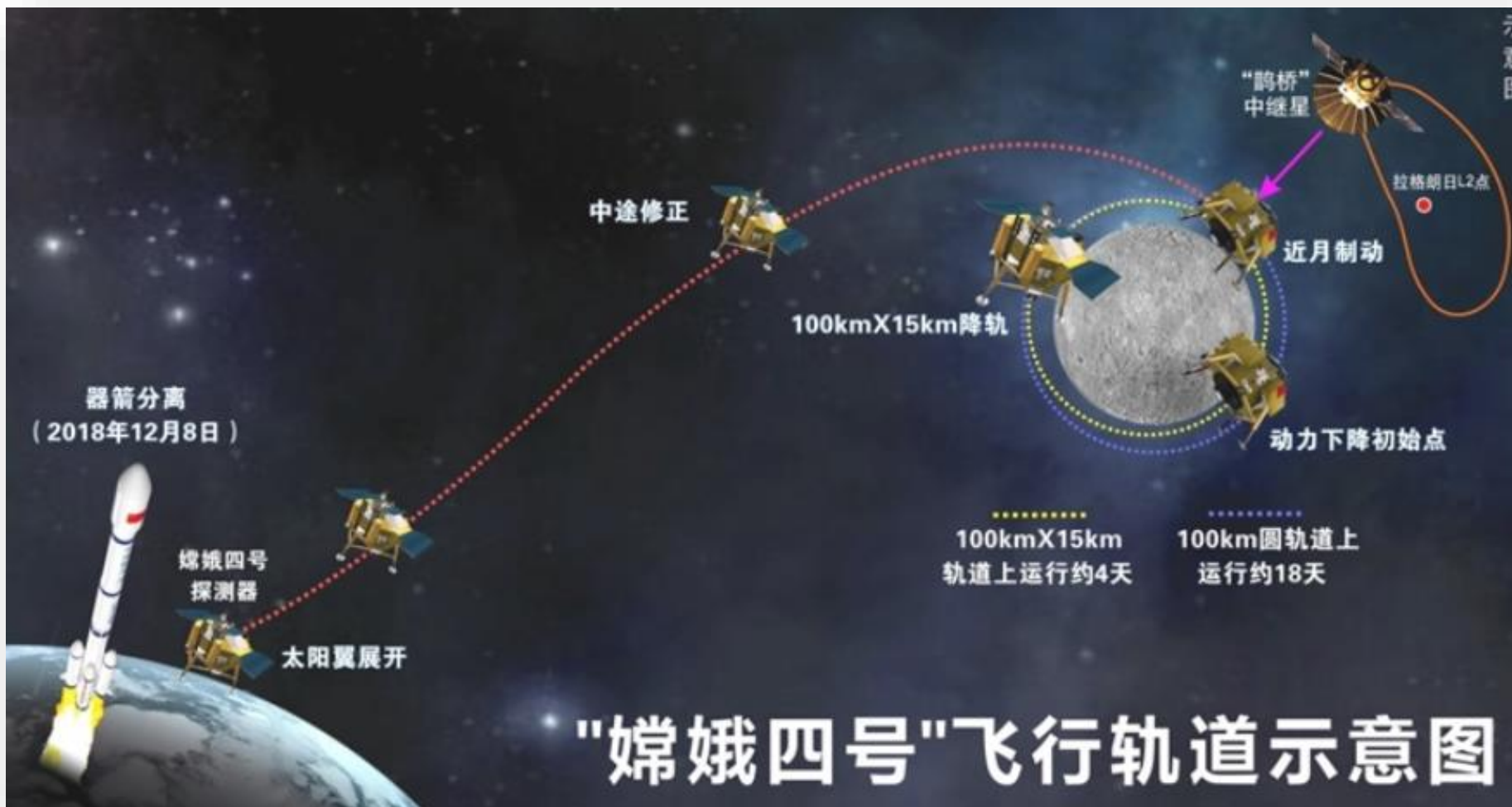


*Knowledge is not free. You have  
to pay attention.*

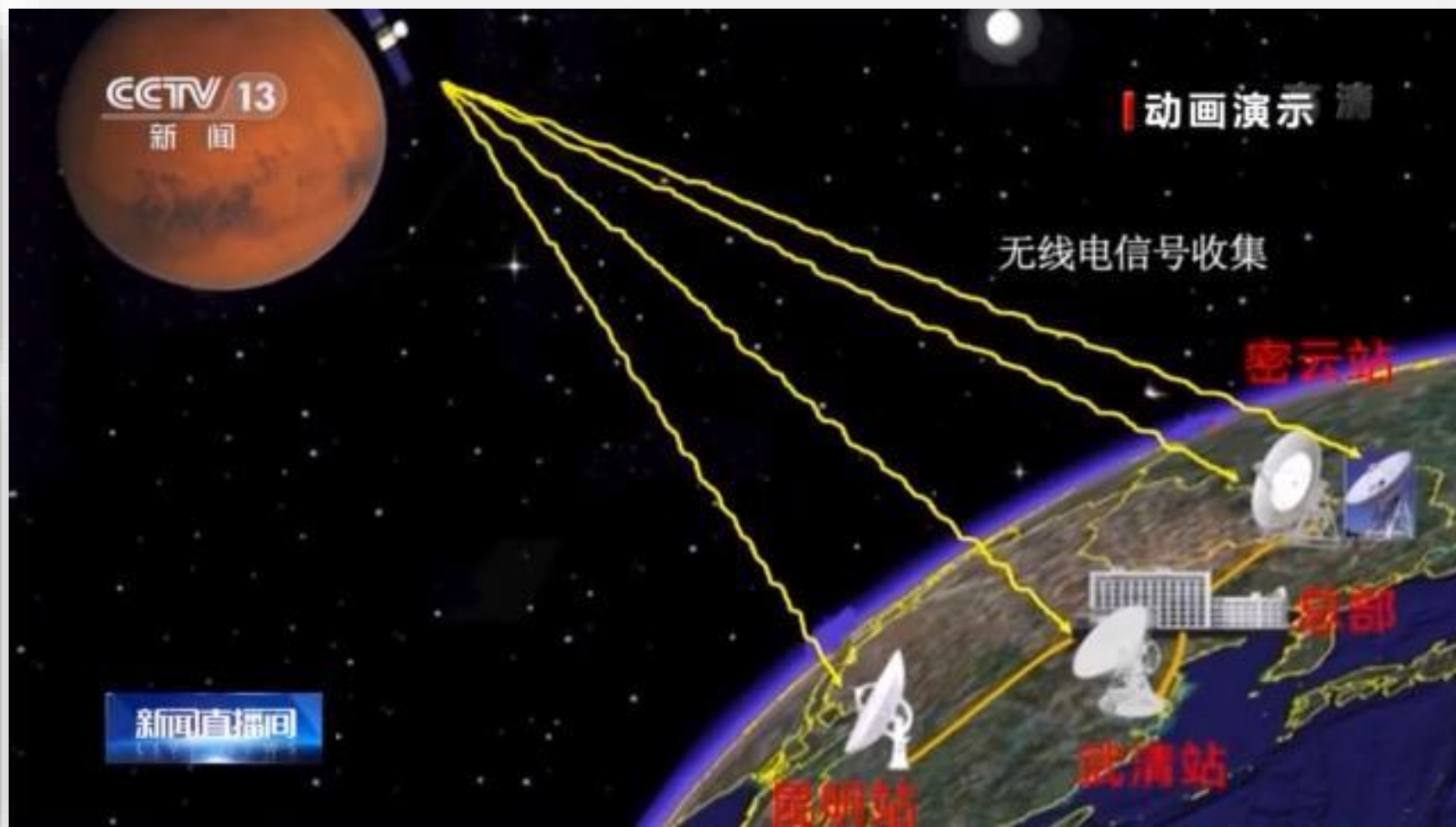
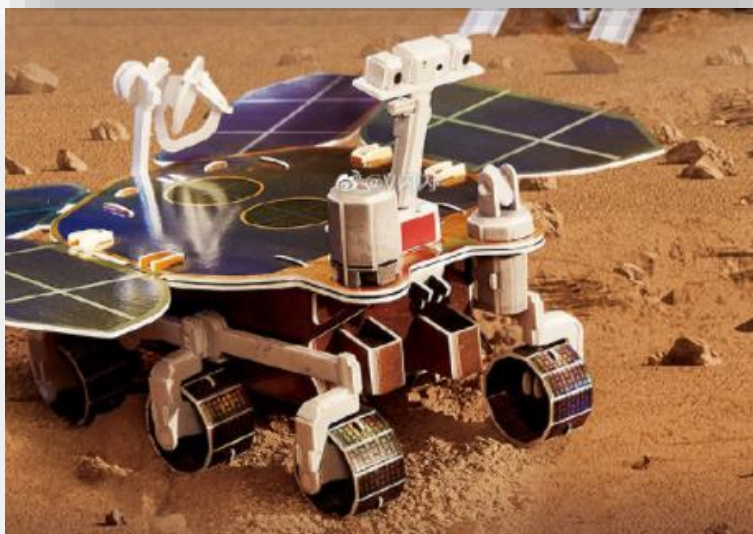
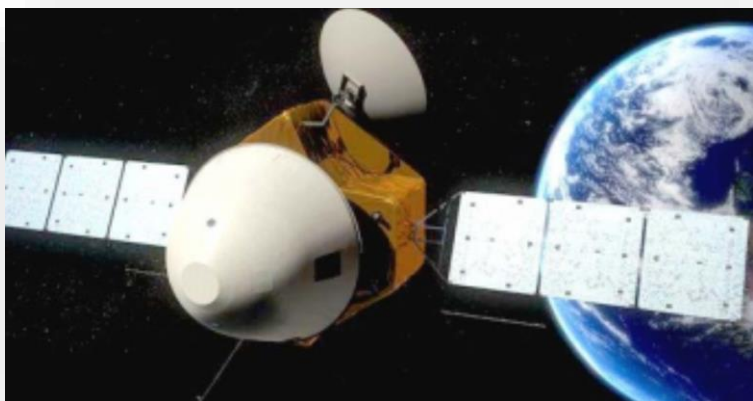
——*Richard Feynman*  
诺贝尔物理学奖获得者  
美国科学院院士  
加州理工学院教授



## 嫦娥月球探测器，与地球传输图像，需进行数据压缩



# 天问一号火星探测器与祝融号火星车，与地球传输图像，需进行数据压缩



## 数据压缩

- **数据压缩**是计算机科学中的重要技术。
- 数据压缩过程称为**编码**，即将文件中的每个字符均转换为一个唯一的二进制位串。
- 数据解压过程称为**解码**，即将二进制位串转换为对应的字符。



# 信息编码

假设有一个文本文件仅包含4种字符：A、B、C、D，且文件中有11个A，4个B，3个C，2个D。

若采用等长编码，因为 $\lceil \log_2 4 \rceil = 2$ ，所以每个字符都至少由一个2位的二进制数表示。于是文件所需存储空间（文件的总编码长度）为：

$$(11 + 4 + 3 + 2) \times 2 = 40 \text{ bit} = 5 \text{ Byte}$$

还有更好的方案么？



# 信息编码

假设有一个文本文件仅包含4种字符：A、B、C、D，且文件中有11个A，4个B，3个C，2个D。

若采用等长编码，因为 $\lceil \log_2 4 \rceil = 2$ ，所以每个字符都至少由一个2位的二进制数表示。于是文件所需存储空间（文件的总编码长度）为：

$$(11 + 4 + 3 + 2) \times 2 = 40 \text{ bit} = 5 \text{ Byte}$$

字符被出现的频率不同，可否借助这一信息，设计不等长编码，使文件总长度更短？



## 如何才能压缩总编码长度？

假设有一个文件中有**100个A**，**1个B**，**1个C**，**1个D**。

编码策略1:

A、B、C、D都用2个二进制位表示。

总编码长度:  $103 \times 2 = 206 \text{ bit}$

编码策略2:

A用1个二进制位表示; B、C、D用5个二进制位表示。

总编码长度:  $100 + 15 = 115 \text{ bit}$

采用**不等长**编码，希望:

① **熵编码**: 文件中出现频率高的字符的编码长度尽可能短。



## 解码过程不能出现歧义性

| 字符 | 编码   |
|----|------|
| A  | 10   |
| B  | 01   |
| C  | 1001 |

歧义：1001 = C 还是 AB?

原因：A的编码是 C 的前缀。

采用不等长编码，希望：

② 前缀码（无前缀冲突编码，Prefix-Free Codes）：字符集中任何字符的编码都不是其它字符的编码的前缀。

怎样的前缀码才能使文件的总编码长度最短？

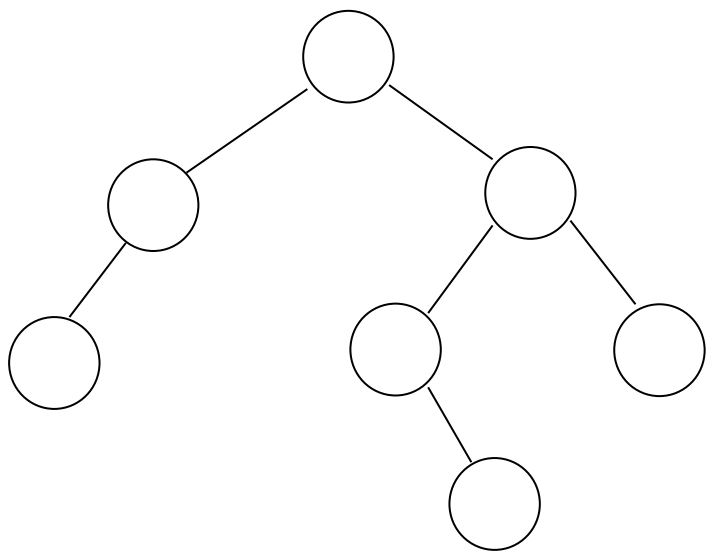
**最优编码问题描述：**

设组成文件的字符集 $A=\{a_1, a_2, \dots, a_n\}$ ，其中 $a_i$ 出现的次数为 $c_i$ ， $a_i$ 的编码长度为 $l_i$ 。设计一个前缀码方案，使文件的总编码长度最小：

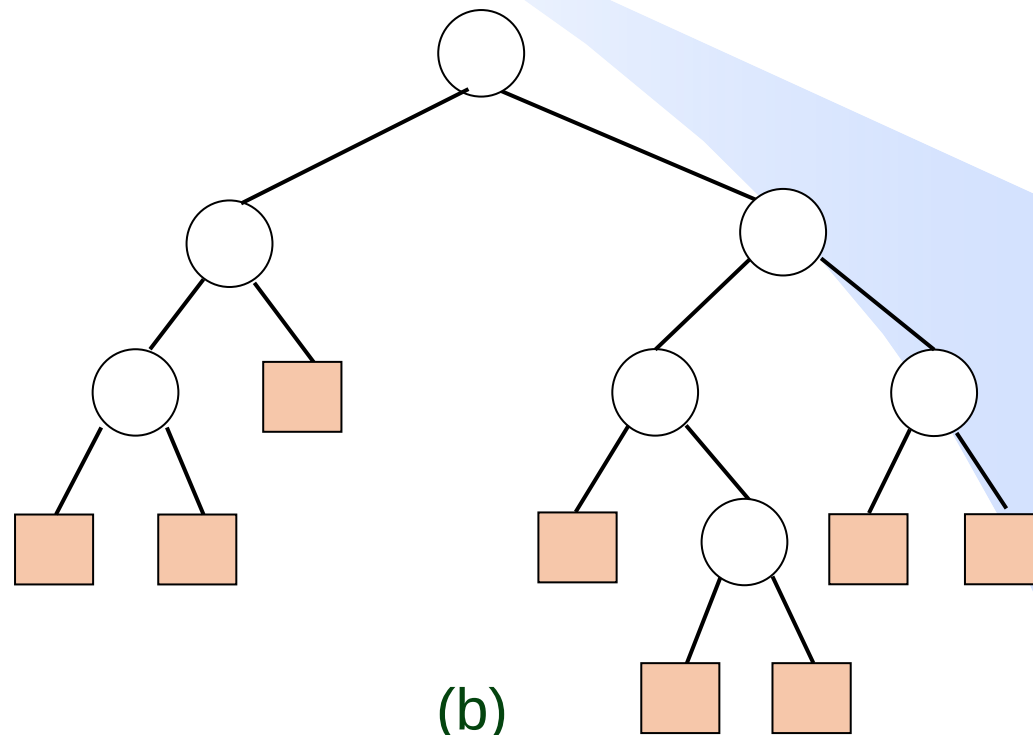
$$\min \sum_{i=1}^n c_i \cdot l_i$$

## 扩充二叉树

**定义** 在二叉树中空指针的位置，都增加特殊的结点（空叶结点），由此生成的二叉树称为扩充二叉树。



(a)

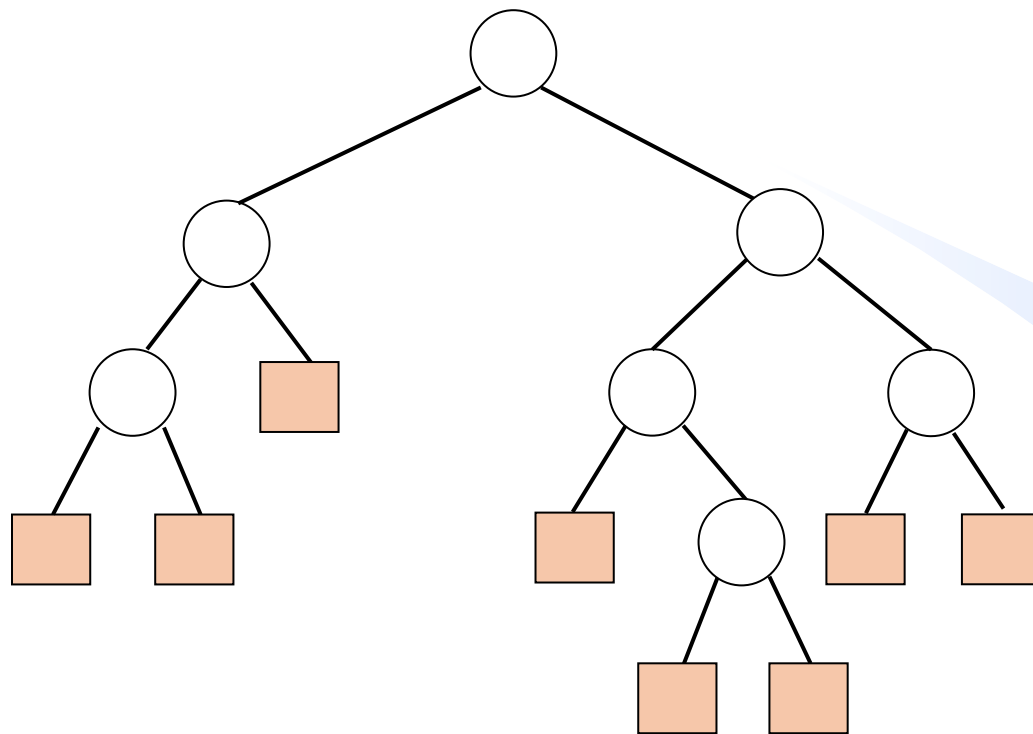


(b)

二叉树及其对应的扩充二叉树



## 扩充二叉树



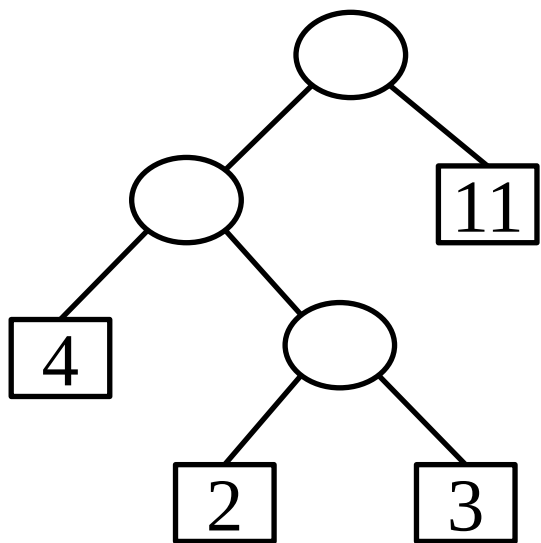
- 称圆形结点为内结点，方形结点为外结点。
- 每个内结点都有2个孩子，每个外结点没有孩子。
- 规定空二叉树的扩充二叉树是只有一个外结点。

# 加权路径长度 (*Weighted Path Length, WPL*)

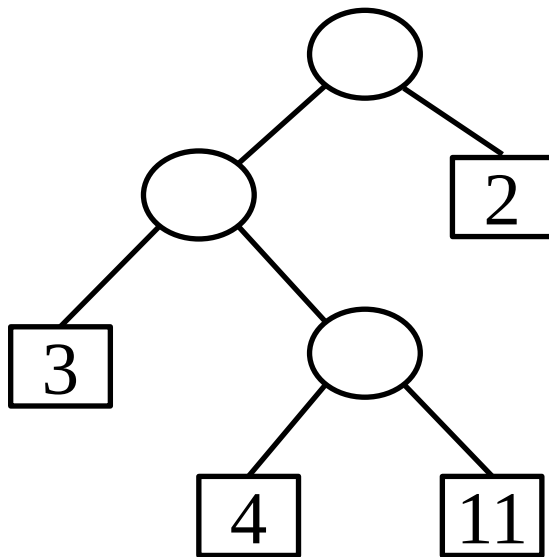
设扩充二叉树有 $n$ 个外结点，为每个外结点赋予一个实数，称为该结点的权值，第 $i$ 个外结点的权值为 $w_i$ ，深度为 $L_i$ ，则**加权路径长度**定义为：

$$WPL = \sum_{i=1}^n w_i L_i$$

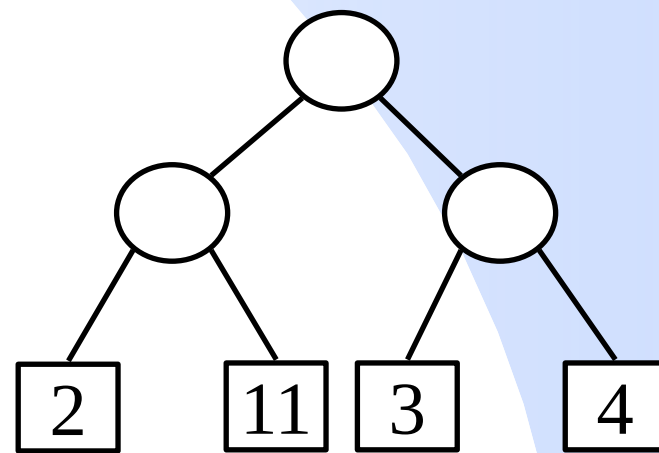
$n$ 个带权外结点构成的所有扩充二叉树中，WPL值最小者称为**最优二叉树**。



$$4 \times 2 + 2 \times 3 + 3 \times 3 + 11 \times 1 = 34$$



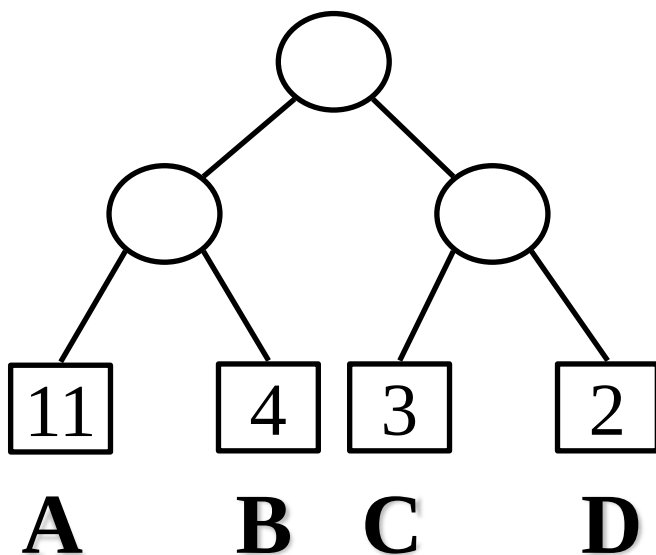
$$3 \times 2 + 4 \times 3 + 11 \times 3 + 2 \times 1 = 53$$



$$2 \times 2 + 11 \times 2 + 3 \times 2 + 4 \times 2 = 40$$

➤ 一种文件编码方案可以映射为一棵扩充二叉树。

| 文件编码          | 扩充二叉树              |
|---------------|--------------------|
| 字符 $a_i$      | 外结点 $i$            |
| 字符的出现次数 $c_i$ | 外结点的权值 $w_i$       |
| 字符的编码长度 $l_i$ | 外结点的深度 $L_i$       |
| 文件总编码长度       | 扩充二叉树的 <b>WPL值</b> |

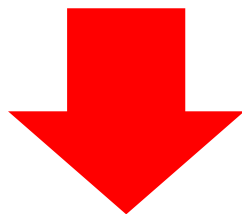


$$\text{文件编码长度} = \sum_{i=1}^n c_i \cdot l_i$$

$$WPL = \sum_{i=1}^n w_i \cdot L_i$$

## 最优编码问题

给定 $n$ 种字符和每种字符出现的次数  
构建一种总编码长度最短的编码方案（最优编码方案）

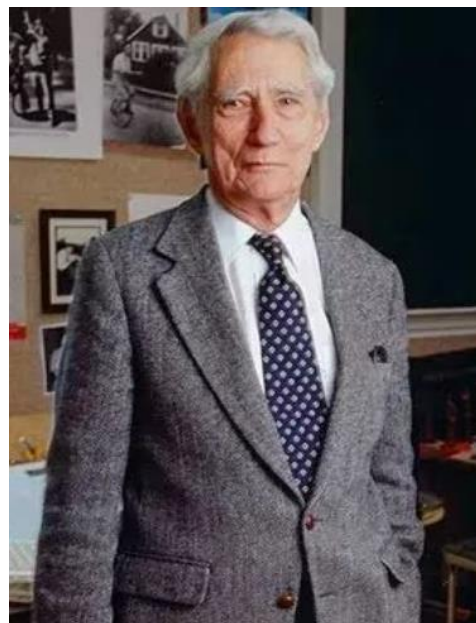


## 构造最优二叉树问题

给定 $n$ 个外结点和每个结点的权值  
构建一棵WPL值最小的扩充二叉树（最优二叉树）



# Shannon-Fano算法



**Claude Shannon**  
**(1916-2001)**

麻省理工学院 教授  
美国科学院院士  
美国工程院院士  
信息论之父



**Robert Fano**  
**(1917-2016)**

麻省理工学院 教授  
美国科学院院士  
美国工程院院士

# Shannon-Fano算法

将字符按频率递减排序

重复做：将字符集切分成两部分，使两部分频率之和尽可能相等

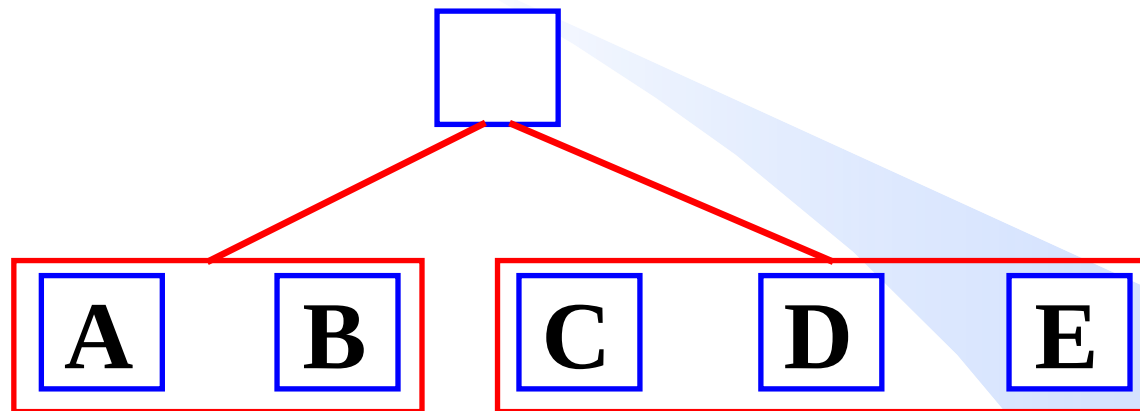
| 字符 | 出现次数 |
|----|------|
| A  | 15   |
| B  | 7    |
| C  | 6    |
| D  | 6    |
| E  | 5    |

A B C D E

# Shannon-Fano算法

重复做：将字符集分成两部分，使两部分频率之和尽可能相等

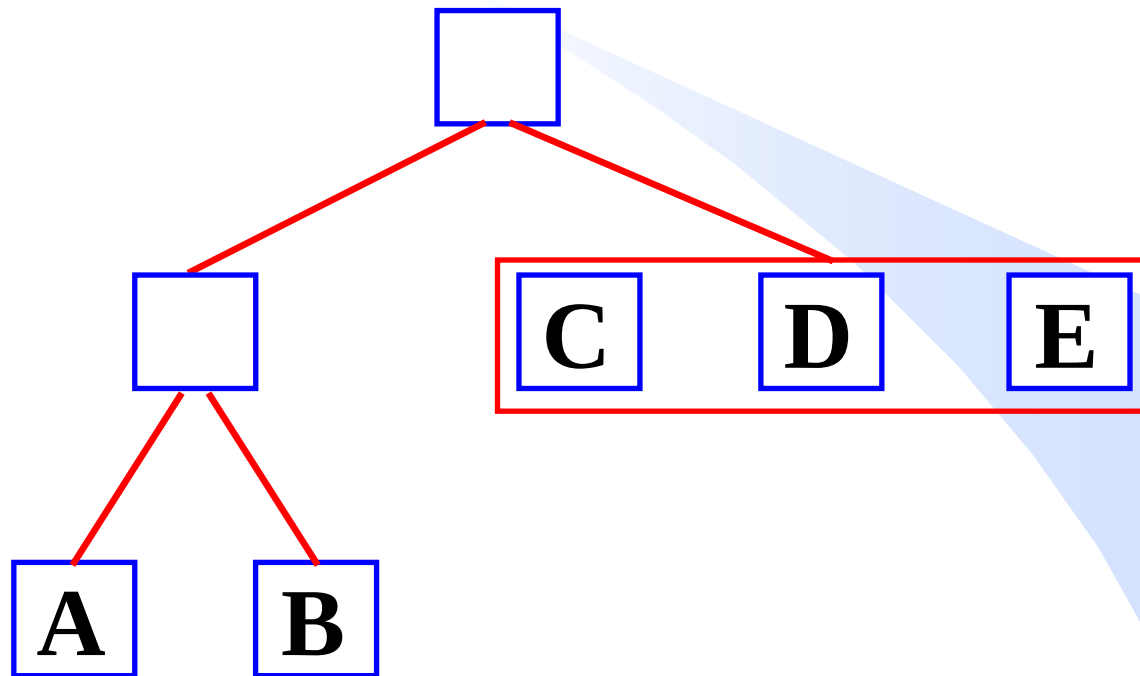
| 字符 | 出现次数 |
|----|------|
| A  | 15   |
| B  | 7    |
| C  | 6    |
| D  | 6    |
| E  | 5    |



# Shannon-Fano算法

重复做：将字符集分成两部分，使两部分频率之和尽可能相等

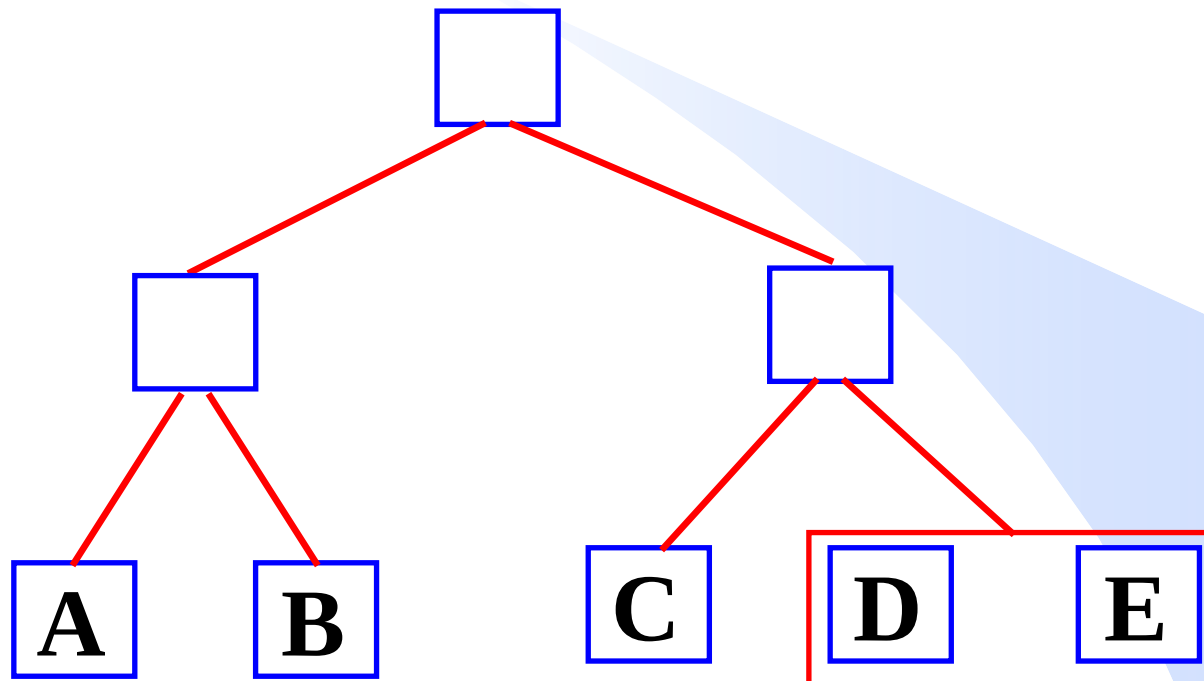
| 字符 | 出现次数 |
|----|------|
| A  | 15   |
| B  | 7    |
| C  | 6    |
| D  | 6    |
| E  | 5    |



# Shannon-Fano算法

重复做：将字符集分成两部分，使两部分频率之和尽可能相等

| 字符 | 出现次数 |
|----|------|
| A  | 15   |
| B  | 7    |
| C  | 6    |
| D  | 6    |
| E  | 5    |



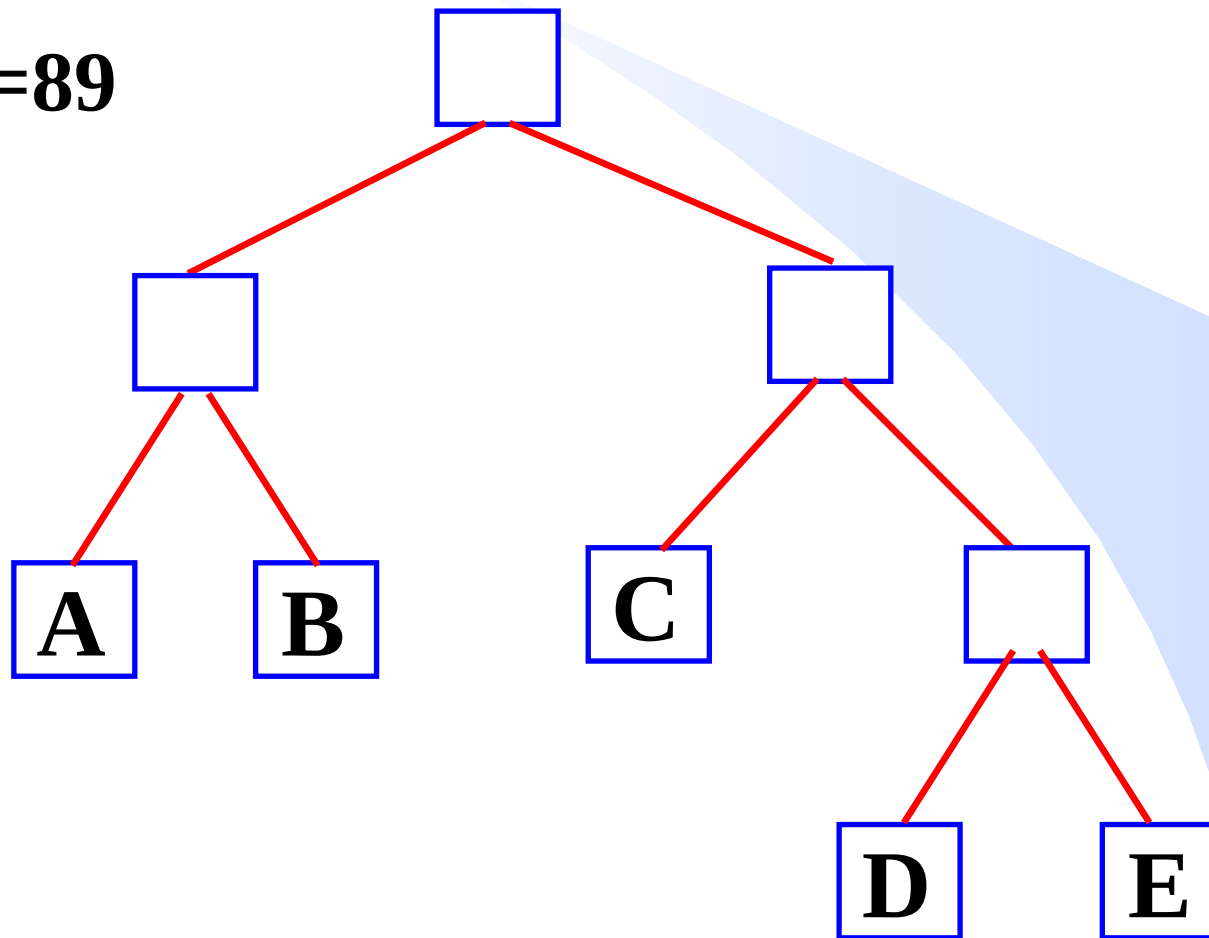


# Shannon-Fano算法

自顶向下建树，并非最优解

$$WPL=(15+7+6)*2+(6+5)*3=89$$

| 字符 | 出现次数 |
|----|------|
| A  | 15   |
| B  | 7    |
| C  | 6    |
| D  | 6    |
| E  | 5    |

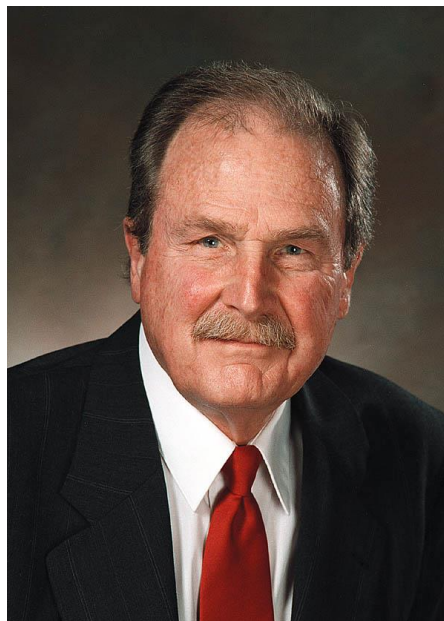


# Huffman算法



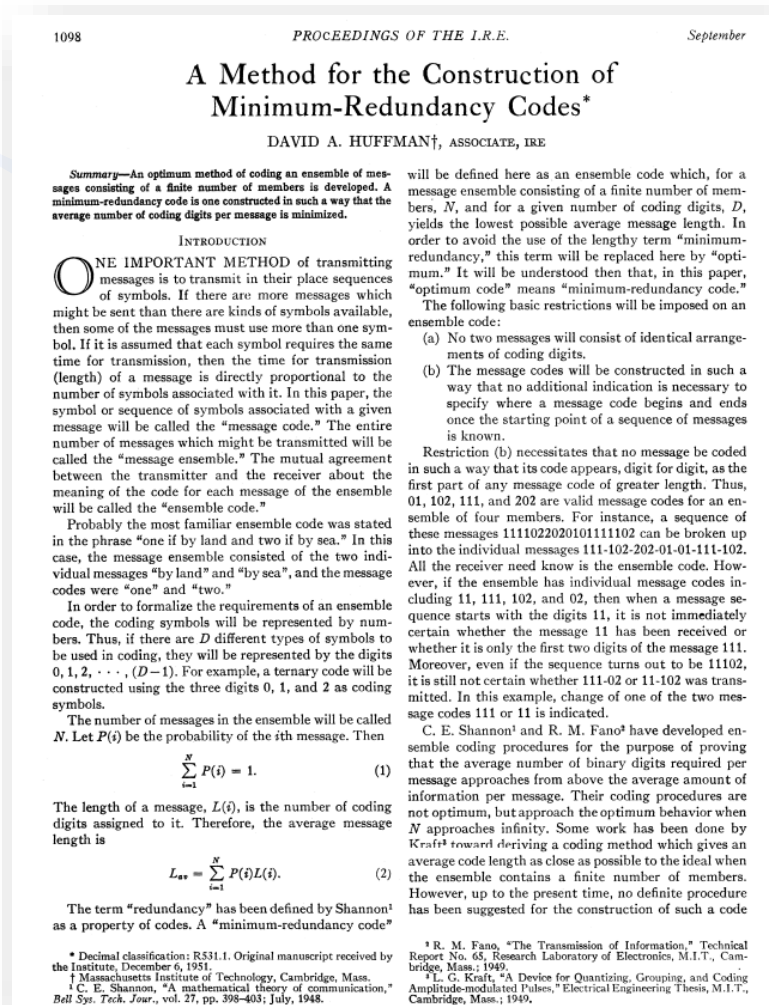
**Robert Fano**  
(1917-2016)

麻省理工学院 教授  
美国科学院院士  
美国工程院院士



**David Huffman**  
(1925-1999)

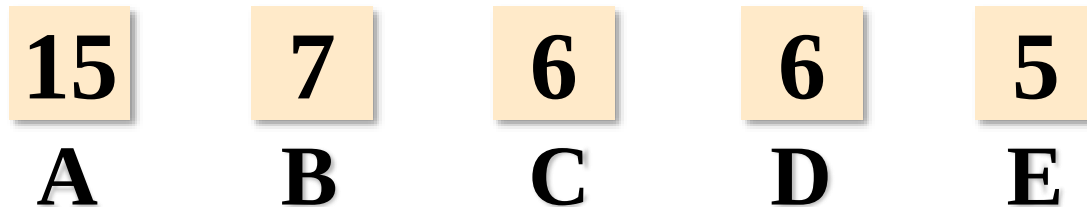
麻省理工学院 博士  
加州大学圣克鲁兹分校 教授



# Huffman算法

重复做：选择权值最小的两个结点生成新结点，新结点作为原结点的父亲，权值是原来两个结点权值之和。

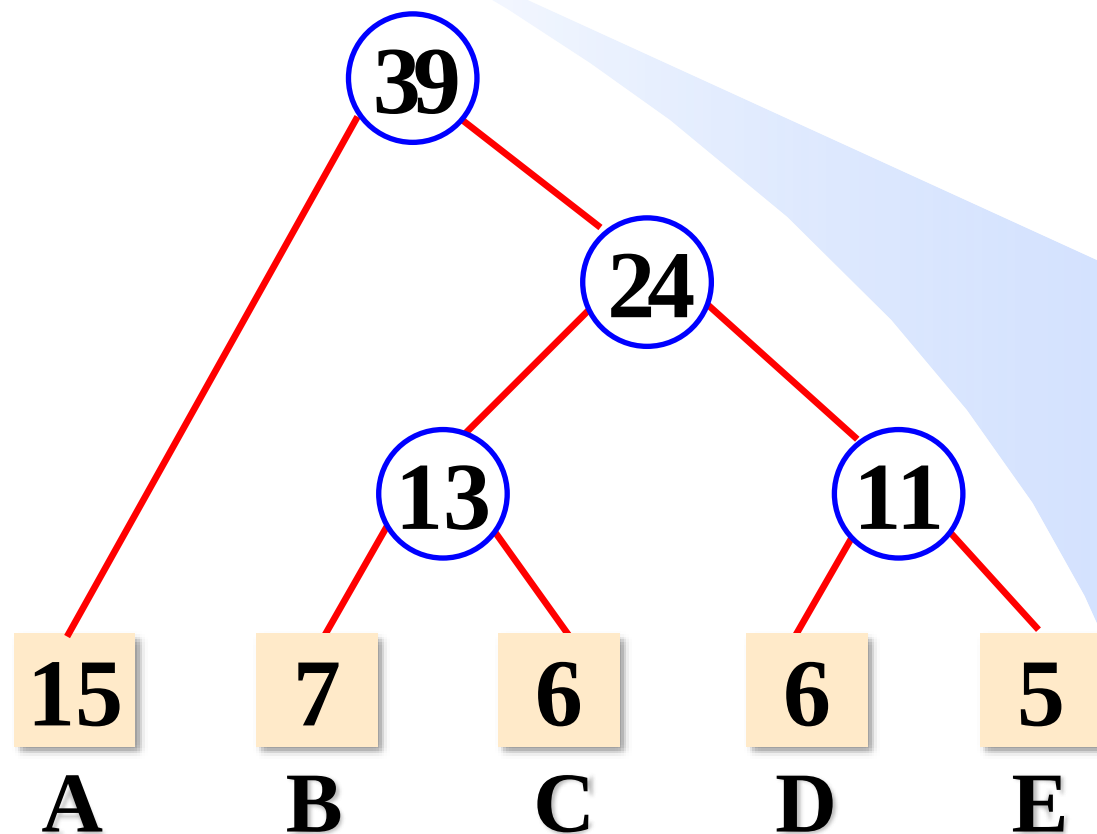
| 字符 | 出现次数 |
|----|------|
| A  | 15   |
| B  | 7    |
| C  | 6    |
| D  | 6    |
| E  | 5    |



# Huffman算法

重复做：选择权值最小的两个结点生成新结点，新结点作为原结点的父亲，权值是原来两个结点权值之和。

| 字符 | 出现次数 |
|----|------|
| A  | 15   |
| B  | 7    |
| C  | 6    |
| D  | 6    |
| E  | 5    |

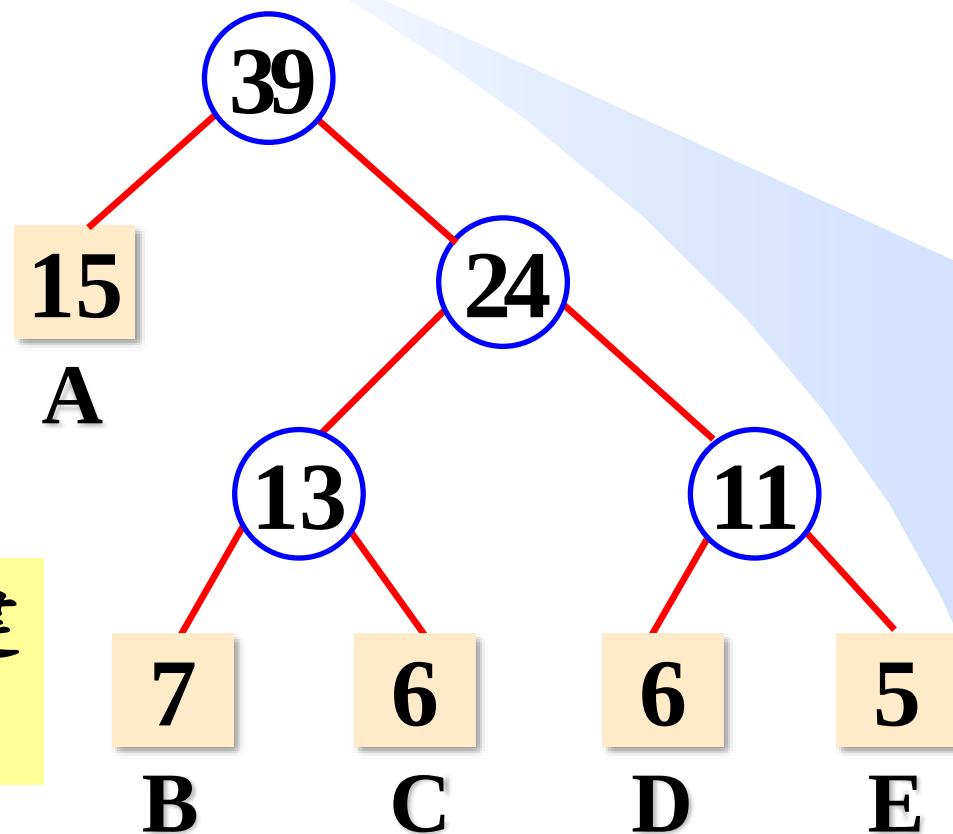


# Huffman算法

重复做：选择权值最小的两个结点生成新结点，新结点作为原结点的父亲，权值是原来两个结点权值之和。

| 字符 | 出现次数 |
|----|------|
| A  | 15   |
| B  | 7    |
| C  | 6    |
| D  | 6    |
| E  | 5    |

自下而上构建  
Huffman树

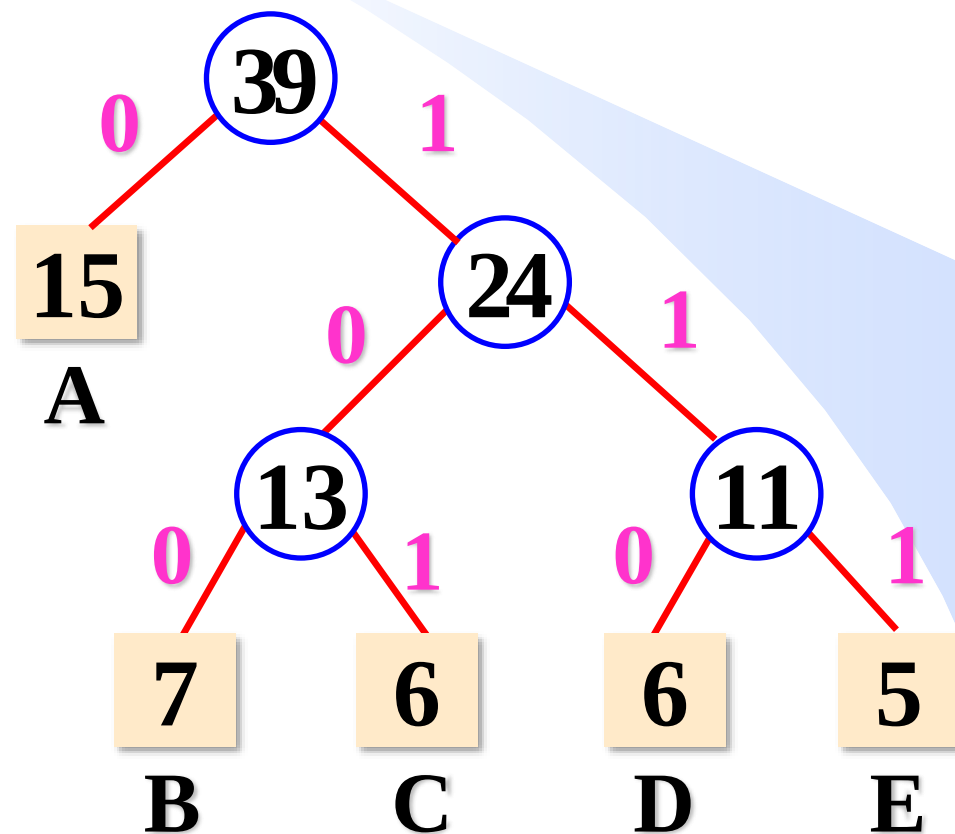


# Huffman编码

每个左分支标记0，右分支标记1。把从根到叶的路径上的标号连接起来，作为该叶结点所代表的字符的编码。

| 字符 | 出现次数 | 编码  |
|----|------|-----|
| A  | 15   | 0   |
| B  | 7    | 100 |
| C  | 6    | 101 |
| D  | 6    | 110 |
| E  | 5    | 111 |

$$WPL=15*1+(7+6+6+5)*3=87$$



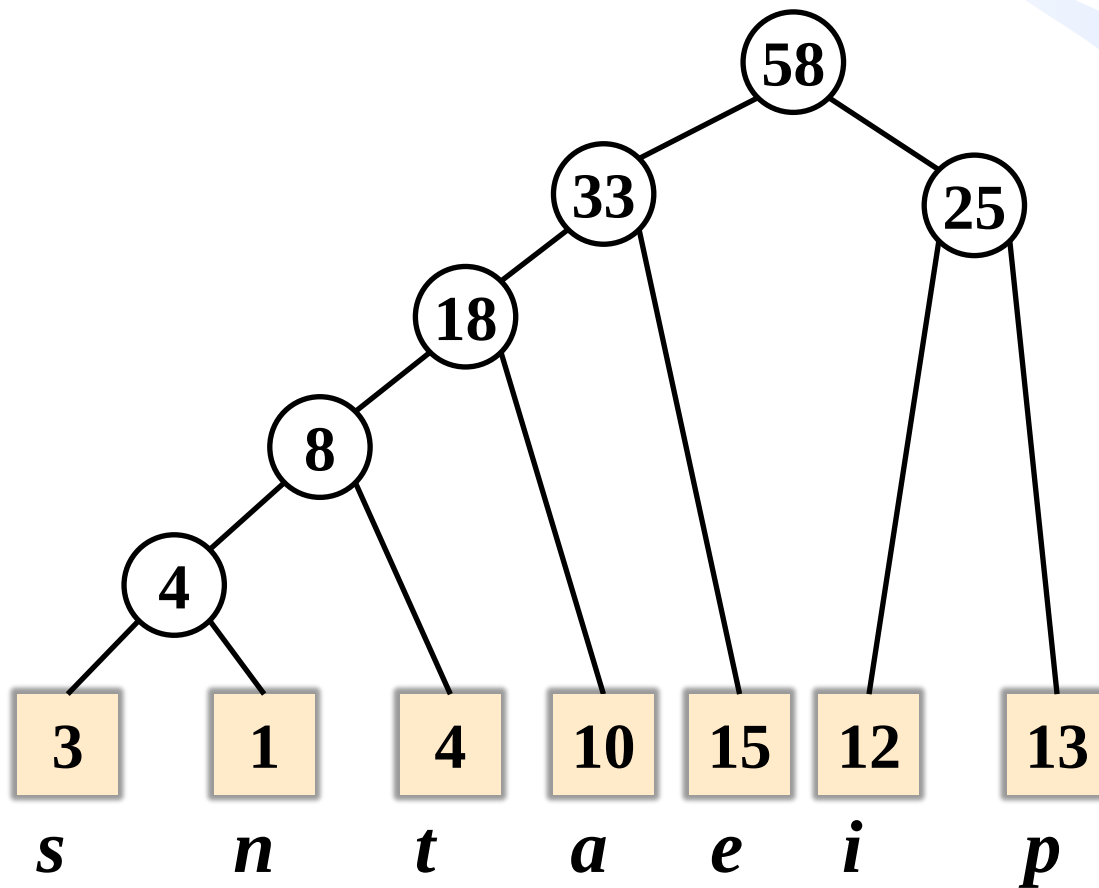


# Huffman算法

- ① 根据给定的 $n$ 个权值 $w_1, w_2, \dots, w_n$ 构成 $n$ 棵二叉树 $T_1, T_2, \dots, T_n$ , 每棵二叉树 $T_i$ 中都只有一个结点, 其权值为 $w_i$ ;
- ② 选出权值最小的两个根结点合并成一棵二叉树: 生成一个新结点作为这两个结点的父结点, 新结点的权值为其两个孩子的权值之和。
- ③ 重复步骤②, 直至只剩一棵二叉树为止, 此二叉树便是哈夫曼树。

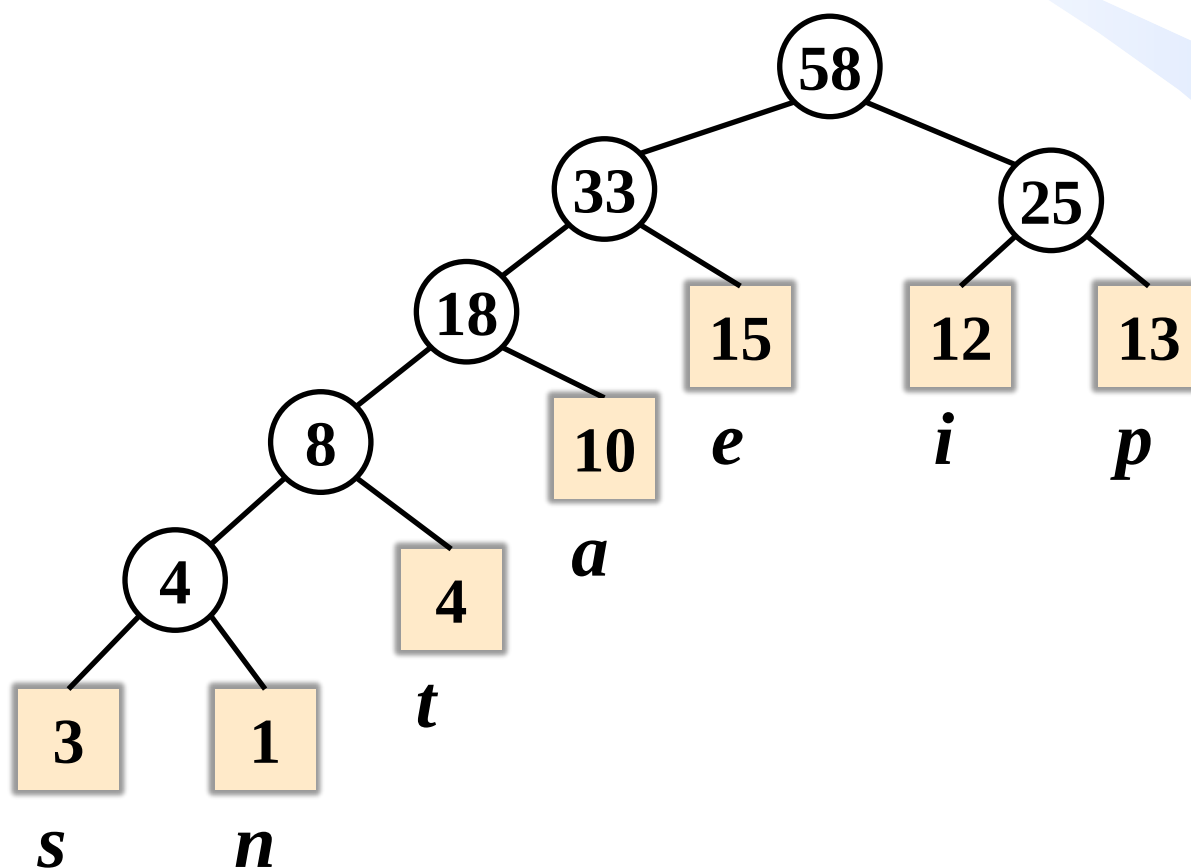
## 构建Huffman树示例

假设有一文件包含7种字符： $a$ 、 $e$ 、 $i$ 、 $s$ 、 $t$ 、 $p$ 、 $n$ ，且文件中有10个 $a$ ，15个 $e$ ，12个 $i$ ，3个 $s$ ，4个 $t$ ，13个 $p$ ，1个 $n$ 。



# 构建Huffman树示例

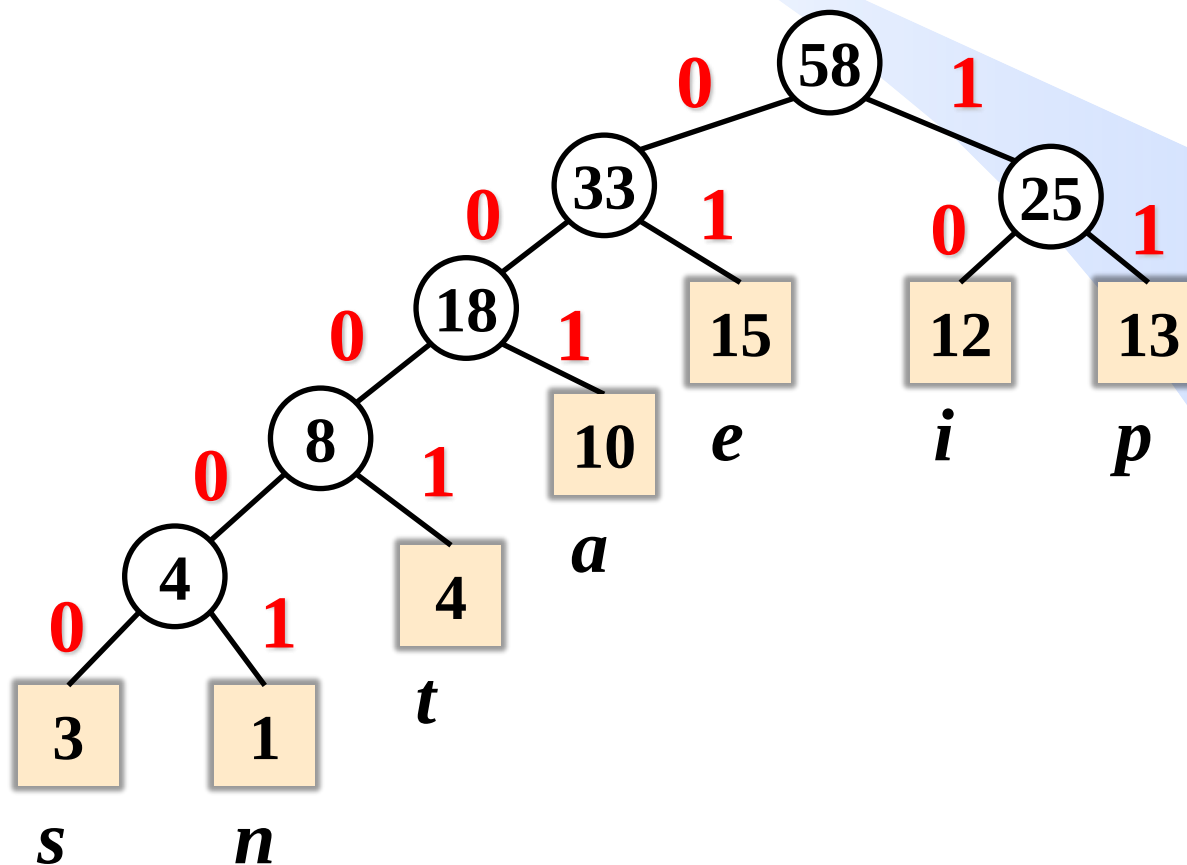
假设有一文件包含7种字符： $a$ 、 $e$ 、 $i$ 、 $s$ 、 $t$ 、 $p$ 、 $n$ ，且文件中有10个 $a$ ，15个 $e$ ，12个 $i$ ，3个 $s$ ，4个 $t$ ，13个 $p$ ，1个 $n$ 。



# Huffman编码

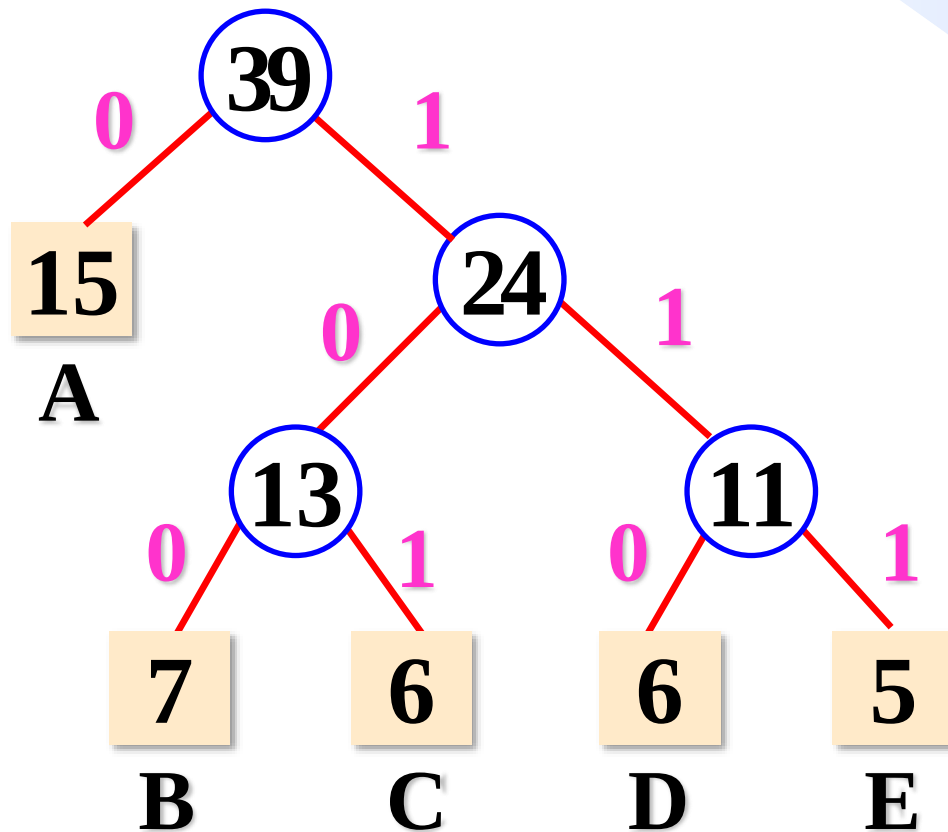
- 对哈夫曼树每个非叶结点的左分支标记0，右分支标记1。
- 把从根到叶的路径上的标号连接起来，作为该叶结点所代表的字符的编码。

|                |       |
|----------------|-------|
| <i>s</i> 的编码是: | 00000 |
| <i>n</i> 的编码是  | 00001 |
| <i>t</i> 的编码是: | 0001  |
| <i>a</i> 的编码是: | 001   |
| <i>e</i> 的编码是: | 01    |
| <i>i</i> 的编码是: | 10    |
| <i>p</i> 的编码是: | 11    |



# 哈夫曼编码是否是“无前缀冲突编码”？

字符对应叶结点，任意一个叶结点不可能是其他叶结点的祖先，每个叶结点对应的编码不可能是其他叶结点对应的编码的前缀，故哈夫曼编码是无前缀冲突编码。



## 课下思考

字符串"*alibaba*"的Huffman编码总长度有\_\_\_\_位 (bit) 。

【阿里笔试题】

A. 11

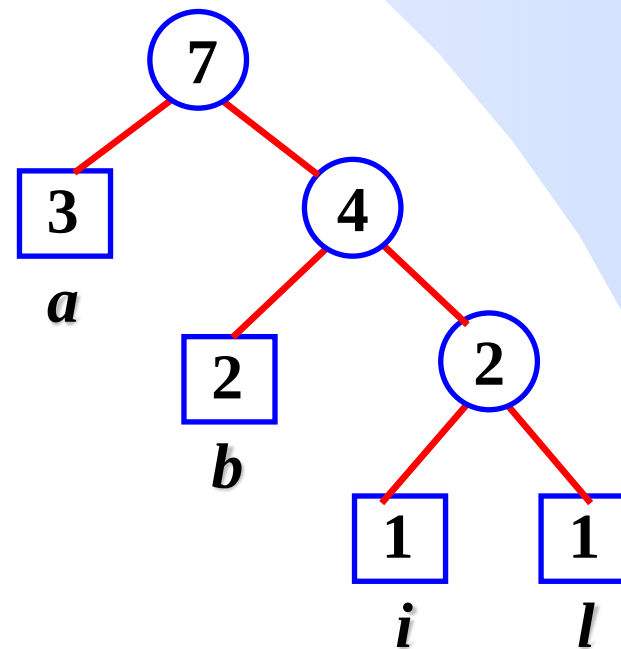
B. 12

C. 13

D. 14

文本中有3个*a*, 2个*b*, 1个*i*, 1个*l*

WPL=13





## 课下思考

在有6个字符组成的字符集S中，各字符出现的频次分别为3、4、5、6、8、10，为S构造的哈夫曼树的加权路径长度WPL为\_\_\_\_\_。【2023年考研题全国卷】

A. 86

B. 90

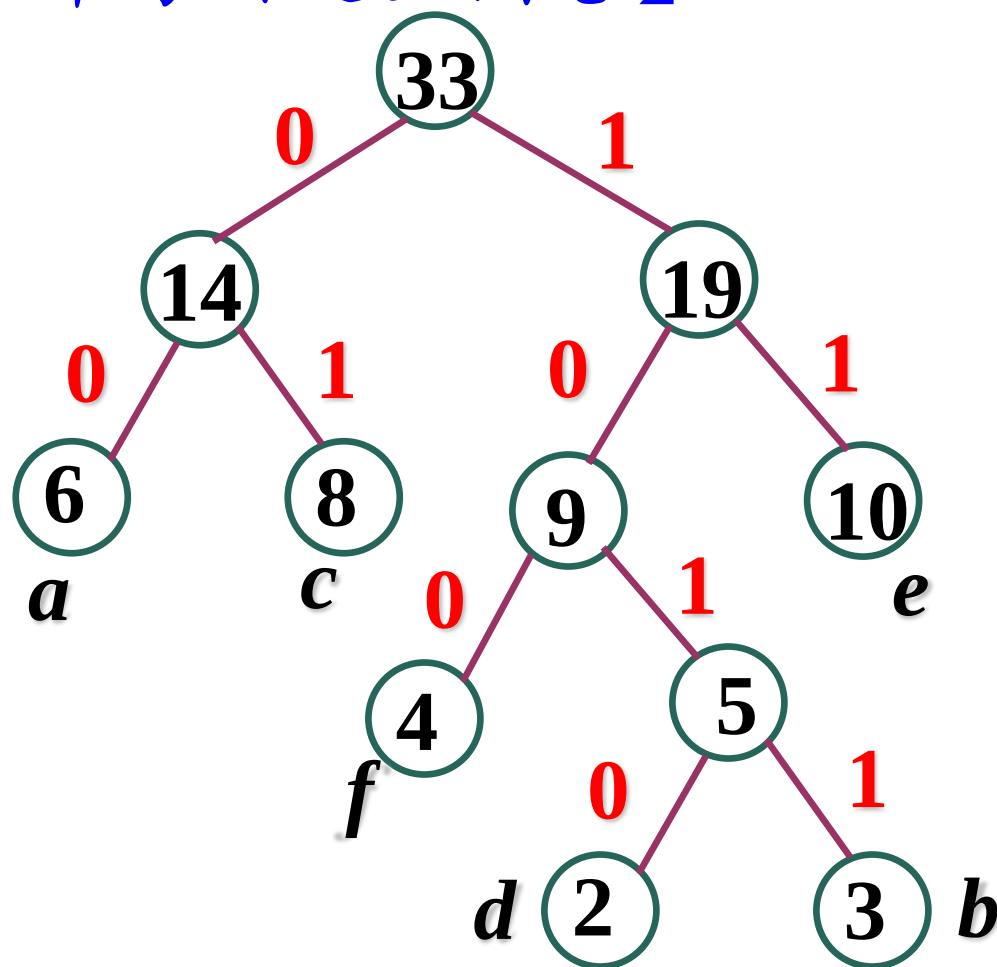
C. 96

D. 99

## 课下思考

已知字符集合是 { a, b, c, d, e, f }, 各个字符出现的次数分别是 6, 3, 8, 2, 10, 4, 则各字符对应的哈夫曼编码为\_\_\_\_\_

【2018年考研题全国卷】



编码

*a* (00)

*b* (1011)

*c* (01)

*d* (1010)

*e* (11)

*f* (100)

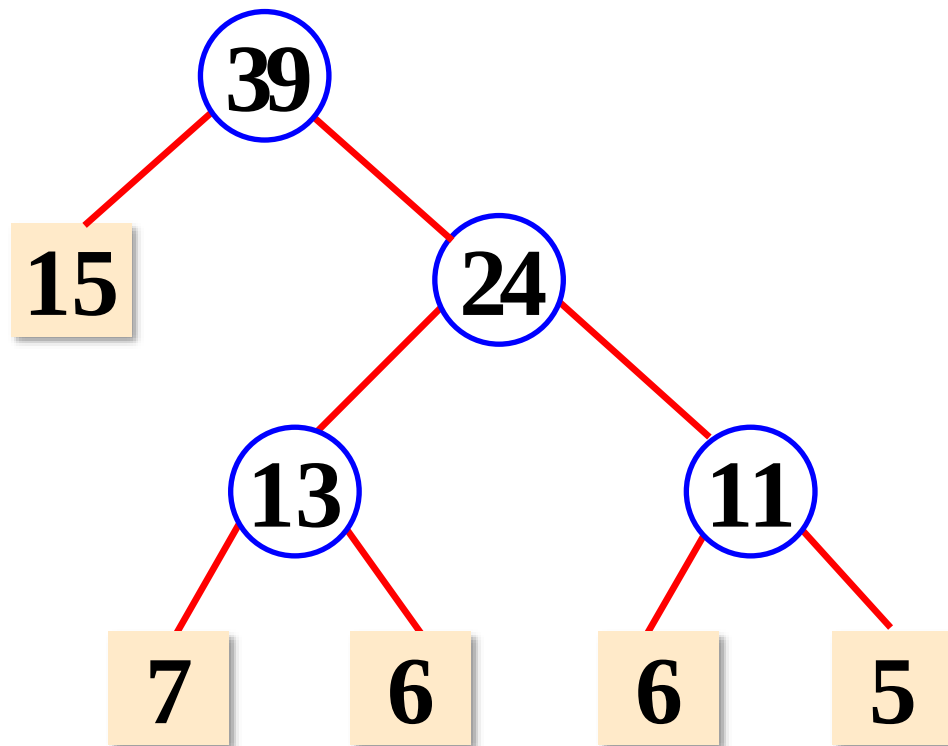
# 哈夫曼树——二子性

➤ 哈夫曼树中包含度为1的结点么？

✓ 哈夫曼树不包含度为1的结点。

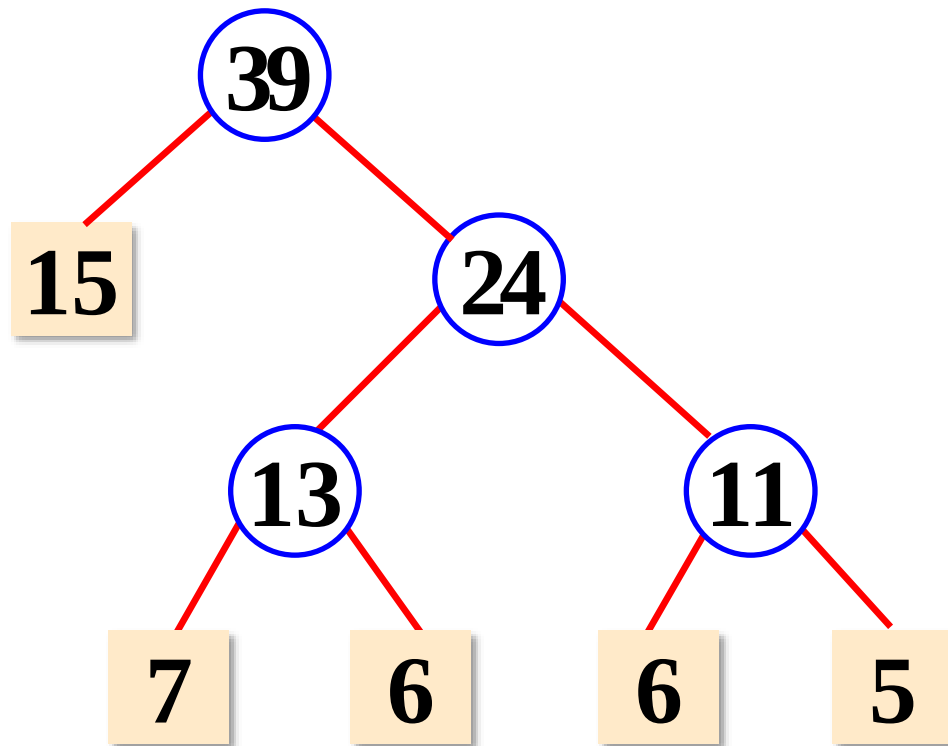
✓ 若哈夫曼树 $n$ 个叶结点，则必有 $n-1$ 个非叶结点，一共 $2n-1$ 个结点。

引理  $n_0 = n_2 + 1$ .



# 哈夫曼树——同权不同构

- 哈夫曼树、哈夫曼编码、最小编码长度唯一么？
  - ✓ 哈夫曼树形态不唯一、编码不唯一。
  - ✓ 对任意内结点而言，其左右子树互换后WPL（加权路径长度）不变，故最小编码长度唯一。



# Huffman算法实现

哈夫曼树中每个结点的结构为：

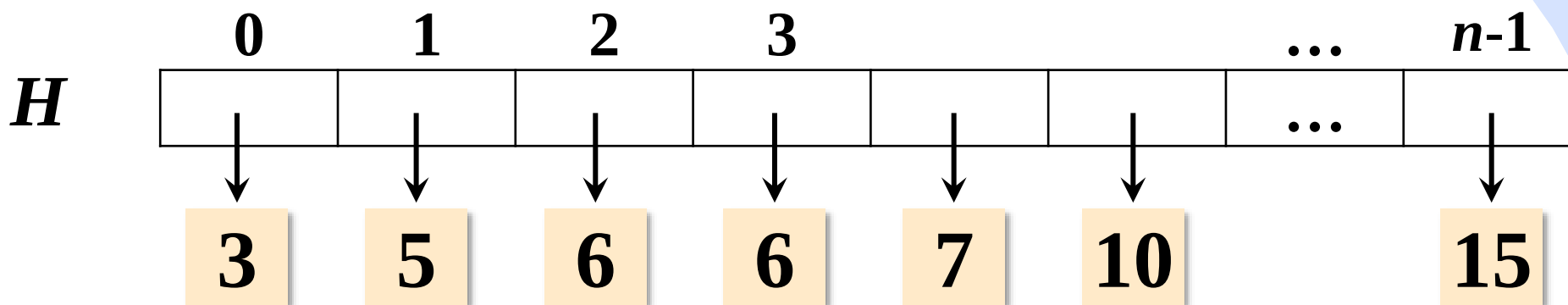
| <i>left</i> | <i>data</i> | <i>weight</i> | <i>right</i> |
|-------------|-------------|---------------|--------------|
|-------------|-------------|---------------|--------------|

其中 *left* 和 *right* 为链接域，*data* 为字符值，*weight* 为该结点的权值。

```
struct HuffmanNode{
    char data=' ';
    int weight;
    HuffmanNode *left,*right;
};
```

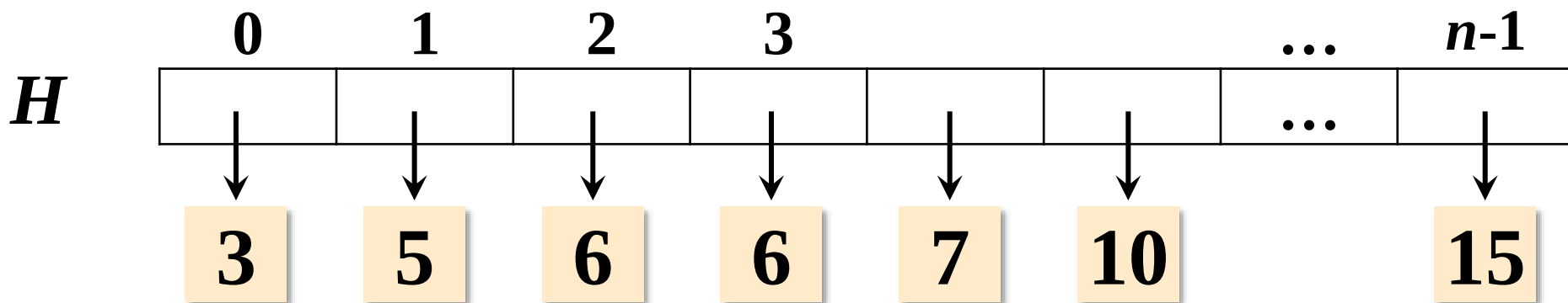
将哈夫曼树结点的指针存于一维数组  $H[ ]$  中

```
const int maxn=300;
HuffmanNode *H[maxn];
```



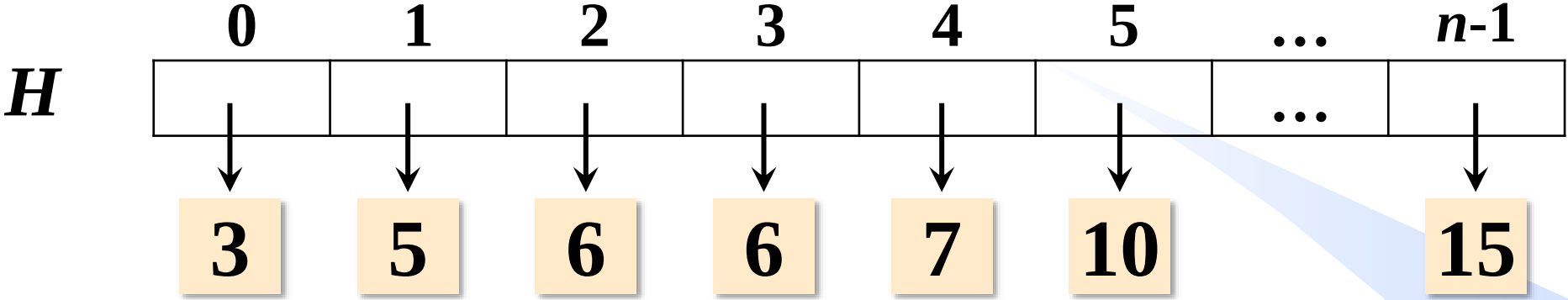
# Huffman算法实现

```
const int maxn=300; HuffmanNode *H[maxn];
void CreateHuffmanNode(char ch[], int freq[], int n){
    //有n种字符，第i种字符存放在ch[i]，其频率存放在freq[i]中
    for(int i = 0; i < n; i++) {
        H[i] = new HuffmanNode;
        H[i]->data = ch[i];  H[i]->weight = freq[i];
        H[i]->left = H[i]->right = NULL;
    }
    sort(H, n); //对H按weight值递增排序，sort函数需预先实现
}
```

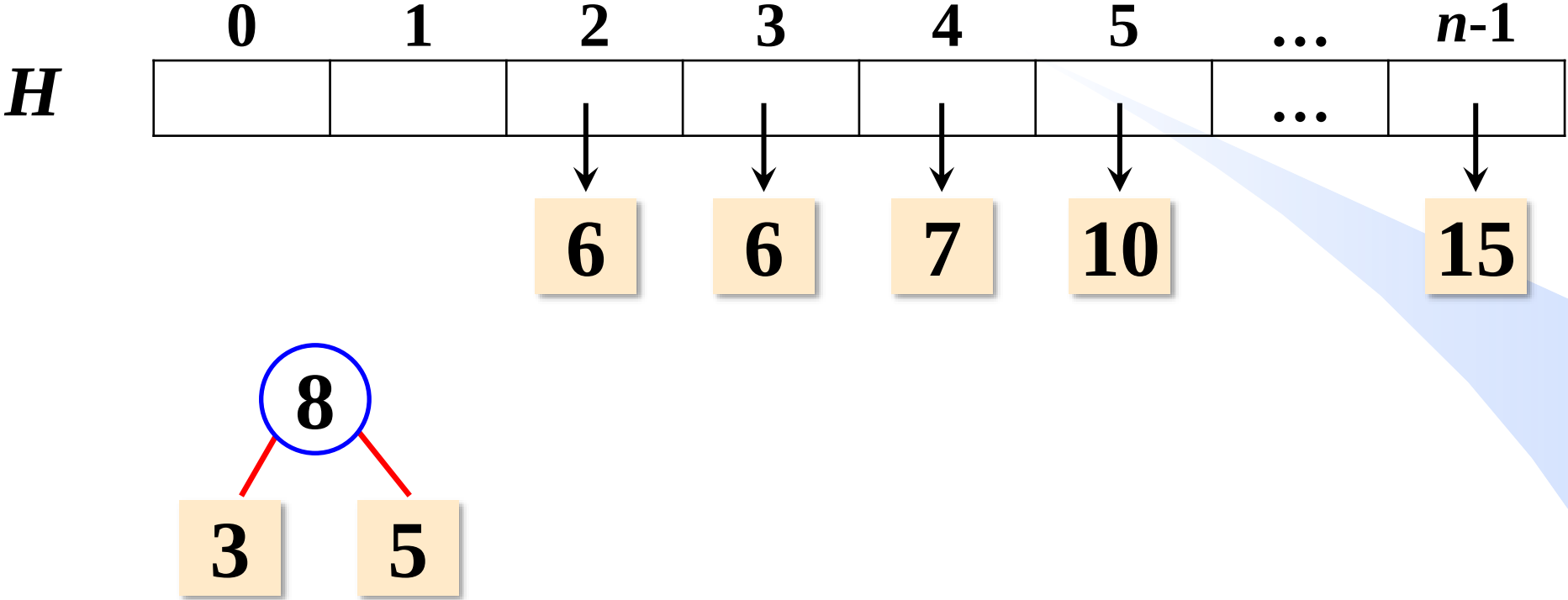




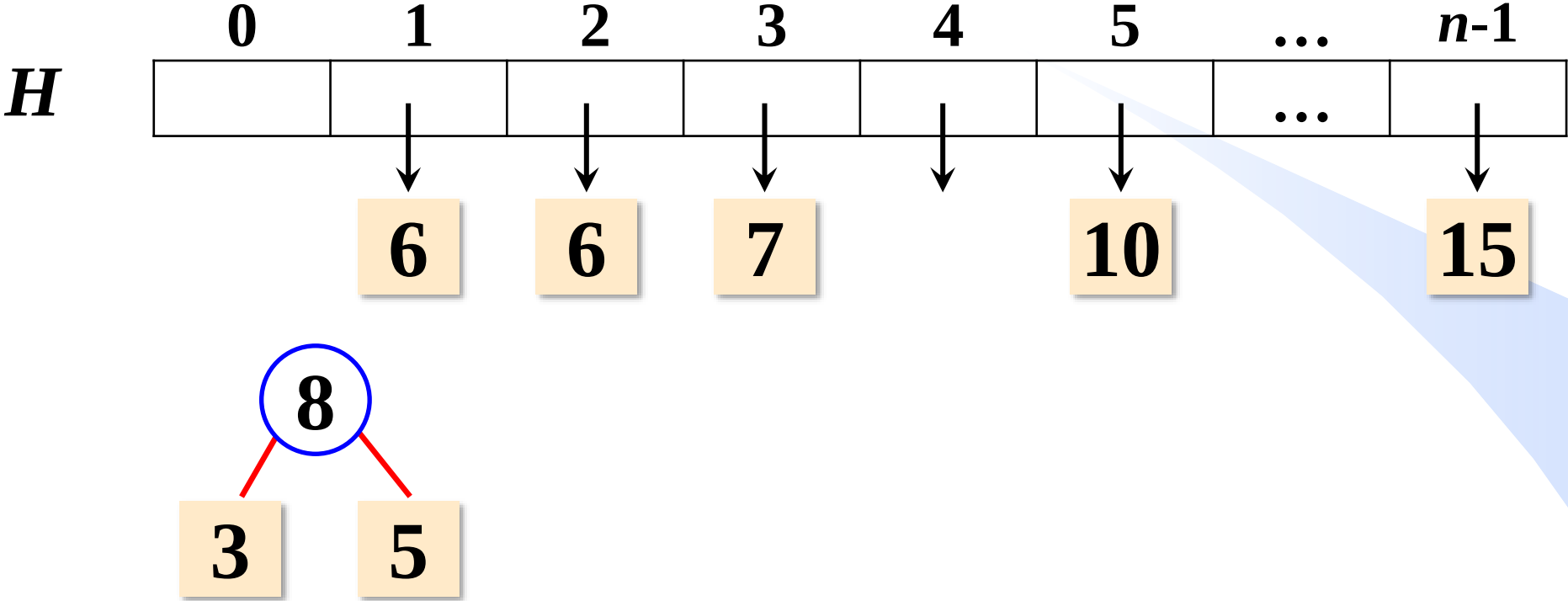
# Huffman算法实现



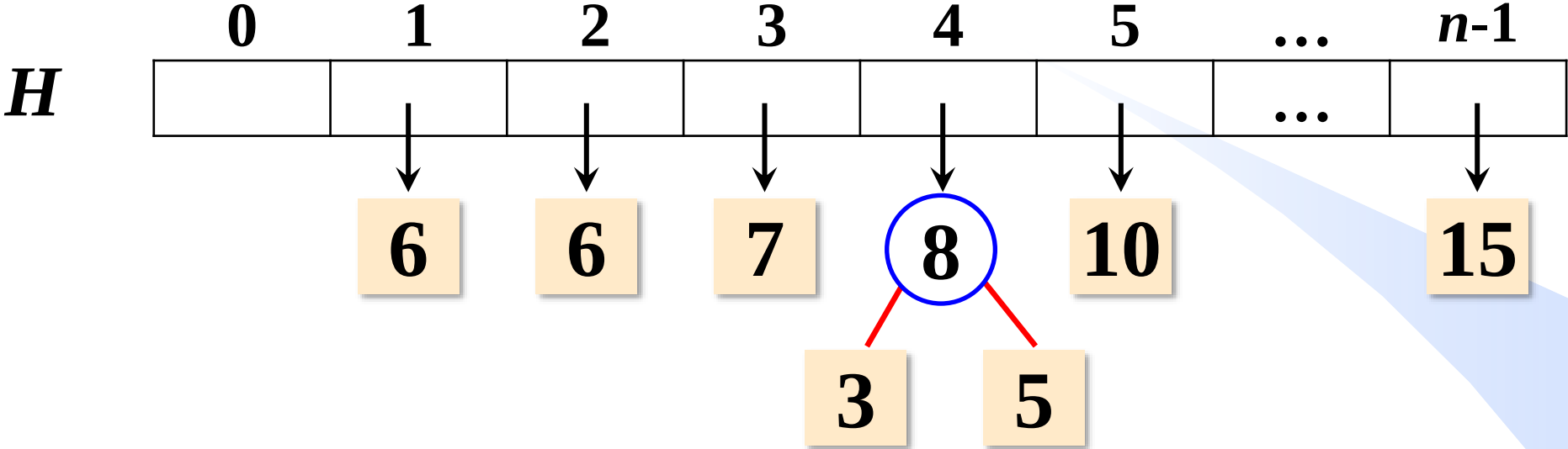
# Huffman算法实现



# Huffman算法实现

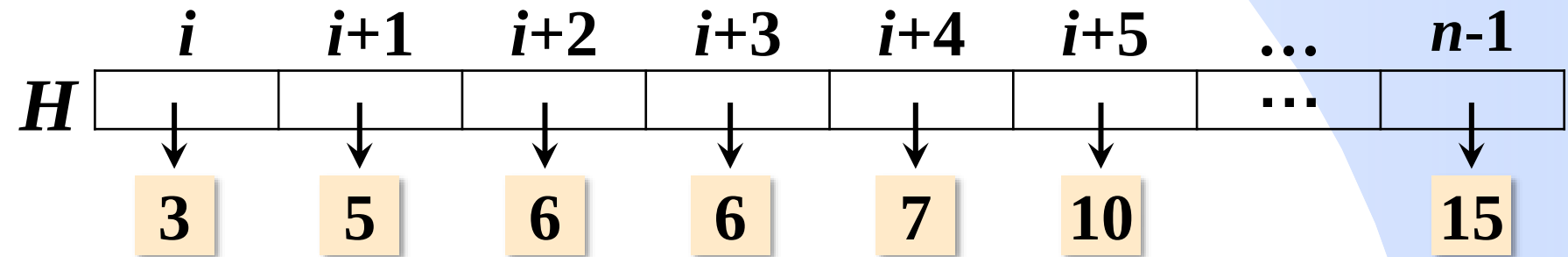


# Huffman算法实现



```
HuffmanNode* BuildHuffmanTree(HuffmanNode *H[], int n){  
    for(int i=0; i<n-1; i++){  
        //未完待续
```

**B**

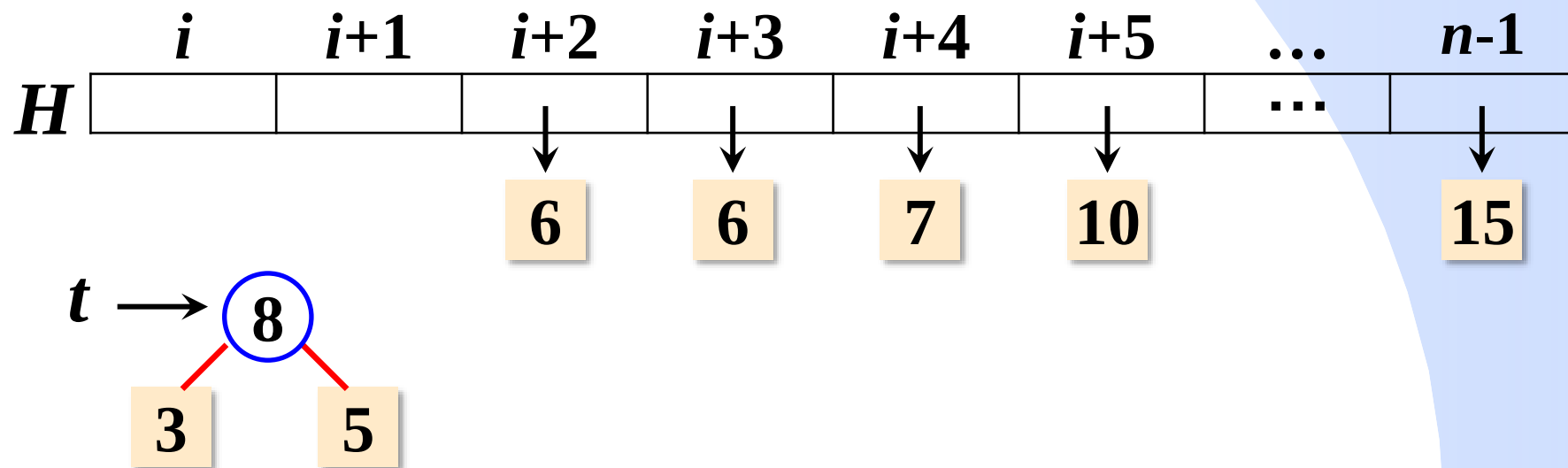


```

HuffmanNode* BuildHuffmanTree(HuffmanNode *H[], int n){
    for(int i=0; i<n-1; i++){
        HuffmanNode *t = new HuffmanNode;
        t->left=H[i];  t->right=H[i+1];
        t->weight = t->left->weight + t->right->weight;
        //把新结点t插入到数组H中，使得数组保持按权值升序排列
        int j = i+2;
        while(j < n && H[j]->weight < t->weight){
            H[j-1]=H[j];  j++;
        }
    }
}

```

//未完待续

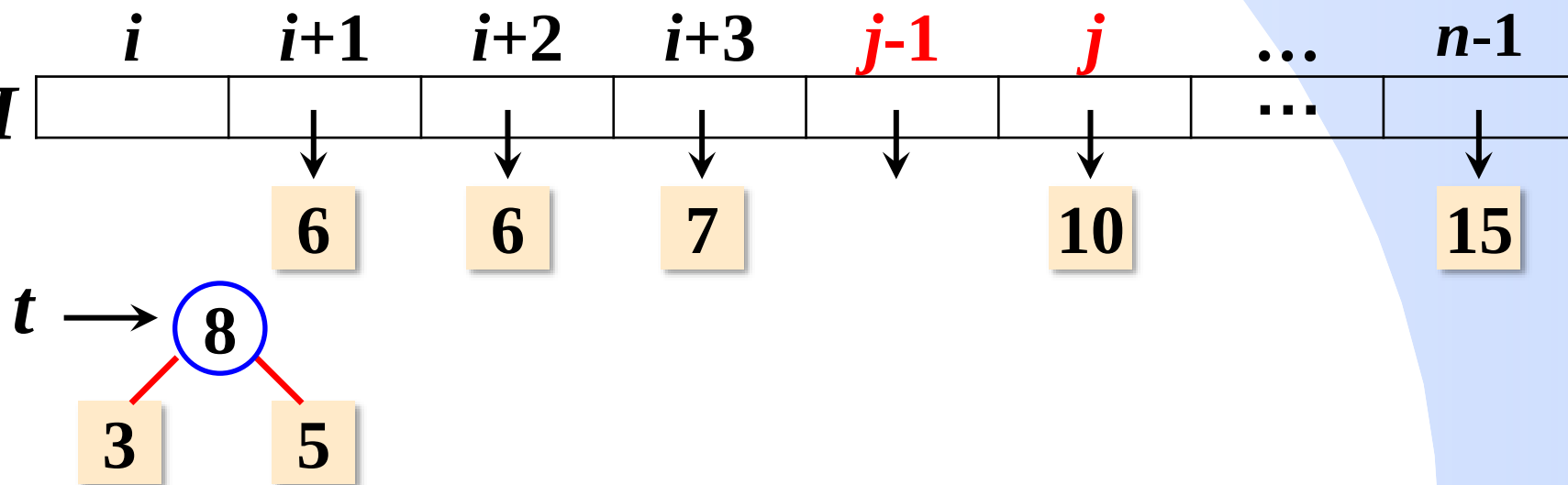


```

HuffmanNode* BuildHuffmanTree(HuffmanNode *H[], int n){
    for(int i=0; i<n-1; i++){
        HuffmanNode *t = new HuffmanNode;
        t->left=H[i];  t->right=H[i+1];
        t->weight = t->left->weight + t->right->weight;
        //把新结点t插入到数组H中，使得数组保持按权值升序排列
        int j = i+2;
        while(j < n && H[j]->weight < t->weight){
            H[j-1]=H[j];  j++;
        }
        H[j-1] = t;
    }
}

```

//未完待续

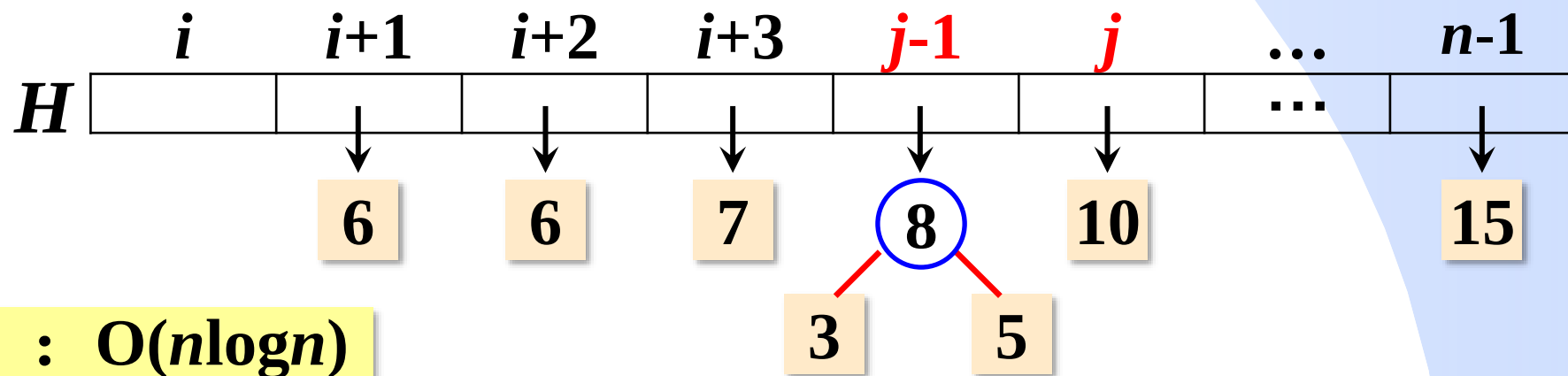




```

HuffmanNode* BuildHuffmanTree(HuffmanNode *H[], int n){
    for(int i=0; i<n-1; i++){
        HuffmanNode *t = new HuffmanNode;
        t->left=H[i];  t->right=H[i+1];
        t->weight = t->left->weight + t->right->weight;
        //把新结点t插入到数组H中，使得数组保持按权值升序排列
        int j = i+2;
        while(j < n && H[j]->weight < t->weight){
            H[j-1]=H[j];  j++;
        }
        H[j-1] = t;
    }
    return H[n-1];
}

```



- ✓ 预处理（排序）：  $O(n \log n)$
- ✓ 构建哈夫曼树：  $O(n^2)$
- ✓ 总时间复杂度  $O(n^2)$

## 一种改进方案（队列）

- 初始时外结点按权值递增顺序入“队列1”。
- 每次从队头附近（最多4个结点）选出最小的两个结点。
- 生成的中间结点按递增入“队列2”。

37 19 16 13 5 2

37

13

16

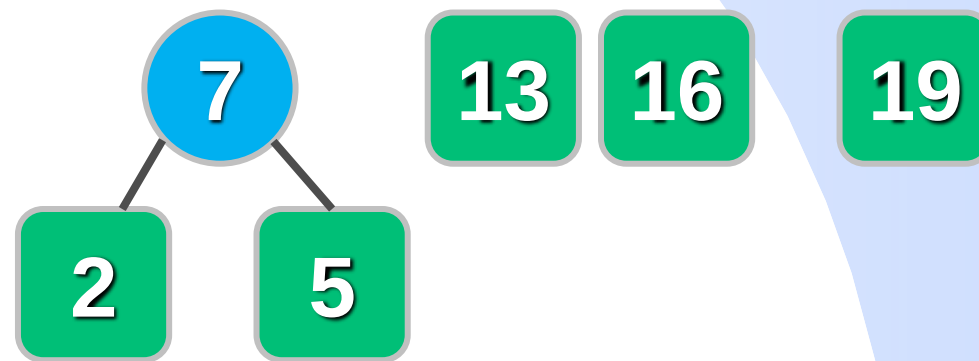
19

2

5

## 一种改进方案（队列）

- 初始时外结点按权值递增顺序入“队列1”。
- 每次从队头附近（最多4个结点）选出最小的两个结点。
- 生成的中间结点按递增入“队列2”。



37

19

16

13

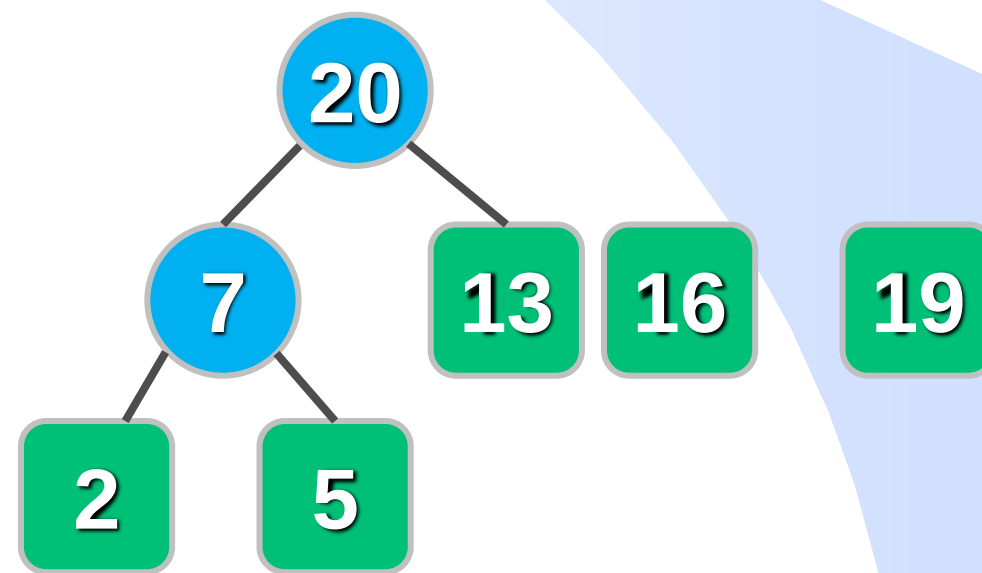
7

5

2

## 一种改进方案（队列）

- 初始时外结点按权值递增顺序入“队列1”。
- 每次从队头附近（最多4个结点）选出最小的两个结点。
- 生成的中间结点按递增入“队列2”。



## 一种改进方案（队列）

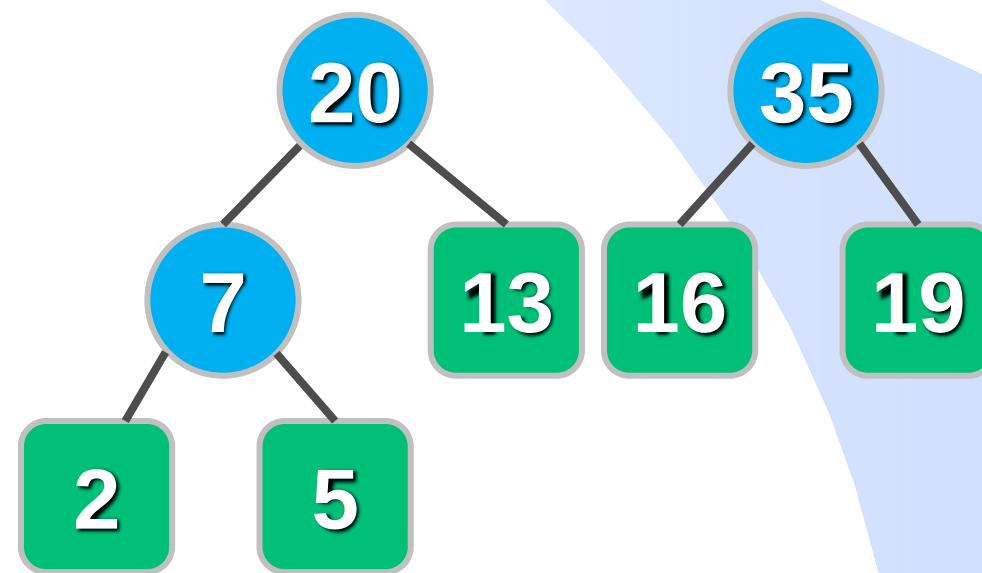
- 初始时外结点按权值递增顺序入“队列1”。
- 每次从队头附近（最多4个结点）选出最小的两个结点。
- 生成的中间结点按递增入“队列2”。

37

35

20

37



## 一种改进方案（队列）

- 初始时外结点按权值递增顺序入“队列1”。
- 每次从队头附近（最多4个结点）选出最小的两个结点。
- 生成的中间结点按递增入“队列2”。

---

---

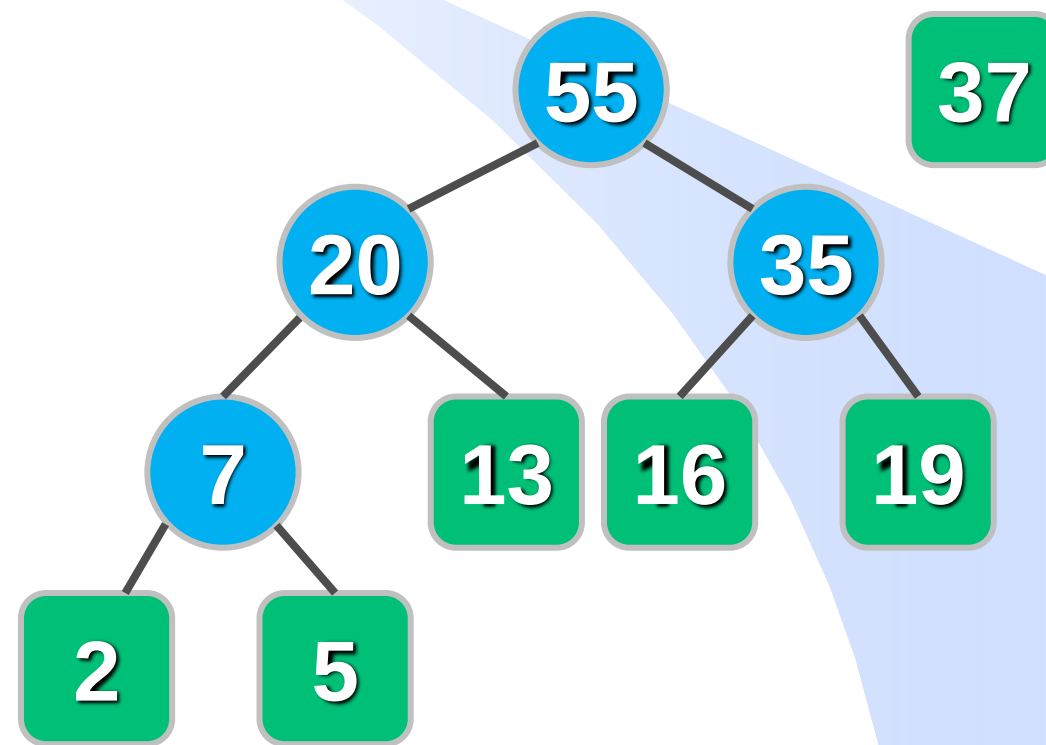
37

---

---

55

---

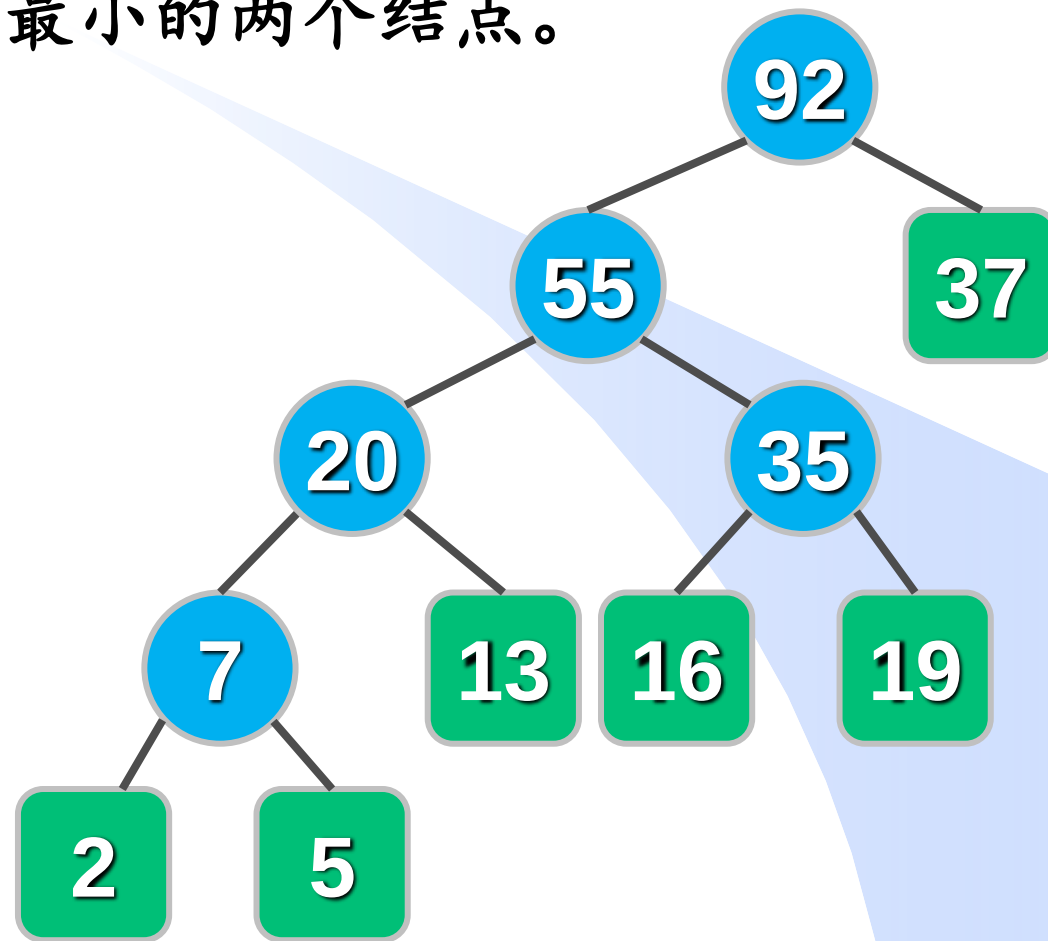


## 一种改进方案（队列）

- 初始时外结点按权值递增顺序入“队列1”。
- 每次从队头附近（最多4个结点）选出最小的两个结点。
- 生成的中间结点按递增入“队列2”。

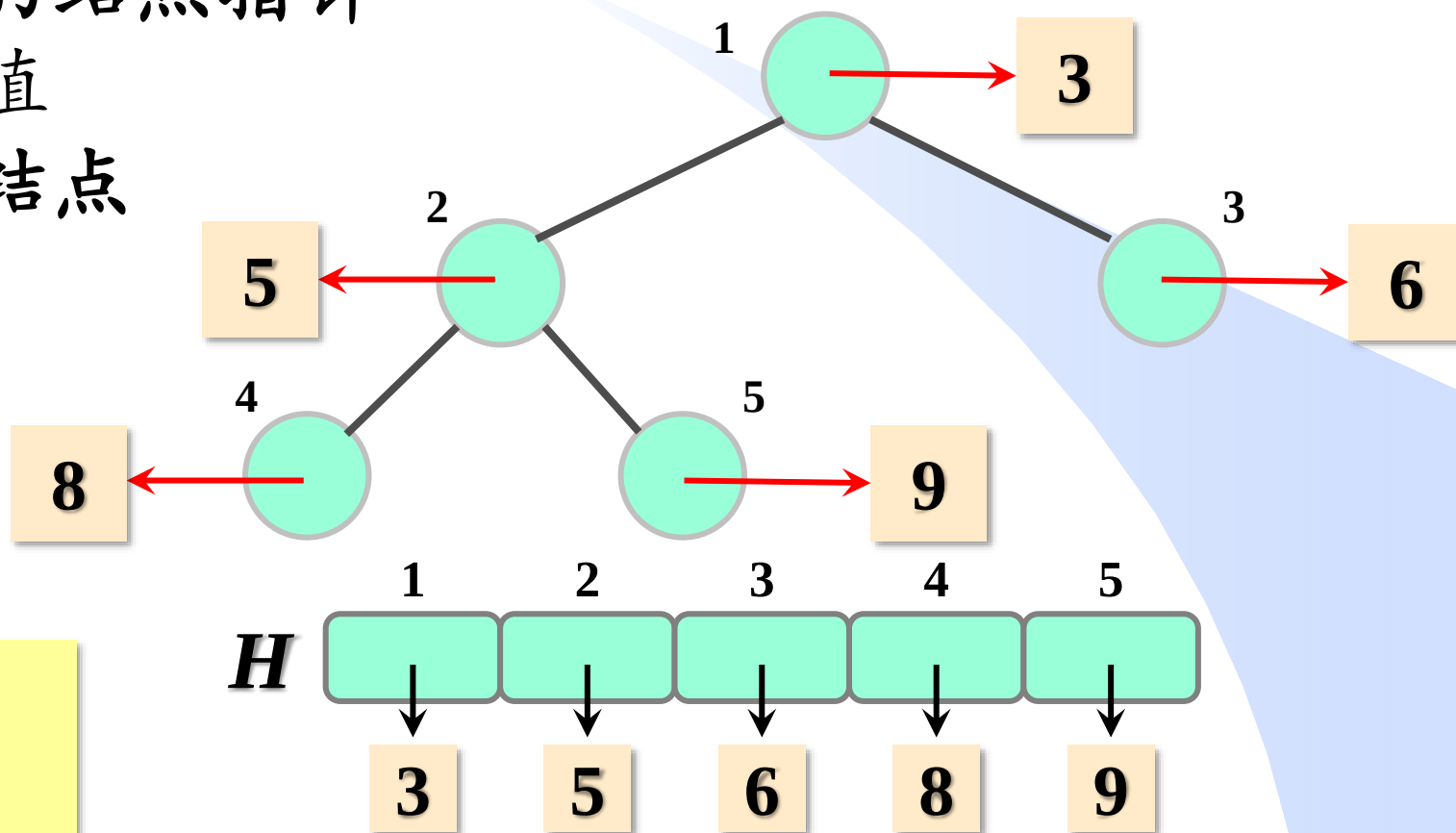
92

- ✓ 预处理（排序）： $O(n\log n)$
- ✓ 构建哈夫曼树： $O(n)$
- ✓ 总时间复杂度 $O(n\log n)$



## 另一种改进方案（优先级队列）

- 使用“最小堆”实现优先级队列
- 堆的元素：哈夫曼树的结点指针
- 堆元素的关键词：权值
- 用堆选取权值最小的结点
- 将在第7章中介绍



- ✓ 预处理（建堆）： $O(n)$
- ✓ 构建哈夫曼树： $O(n \log n)$
- ✓ 总时间复杂度  $O(n \log n)$

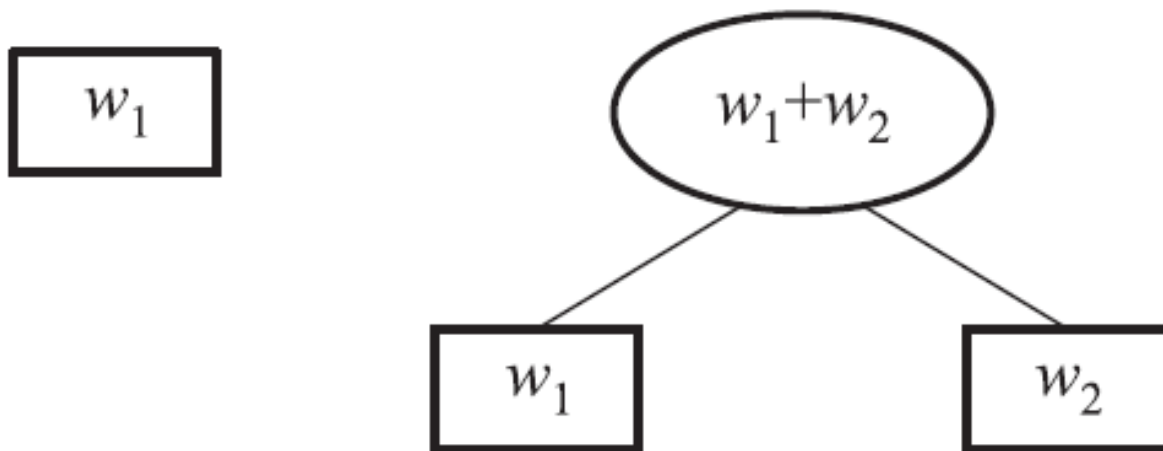


# Huffman算法的正确性

**定理** 外结点权值 $w_1 \leq w_2 \leq \dots \leq w_n$ 的扩充二叉树中，由Huffman算法构造出的是一棵最优二叉树。

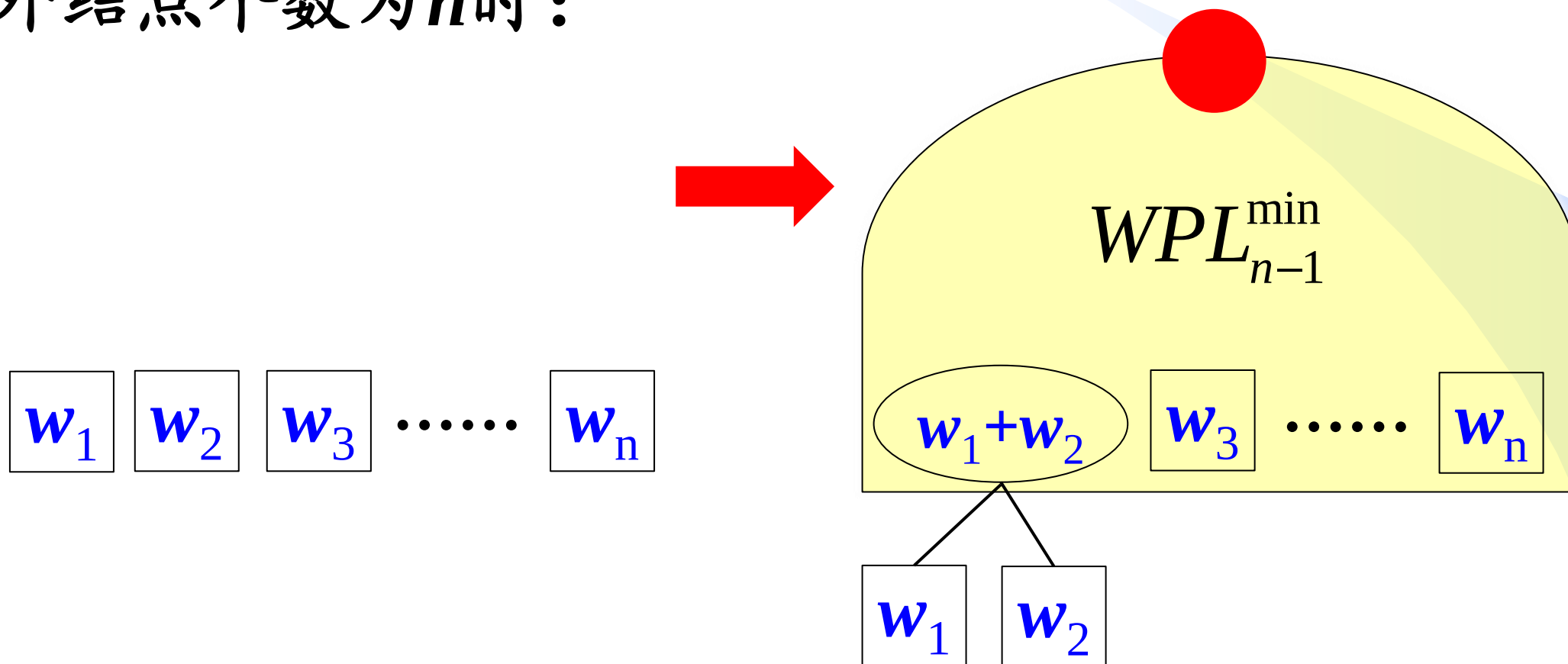
证明：数学归纳法。

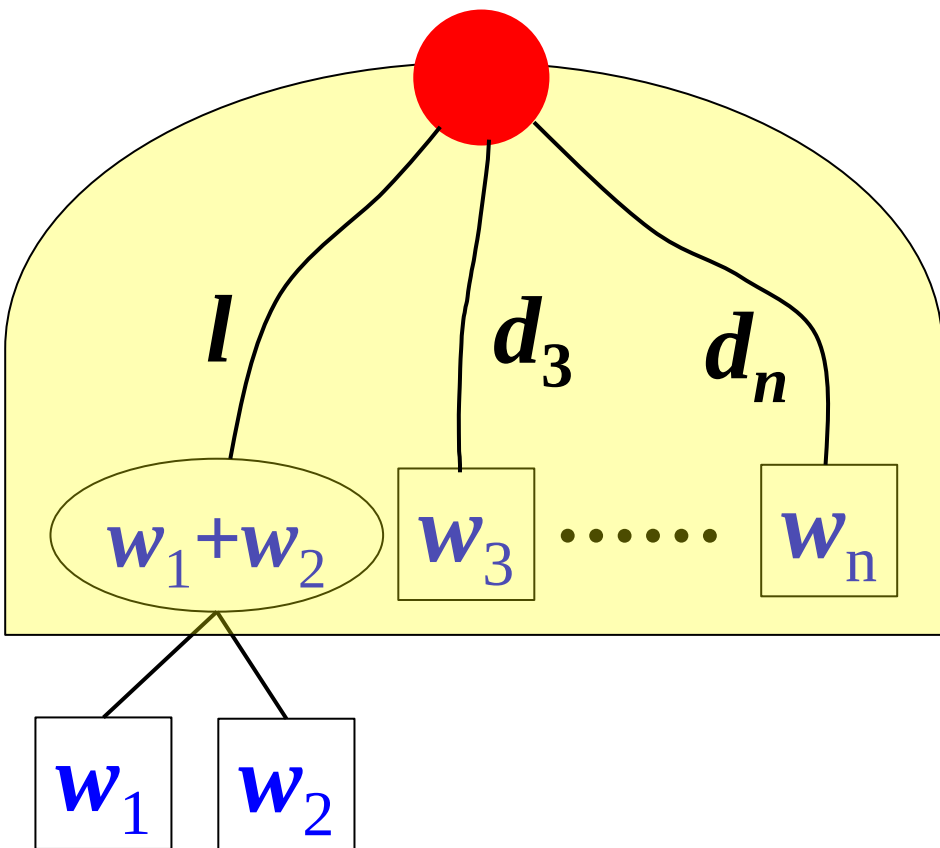
$n=1, 2$ 时



# Huffman算法的正确性

- 设外结点个数为 $n-1$ 时算法正确。
- 外结点个数为 $n$ 时:





$$WPL_n$$

$$= w_1(l+1) + w_2(l+1) + w_3d_3 + \dots + w_nd_n$$

$$= (w_1 + w_2)(l+1) + w_3d_3 + \dots + w_nd_n$$

$$= (w_1 + w_2)l + w_1 + w_2 + w_3d_3 + \dots + w_nd_n$$

$$= WPL_{n-1}^{\min} + w_1 + w_2$$

$$WPL_n^{\min} = WPL_{n-1}^{\min} + w_1 + w_2$$

构造哈夫曼树时如何求WPL?  
每次把两个最小的权值累加起来

```
int WPL=0;
```

```
HuffmanNode* BuildHuffmanTree(HuffmanNode *H[], int n){  
    for(int i=0; i<n-1; i++){  
        HuffmanNode* t = new HuffmanNode;  
        t->left=H[i];  t->right=H[i+1];  
        t->weight = t->left->weight + t->right->weight;  
        WPL += t->weight; //把每次找到的两个最小权值累加起来  
        int j=i+2; //将t插入数组H中，使数组保持按权值升序排列  
        while(j<n && H[j]->weight < t->weight){  
            H[j-1]=H[j];  j++;  
        }  
        H[j-1] = t;  
    }  
    return H[n-1];  
}
```

哈夫曼树构建  
过程中求WPL

给定 $n$ 个外结点的权值，编写程序输出哈夫曼树的WPL值。

【北京邮电大学研究生复试机试】

```
int BuildHuffmanTree(int weight[], int n) {  
    sort(weight, n); //对weight数组递增排序，需预先实现  
    int WPL=0;  
    for(int i=0; i<n-1; i++){  
        int t = weight[i]+weight[i+1];  
        WPL += t;  
        int j=i+2; //将t插入数组weight中，使数组保持按权值升序排列  
        while(j<n && t>weight[j])  
            weight[j-1]=weight[j], j++;  
        weight[j-1] = t;  
    }  
    return WPL;  
}
```

无需建树即可求WPL

给定一棵二叉树 $T$ ，采用二叉链表存储，结点结构为：

|             |               |              |
|-------------|---------------|--------------|
| <i>left</i> | <i>weight</i> | <i>right</i> |
|-------------|---------------|--------------|

先根遍历

其中叶结点的 $weight$ 域保存该结点的非负权值，请设计求 $T$ 的WPL的算法。【考研题全国卷】

```
int WPL = 0;
void WPLPreOrder(TreeNode *root, int k){
    if(root==NULL) return;
    if(root->left==NULL && root->right==NULL)
        WPL += root->weight*k;
    WPLPreOrder(root->left, k+1);
    WPLPreOrder(root->right, k+1);
}
```

$k$ 标识递归深度,亦为当前访问结点在二叉树中所在的层数

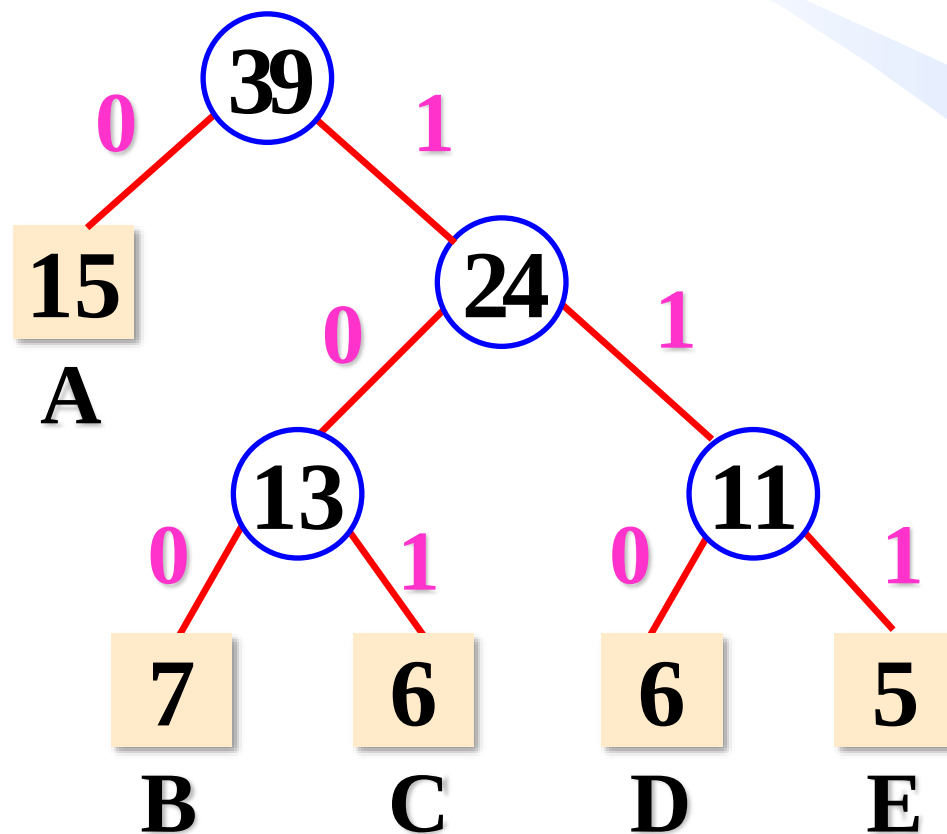
初始调用

WPLPreOrder(root, 0)

# 哈夫曼算法——贪心算法

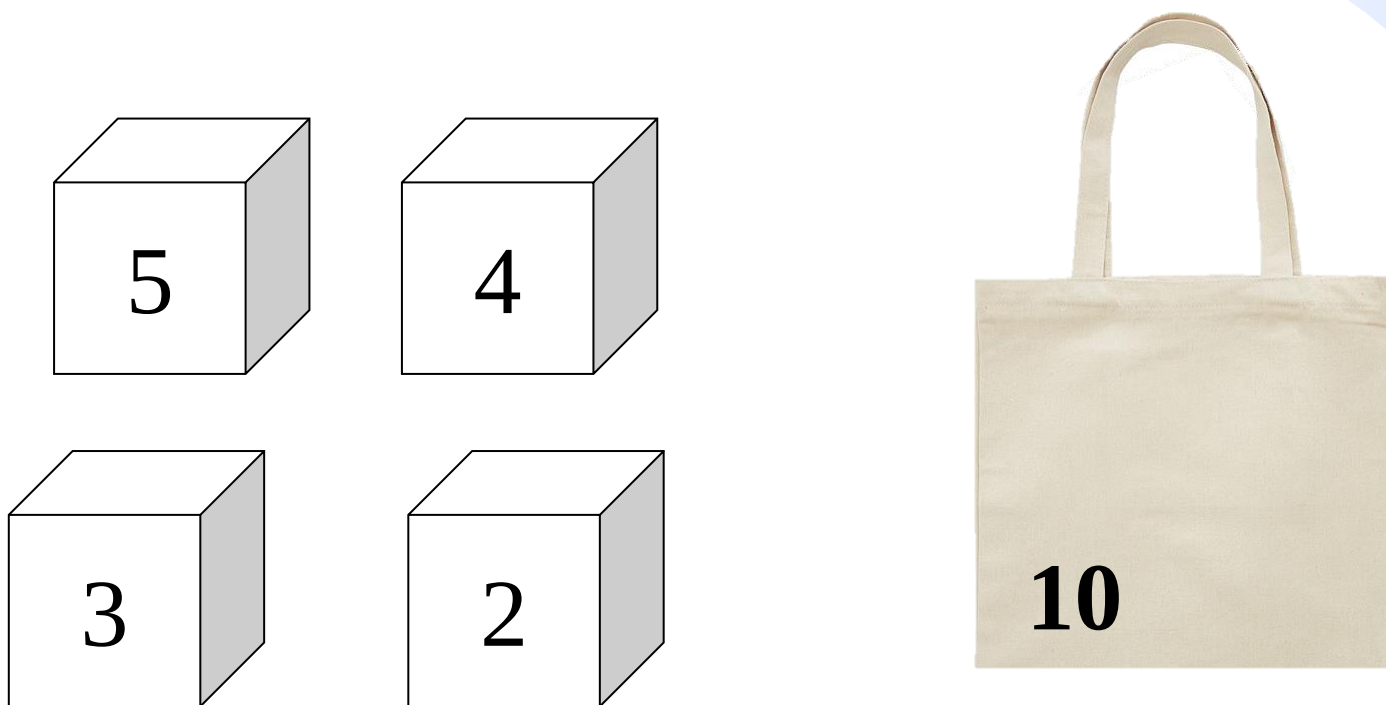
C

➤ 每步都选最优的方案



# 贪心算法

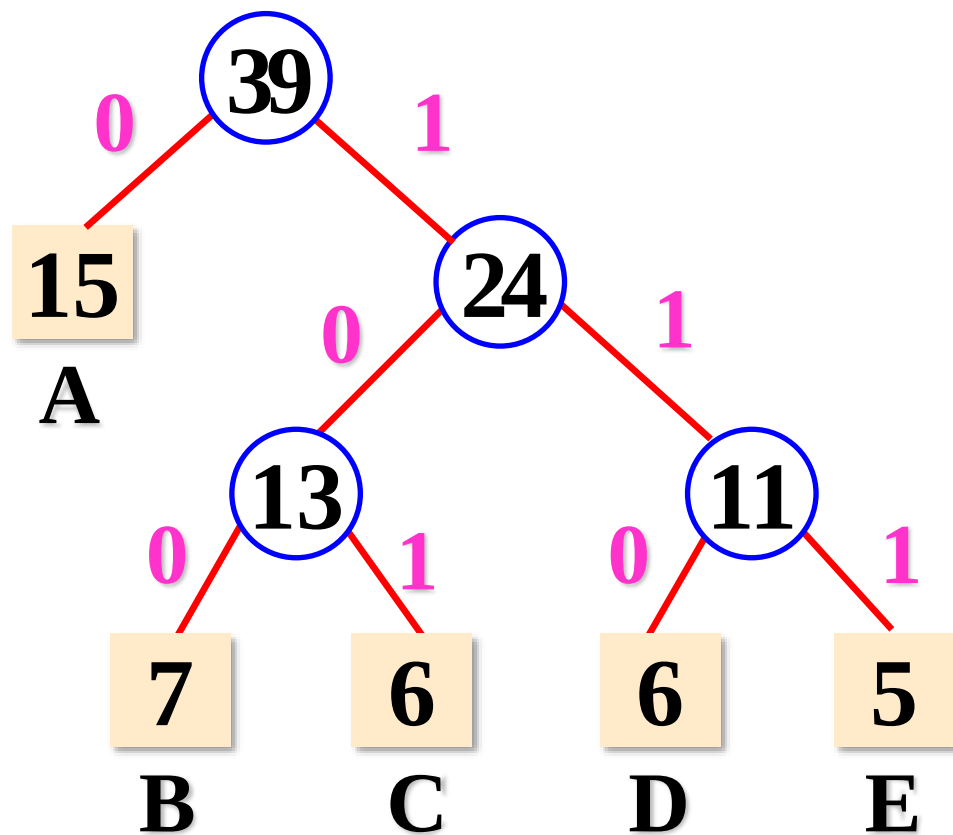
- 每步都选最优的方案，最后不见得总能得到最优解
- 步步最优，并不一定全局最优





## 解码

**解码过程：**依次读入文件的二进制码，从哈夫曼树的根结点出发，若当前读入0，则走向其左孩子，否则走向其右孩子，到达某一叶结点时，便可以译出相应的字符。



| 字符 | 编码  |
|----|-----|
| A  | 0   |
| B  | 100 |
| C  | 101 |
| D  | 110 |
| E  | 111 |

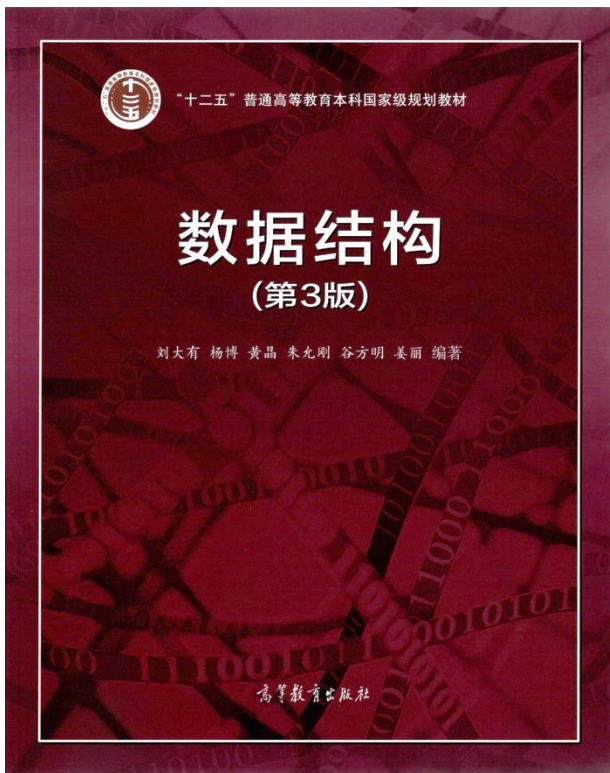
解码?

1 0 1 1 0 0 0

## 再谈解码

- 不仅要保存编码后的文件，还要保存哈夫曼树，解码方使用该哈夫曼树解码。缺点：哈夫曼树占空间较大。
- 保存哈夫曼编码表，解码方通过编码表复原哈夫曼树。
- JPEG：采用改进的哈夫曼编码（Canonical Huffman Code, 范式哈夫曼编码），仅保存编码长度，解码方通过编码长度即可复原编码表，并进行解码。
- *jpeg*、*png*、*mpeg*（*mp3*、*mp4*等）都用到了哈夫曼编码。





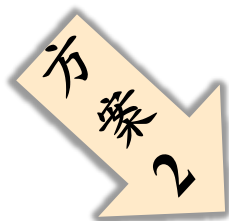
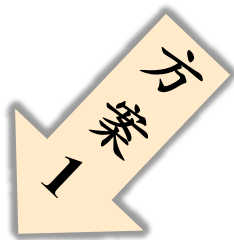
# 树和二叉树的应用

- 压缩与哈夫曼树
- **表达式树**
- 并查集
- 其他应用举例

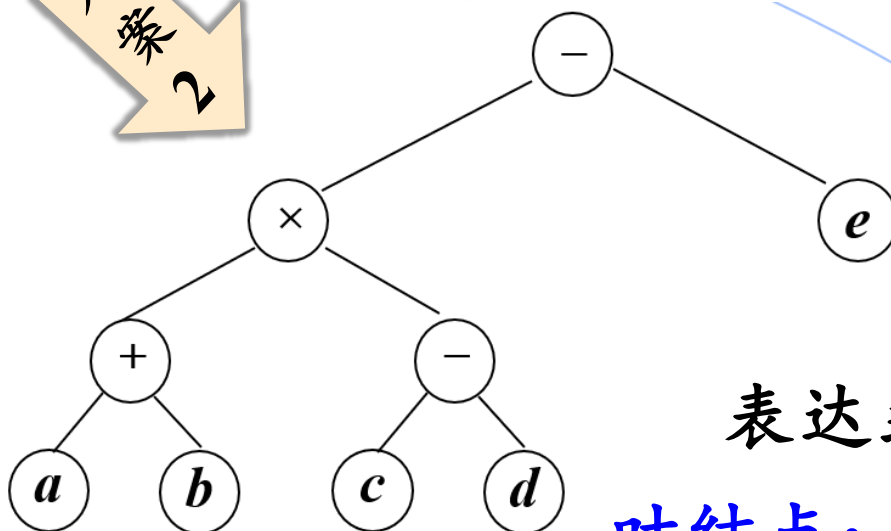
数据之法  
结构之美  
算法之道

zhuyungang@jlu.edu.cn

中缀表达式  
 $(a + b) \times (c - d) - e$



后缀表达式  
 $a b + c d - \times e -$



表达式树

叶结点：操作数

非叶结点：运算符

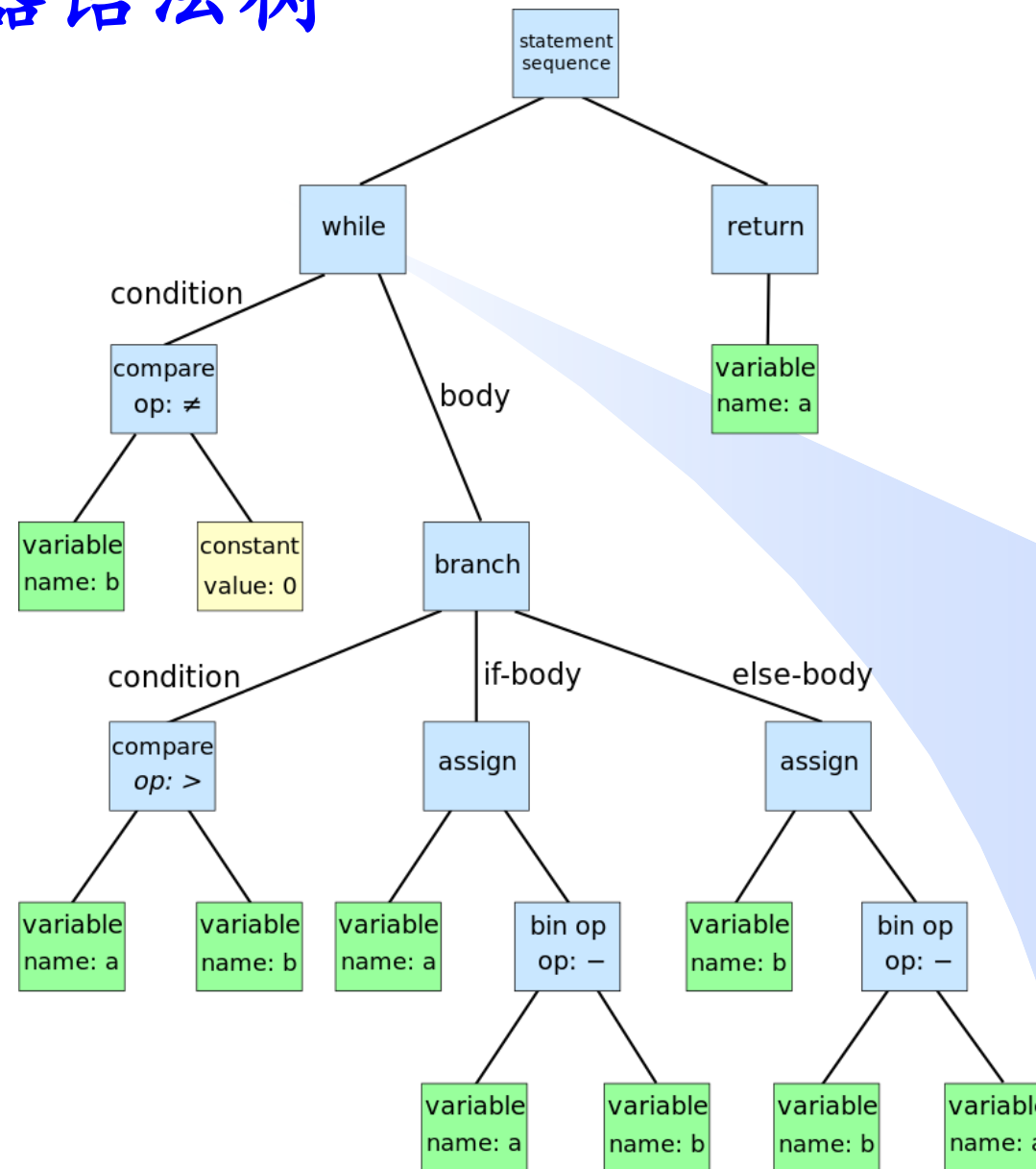
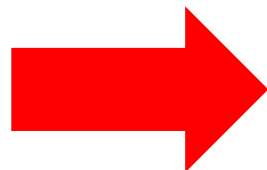
中根序列：中缀表达式

后根序列：后缀表达式

- (1) 灵活：可方便得到中/后缀表达式
- (2) 不仅可以分析表达式，也可以表示整个代码（if, for, while）结构，更有利于编译器对整个代码的分析和优化。

## 编译器语法树

```
while (b != 0)
  if (a > b)
    a = a - b;
  else
    b = b - a;
return a;
```



# 表达式树

如何构造表达式二叉树



已知表达式二叉树，如何计算其对应的值



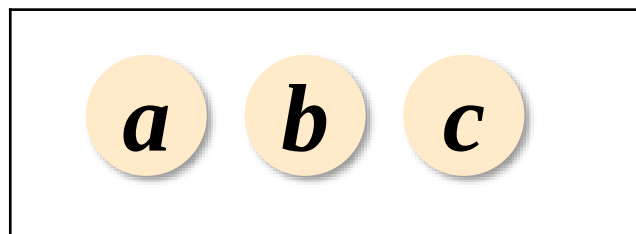
## 根据后缀表达式构造表达式二叉树

从左向右扫描后缀表达式，每扫描到一个符号就生成一个结点，该符号作为结点的数据域值，若扫描到的符号是：

- **操作数**：将此操作数结点压栈。
- **运算符**：从栈中弹出两个结点，分别作为当前运算符结点的右、左孩子，再将当前运算符结点压栈。
- 表达式扫描完成后，栈顶即为表达式二叉树的根结点。

$a b c \times +$

结点栈



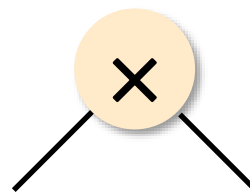
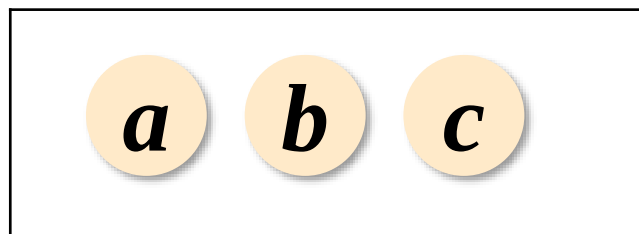
## 根据后缀表达式构造表达式二叉树

从左向右扫描后缀表达式，每扫描到一个符号就生成一个结点，该符号作为结点的数据域值，若扫描到的符号是：

- **操作数**：将此操作数结点压栈。
- **运算符**：从栈中弹出两个结点，分别作为当前运算符结点的右、左孩子，再将当前运算符结点压栈。
- 表达式扫描完成后，栈顶即为表达式二叉树的根结点。

$a b c \times +$

结点栈





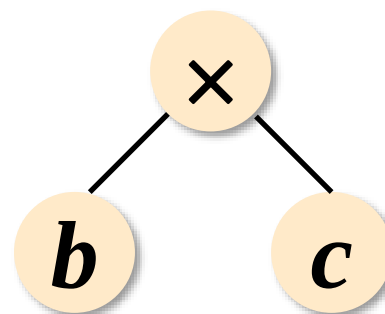
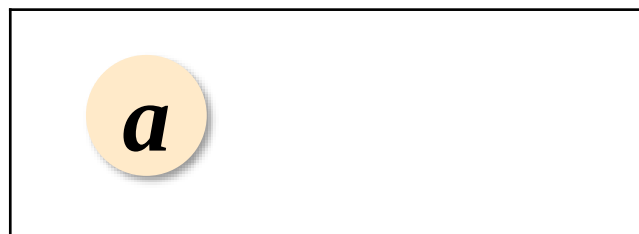
## 根据后缀表达式构造表达式二叉树

从左向右扫描后缀表达式，每扫描到一个符号就生成一个结点，该符号作为结点的数据域值，若扫描到的符号是：

- **操作数**：将此操作数结点压栈。
- **运算符**：从栈中弹出两个结点，分别作为当前运算符结点的右、左孩子，再将当前运算符结点压栈。
- 表达式扫描完成后，栈顶即为表达式二叉树的根结点。

$a b c \times +$

结点栈



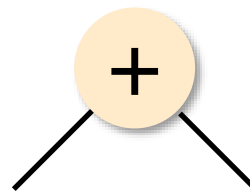
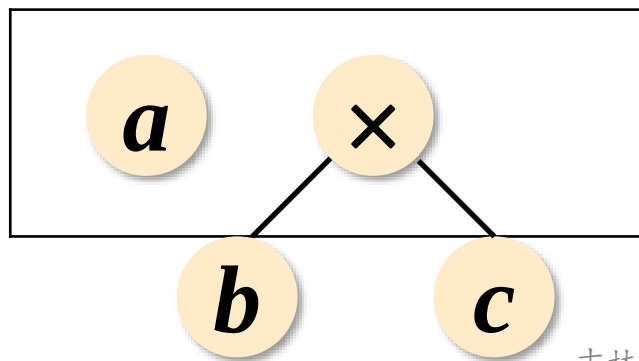
## 根据后缀表达式构造表达式二叉树

从左向右扫描后缀表达式，每扫描到一个符号就生成一个结点，该符号作为结点的数据域值，若扫描到的符号是：

- **操作数**：将此操作数结点压栈。
- **运算符**：从栈中弹出两个结点，分别作为当前运算符结点的右、左孩子，再将当前运算符结点压栈。
- 表达式扫描完成后，栈顶即为表达式二叉树的根结点。

$a b c \times +$

结点栈



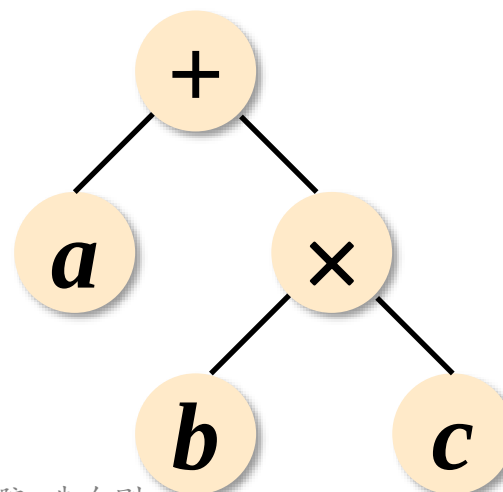
## 根据后缀表达式构造表达式二叉树

从左向右扫描后缀表达式，每扫描到一个符号就生成一个结点，该符号作为结点的数据域值，若扫描到的符号是：

- **操作数**：将此操作数结点压栈。
- **运算符**：从栈中弹出两个结点，分别作为当前运算符结点的右、左孩子，再将当前运算符结点压栈。
- 表达式扫描完成后，栈顶即为表达式二叉树的根结点。

$a b c \times +$

结点栈



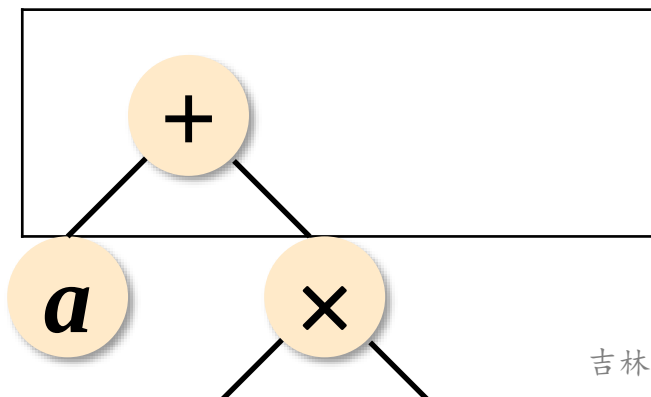
## 根据后缀表达式构造表达式二叉树

从左向右扫描后缀表达式，每扫描到一个符号就生成一个结点，该符号作为结点的数据域值，若扫描到的符号是：

- **操作数**：将此操作数结点压栈。
- **运算符**：从栈中弹出两个结点，分别作为当前运算符结点的右、左孩子，再将当前运算符结点压栈。
- 表达式扫描完成后，栈顶即为表达式二叉树的根结点。

$a b c \times +$

结点栈



```
TreeNode* BuildExpTree(char *s, int n){  
    //将长度为n的后缀表达式s转换为表达式树，返回根结点指针  
    stack<TreeNode*> stk;  
    for (int i = 0; i < n; i++) {  
        TreeNode *p = new TreeNode;  
        p->data = s[i];  
        if(s[i]>='0' && s[i]<='9') //操作数  
            p->left = p->right = NULL;  
        else { //运算符  
            p->right = stk.pop();  
            p->left = stk.pop();  
        }  
        stk.push(p);  
    }  
    return stk.peek();  
}
```

# 课下思考：中缀表达式直接转表达式树

```

TreeNode* build(char* s, int n) { //将长度为n的中缀表达式s转换为表达式树，返回根指针
    for(int i = 0; i < n; i++) {
        if (s[i]>='0' && s[i]<='9') { //假定操作数只有1位
            TreeNode *p = new TreeNode;
            p->data=s[i]; p->left=p->right=NULL;
            EXP.push(p);
        }
        else if(s[i] == '(') OP.push(s[i]);
        else if(s[i] == ')') {
            while (OP.peek() != '(')
                operation( );
            OP.pop();
        }
        else {
            while(!OP.empty() && OP.peek()!='(' && p[s[i]]<=p[OP.peek()])
                operation( );
            OP.push(s[i]);
        }
    }
    while (!OP.empty()) operation( );
    return EXP.peek();
}

```

```

Stack<TreeNode*> EXP; //表达式结点栈
Stack<char> OP; //运算符栈
void operation( ) {
    TreeNode *op= new TreeNode;
    op->data = OP.pop();
    op->right = EXP.pop();
    op->left = EXP.pop();
    EXP.push(op);
}

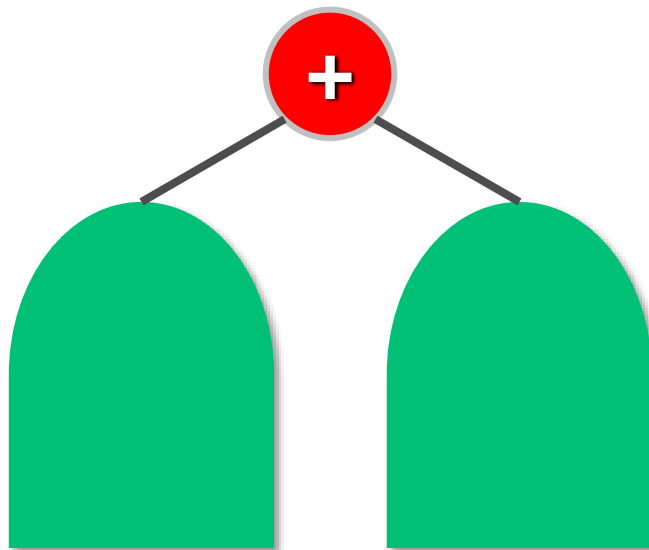
```

用表达式结点栈  
替代操作数栈

## 计算表达式二叉树对应的值

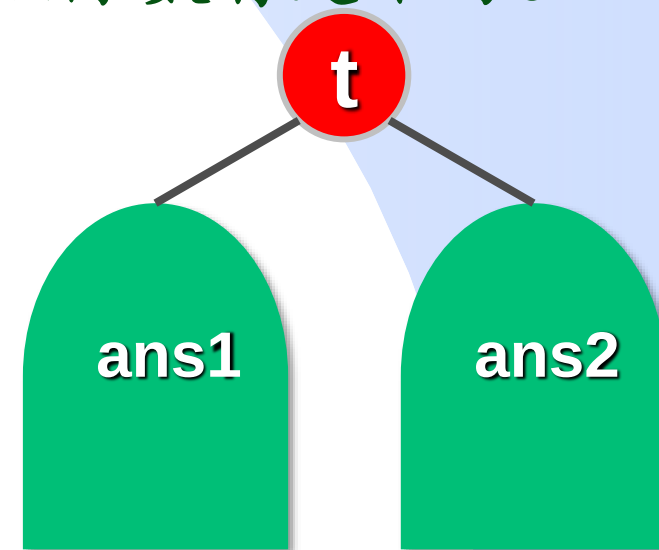
通过后根遍历表达式二叉树，可以计算表达式的值：

- ① 计算左子树对应表达式的值ans1;
- ② 计算右子树对应表达式的值ans2;
- ③ 将ans1和ans2做根结点对应的运算，得到整个表达式的值

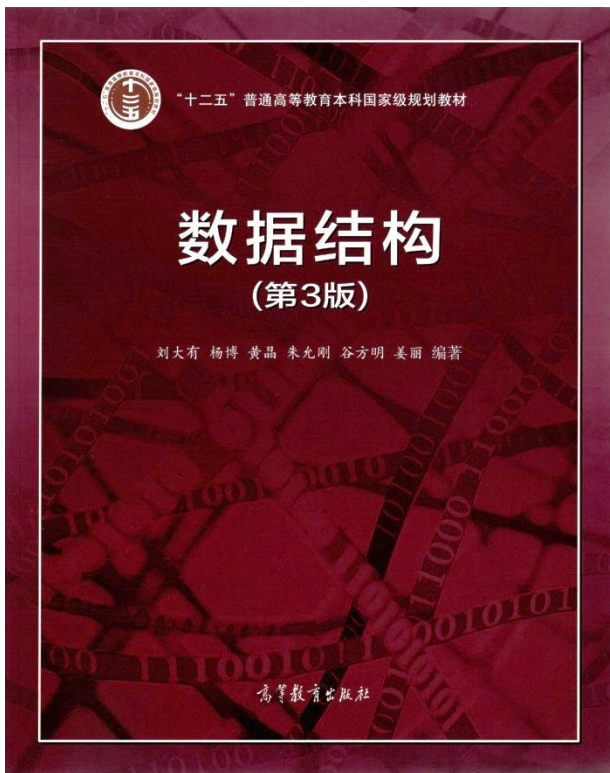


# 计算表达式二叉树对应的值

```
int Calc(TreeNode* t){ // t指向非空且合法的表达式树的根
    if(t->left==NULL && t->right== NULL) //若t是操作数
        return t->data-'0';           //假定操作数为0...9
    //若t是运算符
    int ans1 = Calc(t->left);           //计算左子树的值
    int ans2 = Calc(t->right);          //计算右子树的值
    char op = t->data; //假定仅有+、-、*、/运算,且除法除数肯定不为0
    if (op == '+') return ans1 + ans2;
    else if (op == '-') return ans1 - ans2;
    else if (op == '*') return ans1 * ans2;
    return ans1 / ans2;
}
```







# 树和二叉树的应用

- 压缩与哈夫曼树
- 表达式树
- **并查集**
- 其他应用举例

数据之法  
结构之美  
算法之道

zhuyungang@jlu.edu.cn

# 如何实现集合？

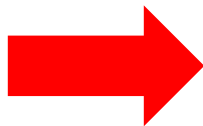
 $a$  $b$  $c$  $d$ 

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

# 如何实现集合？

| <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> |
|----------|----------|----------|----------|
| 1        | 0        | 1        | 1        |
| 1        | 0        | 0        | 1        |

求交集

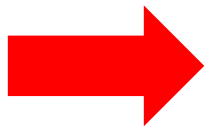


$O(n)$

# 如何实现集合？

| <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> |
|----------|----------|----------|----------|
| 1        | 0        | 1        | 1        |
| 1        | 0        | 0        | 1        |

求并集

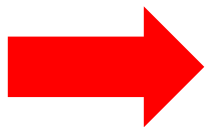


$O(n)$

# 如何实现集合？

| <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> |
|----------|----------|----------|----------|
| 1        | 0        | 1        | 1        |
| 1        | 0        | 0        | 1        |
| 0        | 0        | 1        | 1        |

判断一个元素  
属于哪个集合



$O(n)$

# 并查集

- 一些应用问题涉及将 $n$ 个不同的元素分成一组不相交的集合。
- 经常需要进行两种操作：
  - ① 查询某个元素所属的集合
  - ② 合并两个集合。
- 将维护该不相交集合的数据结构称为并查集。
- 选择集合中的某个元素代表整个集合，称为集合的代表元。

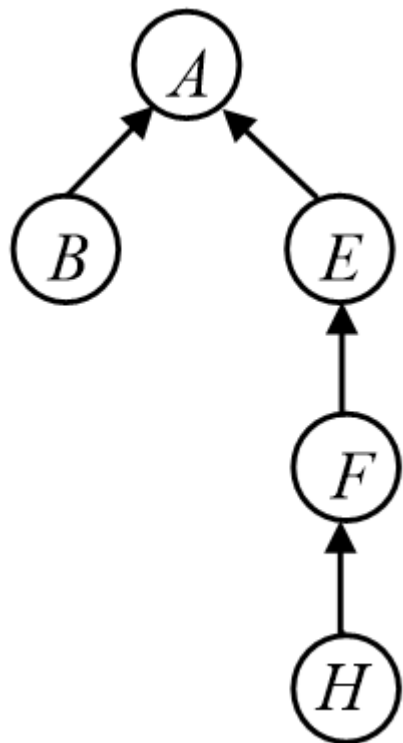
## 并查集的实现

设 $x$ 、 $y$ 表示集合中的元素，并查集的三个操作。

- **MAKE\_SET( $x$ )**: 建立一个新的集合，它的唯一元素是 $x$ ，因而 $x$ 是代表元。
- **UNION( $x, y$ )**: 将元素 $x$ 和 $y$ 所在集合合并成一个集合。
- **FIND( $x$ )**: 找 $x$ 所在的集合，返回该集合的代表元。

## 并查集的实现

➤ 并查集的一种高效实现方式是使用树表示集合。



树

集合

树的结点

集合的元素

树的根结点

集合的代表元



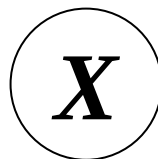
## 并查集的实现

设 $x$ 、 $y$ 表示集合中的元素，并查集的三个操作

**MAKE\_SET( $x$ )**: 建立一个新的集合，其唯一元素是 $x$



为 $x$ 生成一棵单结点树， $x$ 为根结点。



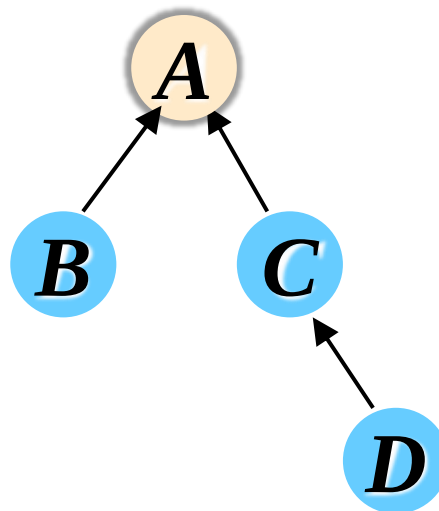
## 并查集的实现

设 $x$ 、 $y$ 表示集合中的元素，并查集的三个操作

**FIND( $x$ )**: 返回 $x$ 所在集合的代表元



查找 $x$ 所在的树的根结点。



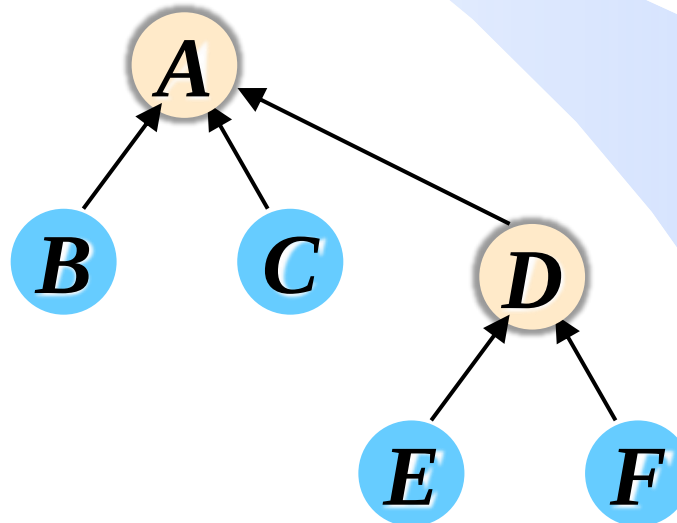
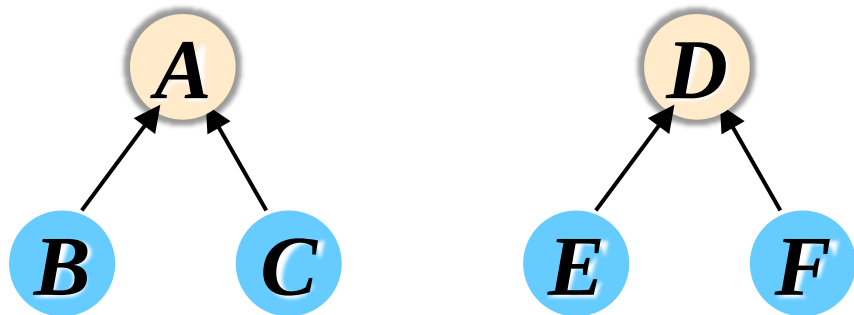
只需要树的向上访问能力，只需存储每个结点的父结点信息。

## 并查集的实现

设 $x$ 、 $y$ 表示集合中的元素，并查集的三个操作

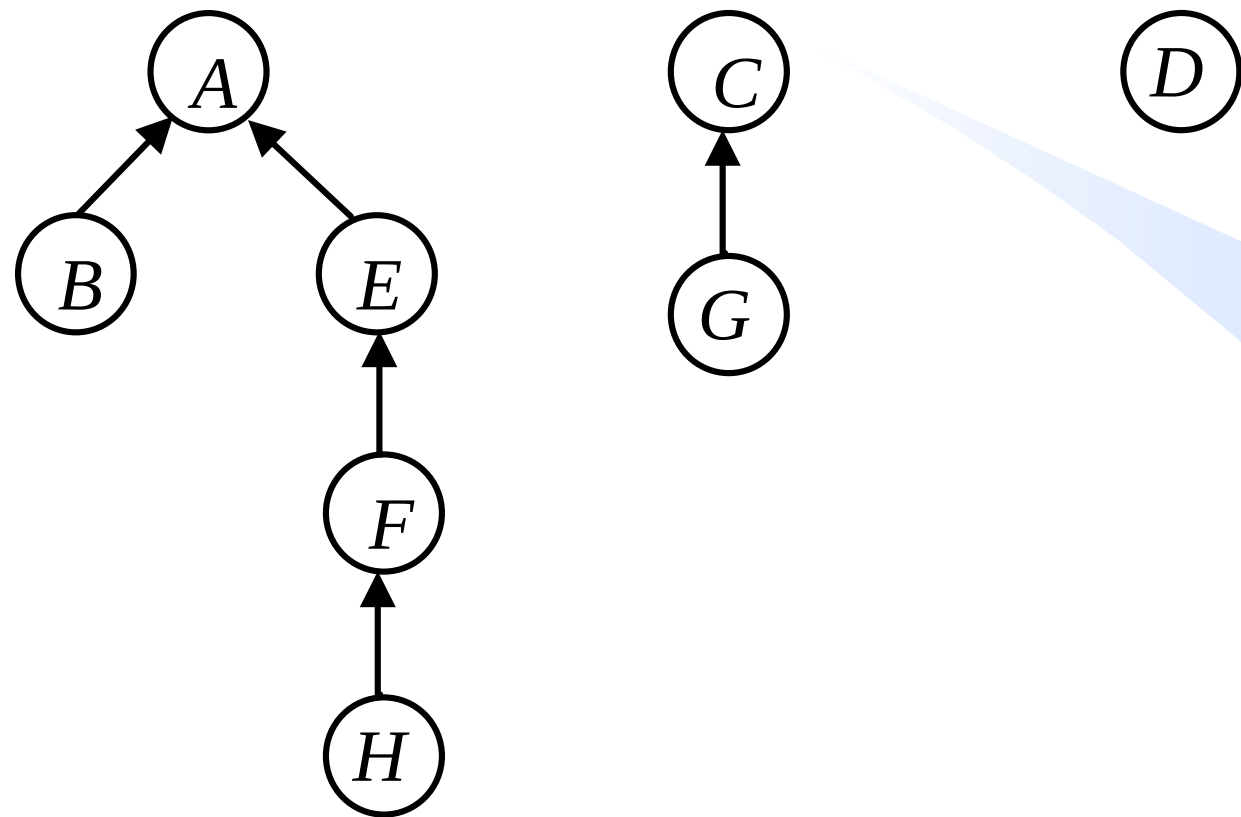
**UNION( $x$ ,  $y$ )**: 将 $x$ 和 $y$ 所在的集合合并成一个集合

合并 $x$ 所在的树和 $y$ 所在的树，让其中一棵树的根的父亲指针指向另一棵树的根。

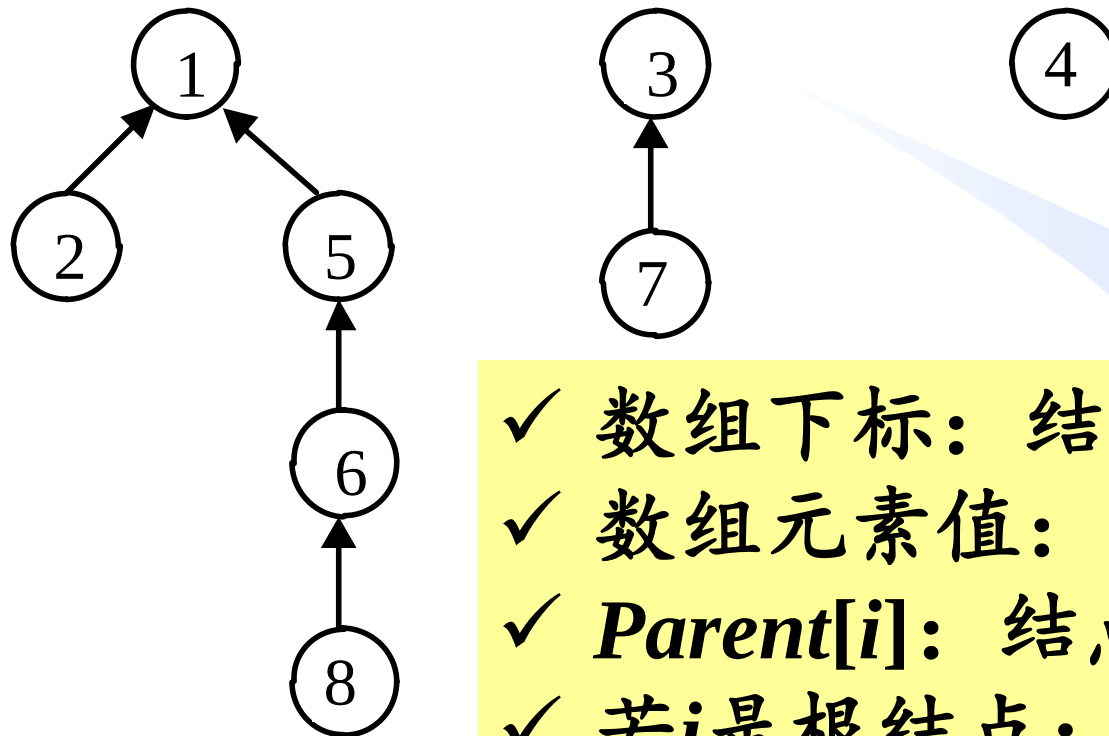


只需要树的向上访问能力，只需存储每个结点的父结点信息。

# 用 *Parent* 数组表示集合



# 用 *Parent* 数组表示集合



- ✓ 数组下标：结点编号
- ✓ 数组元素值：父结点下标
- ✓  $Parent[i]$ ：结点  $i$  的父结点下标
- ✓ 若  $i$  是根结点：  $Parent[i]=0$

数组下标（结点编号）：      1        2        3        4        5        6        7        8

*Parent*

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 1 | 5 | 3 | 6 |
|---|---|---|---|---|---|---|---|

```
void MS(int x){ //MAKE_SET:为元素x生成一棵单结点树
```

```
    Parent[x] = 0;
```

```
}
```

```
int FD(int x){ //FIND:查找x所在树的根结点
```

```
    if(Parent[x] == 0) return x;
```

```
    return FD(Parent[x]);
```

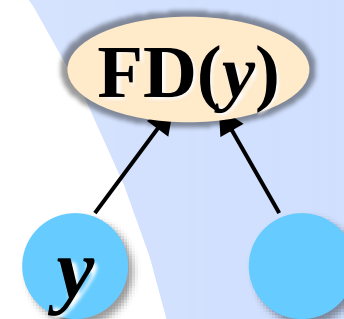
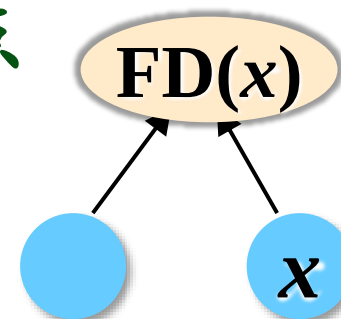
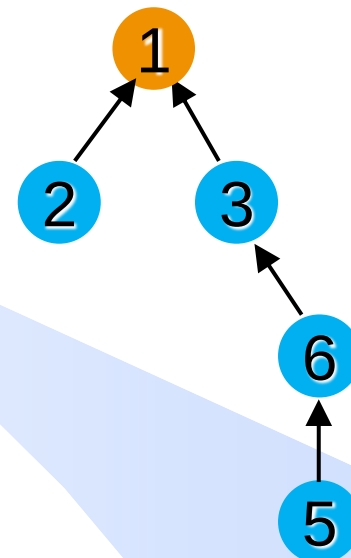
```
}
```

```
void UN(int x, int y){ //UNION:合并x和y的树
```

```
    //y所在树的根结点指向x所在树根结点
```

```
    Parent[FD(y)] = FD(x);
```

```
}
```



# 例



- ✓ 合并1和2
- ✓ 合并1和3
- ✓ 合并5和6

最坏情况  
合并12  
合并31  
合并53  
合并65

$FD(x): O(n)$   
 $UN(x, y): O(n)$

```
void UN(int x, int y){  
    Parent[FD(y)] = FD(x);  
}
```



树太高了，退化成为一条链

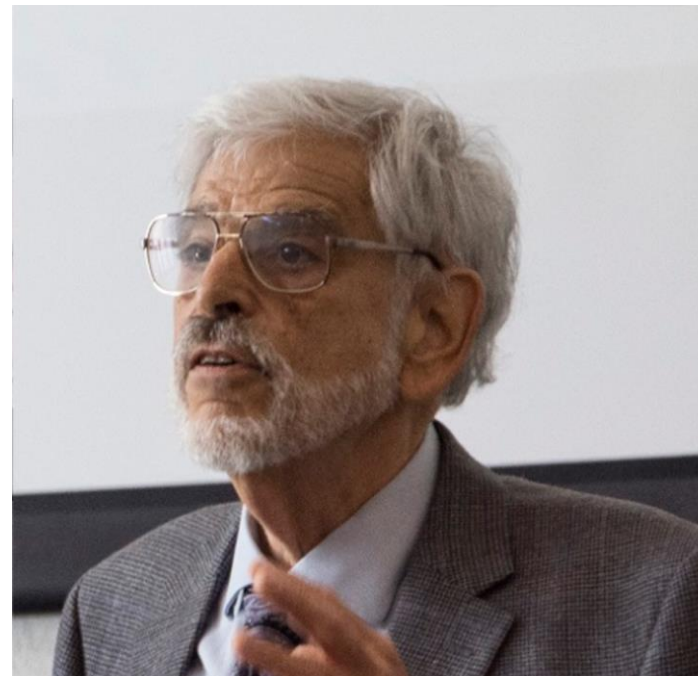
**FIND 和 UNION 操作时间复杂度取决于树的高度。**

A

# 并查集的按秩合并和路径压缩策略



**John Hopcroft**  
图灵奖获得者  
康奈尔大学教授  
美国科学院院士  
美国工程院院士  
中国科学院外籍院士



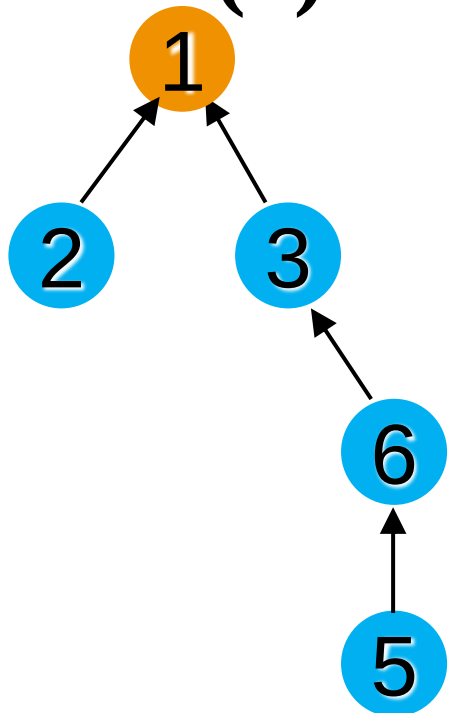
**Jeffrey Ullman**  
图灵奖获得者  
斯坦福大学教授  
美国科学院院士  
美国工程院院士



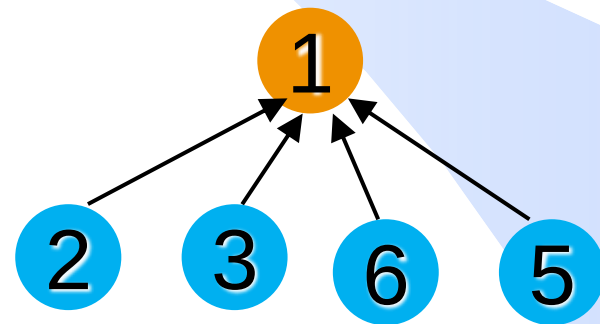
## 并查集的优化：策略1——路径压缩

在 $\text{FIND}(x)$ 操作中，找到元素 $x$ 所在树的根 $f_x$ 之后，将 $x$ 到根 $f_x$ 路径上的所有结点的父亲都改成 $f_x$ 。

例： $\text{FIND}(5)$



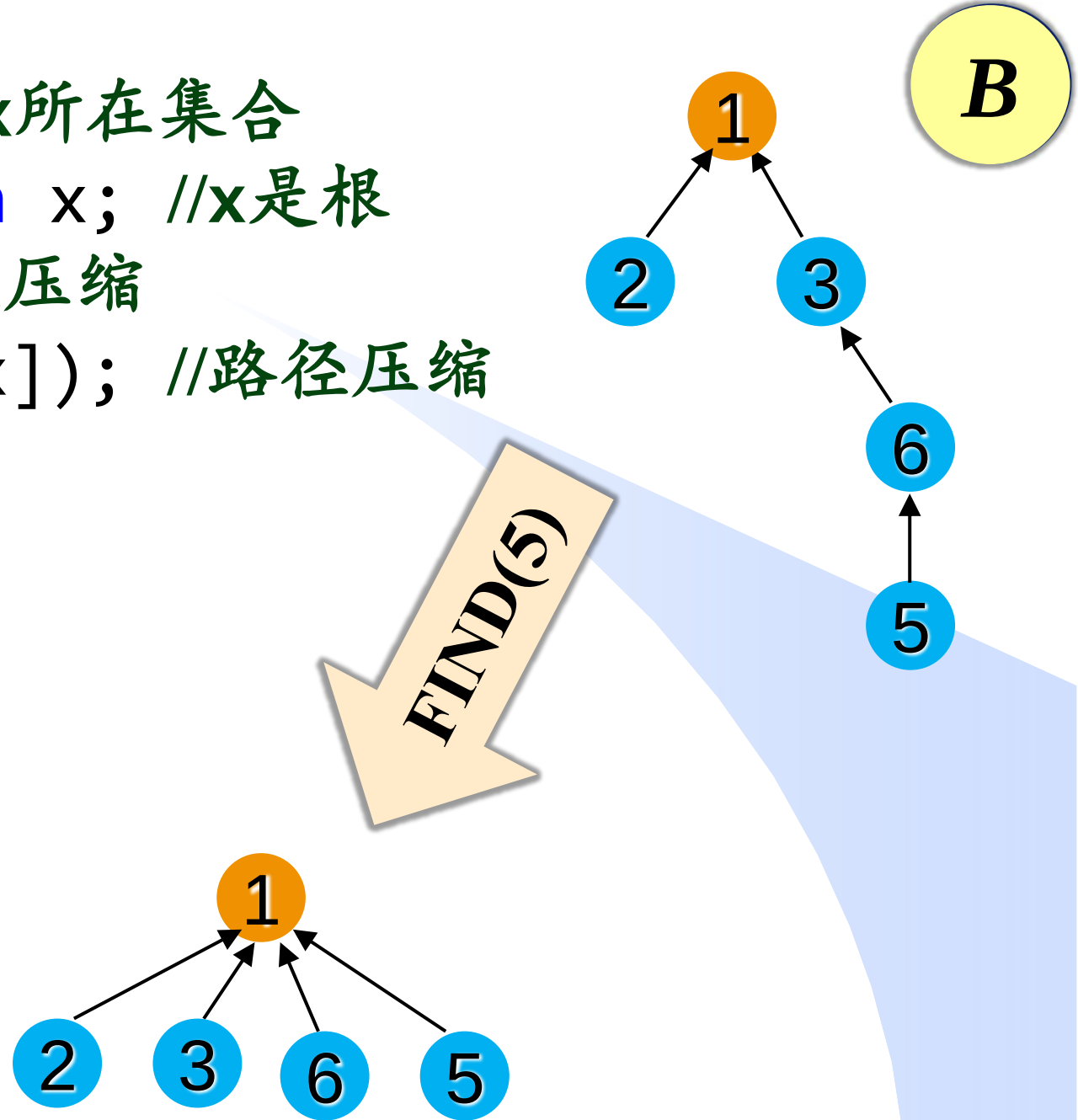
$\text{FIND}(5)$



路径压缩  
把一棵很高的树变矮

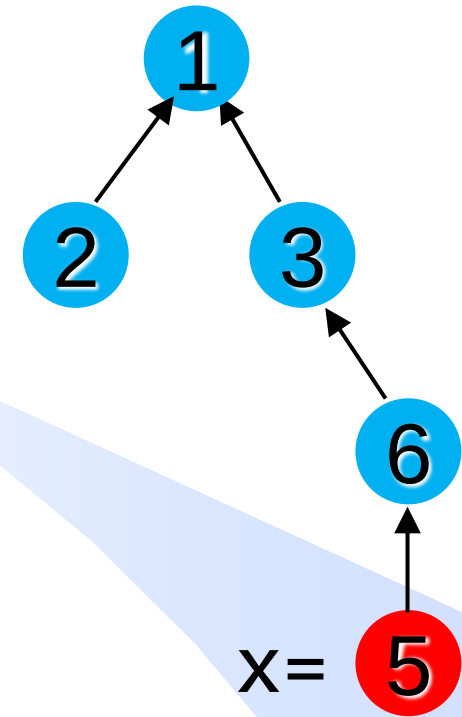
```
int FIND(int x){ //查找元素x所在集合
    if(Parent[x]==0) return x; //x是根
    //递归查找根结点，同时路径压缩
    Parent[x]=FIND(Parent[x]); //路径压缩
    return Parent[x];
}
```

该过程是一个两趟方法，递归时沿着查找路径向上，直到找到根，递归返回时，从根向下顺带更新每个结点的父指针，使其指向根。



```
int FIND(int x){  
    if(Parent[x]==0) return x;  
    Parent[x]=FIND(Parent[x]);  
    return Parent[x];  
}
```

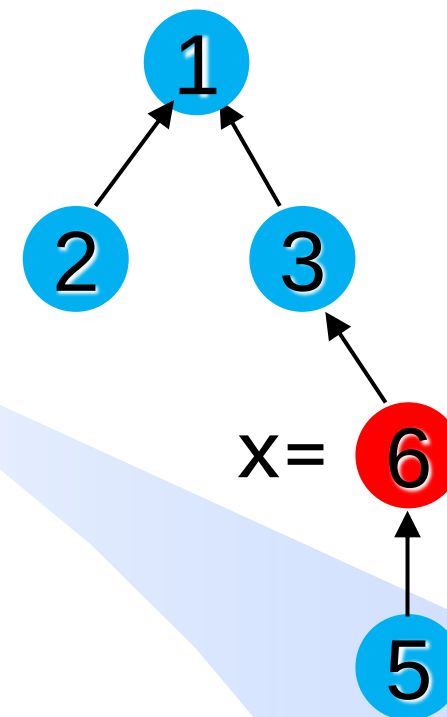
```
int FIND(5){  
    .....  
    Parent[5]=FIND(6);  
    return Parent[5];  
}
```



```
int FIND(int x){  
    if(Parent[x]==0) return x;  
    Parent[x]=FIND(Parent[x]);  
    return Parent[x];  
}
```

```
int FIND(5){  
    .....  
    Parent[5]=FIND(6);  
    return Parent[5];  
}
```

```
int FIND(6){  
    .....  
    Parent[6]=FIND(3);  
    return Parent[6];  
}
```

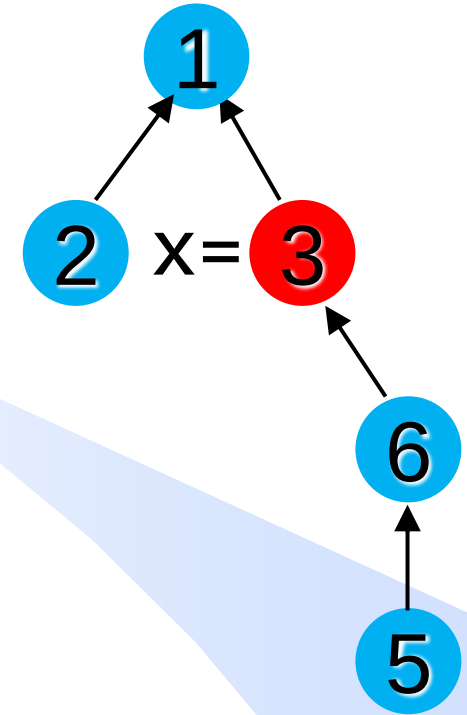


```
int FIND(int x){  
    if(Parent[x]==0) return x;  
    Parent[x]=FIND(Parent[x]);  
    return Parent[x];  
}
```

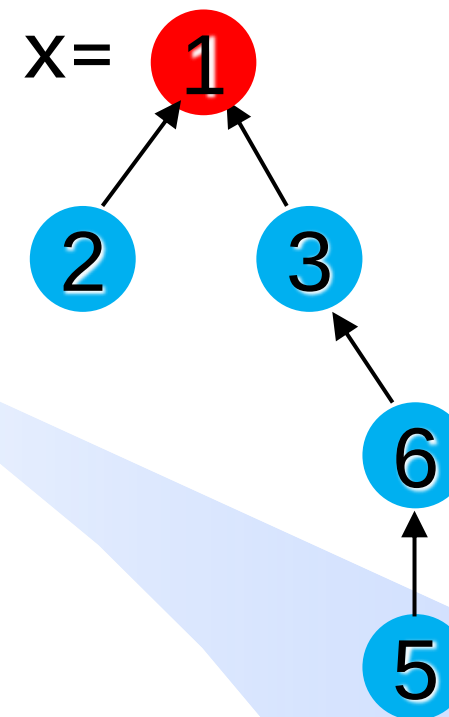
```
int FIND(5){  
    .....  
    Parent[5]=FIND(6);  
    return Parent[5];  
}
```

```
int FIND(6){  
    .....  
    Parent[6]=FIND(3);  
    return Parent[6];  
}
```

```
int FIND(3){  
    .....  
    Parent[3]=FIND(1);  
    return Parent[3];  
}
```



```
int FIND(int x){  
    if(Parent[x]==0) return x;  
    Parent[x]=FIND(Parent[x]);  
    return Parent[x];  
}
```



```
int FIND(5){
```

.....

```
    Parent[5]=FIND(6);  
    return Parent[5];  
}
```

```
int FIND(6){
```

.....

```
    Parent[6]=FIND(3);  
    return Parent[6];  
}
```

```
int FIND(3){
```

.....

```
    Parent[3]=FIND(1);  
    return Parent[3];  
}
```

```
int FIND(1){
```

```
    if(Parent[1]==0)  
        return 1;
```

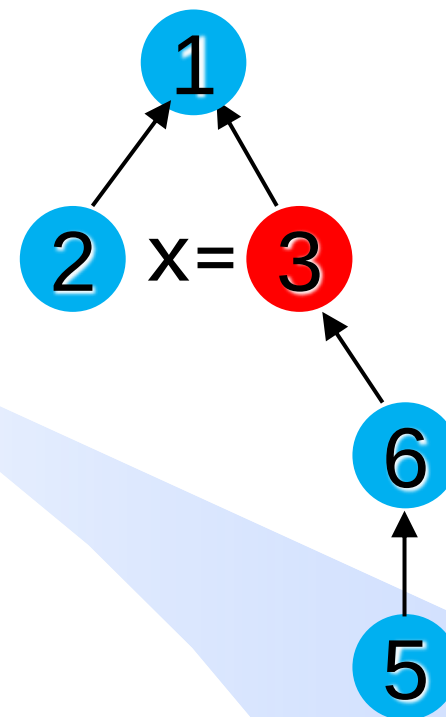
.....

```
}
```

```

int FIND(int x){
    if(Parent[x]==0) return x;
    Parent[x]=FIND(Parent[x]);
    return Parent[x];
}

```



```

int FIND(5){

```

```

.....

```

```

    Parent[5]=FIND(6);
    return Parent[5];
}

```

```

int FIND(6){

```

```

.....

```

```

    Parent[6]=FIND(3);
    return Parent[6];
}

```

```

int FIND(3){

```

```

.....

```

```

    Parent[3]=FIND(1);
    return Parent[3];
}

```

```

int FIND(1){

```

```

    if(Parent[1]==0)

```

```

        return 1;

```

```

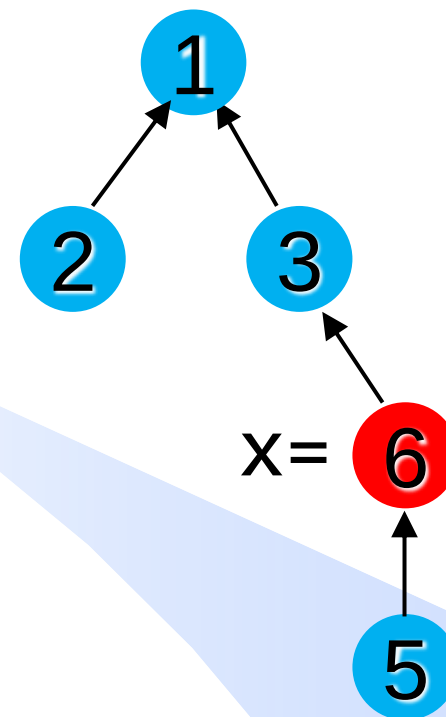
.....
}

```

```

int FIND(int x){
    if(Parent[x]==0) return x;
    Parent[x]=FIND(Parent[x]);
    return Parent[x];
}

```



```

int FIND(5){

```

```

.....

```

```

    Parent[5]=FIND(6);
    return Parent[5];
}

```

```

int FIND(6){

```

```

.....

```

```

    Parent[6]=FIND(3);
    return Parent[6];
}

```

```

int FIND(3){

```

```

.....

```

```

    Parent[3]=FIND(1);
    return Parent[3];
}

```

```

int FIND(1){

```

```

    if(Parent[1]==0)

```

```

        return 1;

```

```

.....
}

```



```

int FIND(int x){
    if(Parent[x]==0) return x;
    Parent[x]=FIND(Parent[x]);
    return Parent[x];
}

```

```

int FIND(5){

```

```

    .....

```

```

    Parent[5]=FIND(6);
    return Parent[5];
}

```

```

int FIND(6){

```

```

    .....

```

```

    Parent[6]=FIND(3);
    return Parent[6];
}

```

```

int FIND(3){

```

```

    .....

```

```

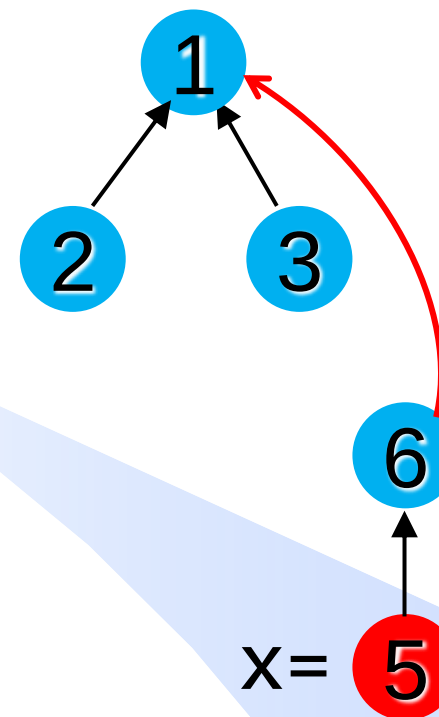
    Parent[3]=FIND(1);
    return Parent[3];
}

```

```

int FIND(1){
    if(Parent[1]==0)
        return 1;
    .....
}

```



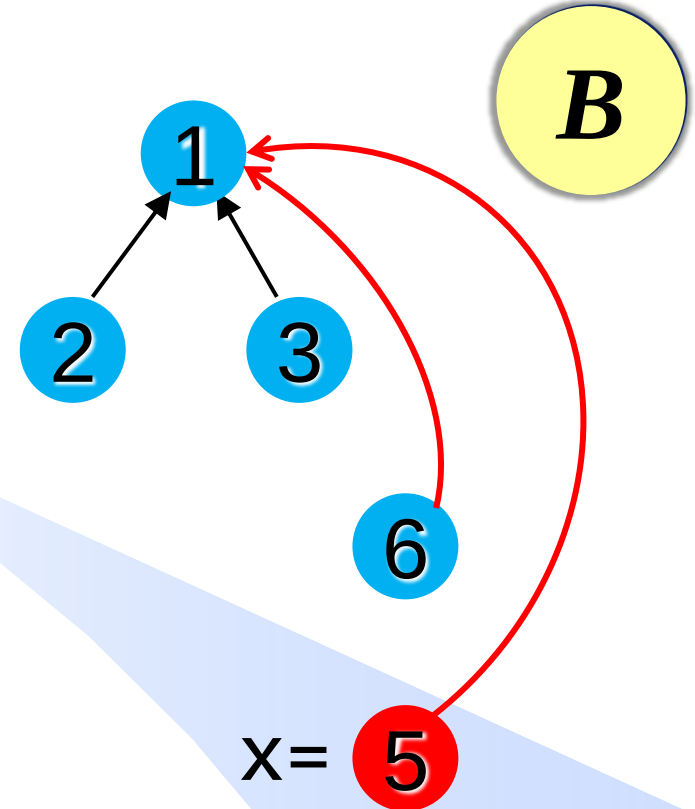
x=

5

```

int FIND(int x){
    if(Parent[x]==0) return x;
    Parent[x]=FIND(Parent[x]);
    return Parent[x];
}

```



```

int FIND(5){

```

.....

```

    Parent[5]=FIND(6);
    return Parent[5];
}

```

```

int FIND(6){

```

.....

```

    Parent[6]=FIND(3);
    return Parent[6];
}

```

```

int FIND(3){

```

.....

```

    Parent[3]=FIND(1);
    return Parent[3];
}

```

```

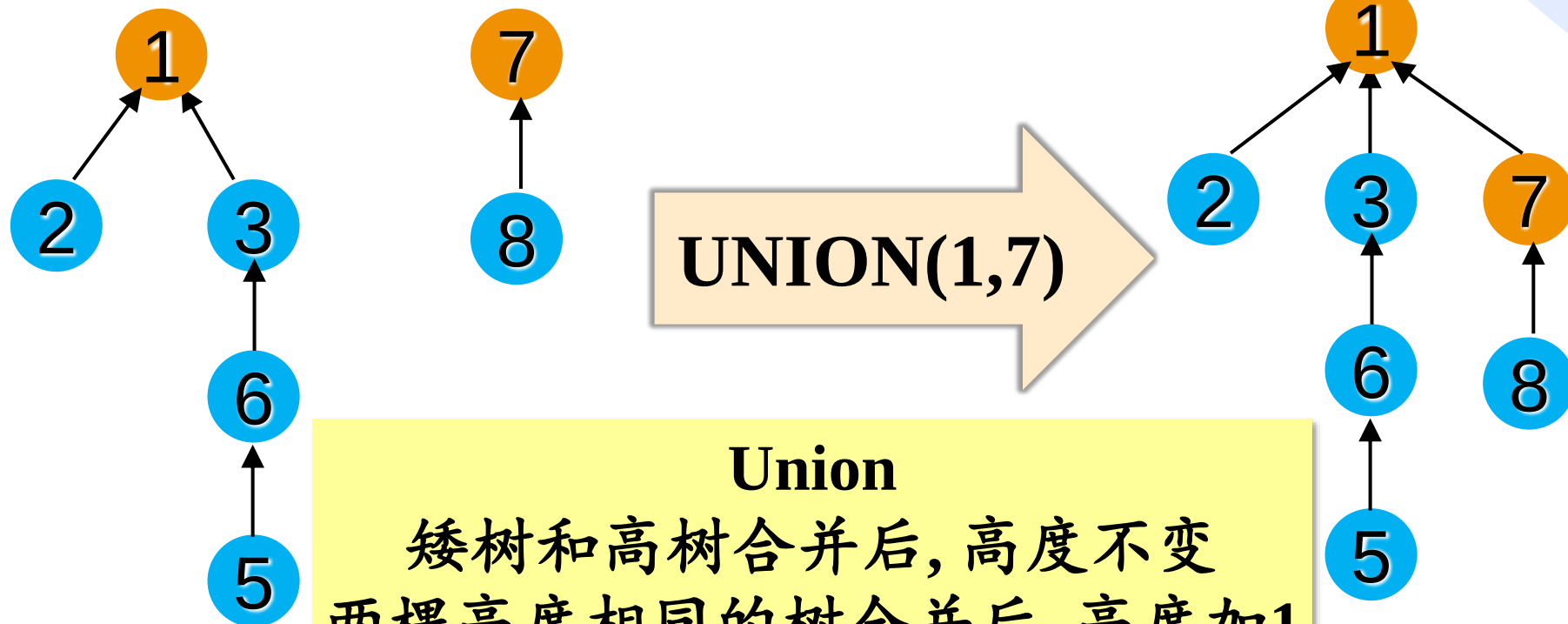
int FIND(1){
    if(Parent[1]==0)
        return 1;
    .....
}

```

通过递归沿着查找路径向上，直到找到根。通过递归的返回过程，从根向下更新每个结点的父指针，使其指向根。

# 并查集的优化：策略2 —— 按“~~高度~~”合并？

- 对每个结点，记录以该结点为根的子树的高度。
- 合并操作：高树的根作为矮树的根的父亲。
- 树高变化时，高度应实时更新（Union操作中可实时更新树高，但Find操作难以实时高效更新树高）。



## Union

矮树和高树合并后，高度不变  
两棵高度相同的树合并后，高度加1

## Find

路径压缩后，树高可能减小或不变，很难高效、精确的更新

- ✓ 树高增加时可以更新树高
- ✓ 树高减小时很难更新树高

## 并查集的优化：策略2——按秩合并

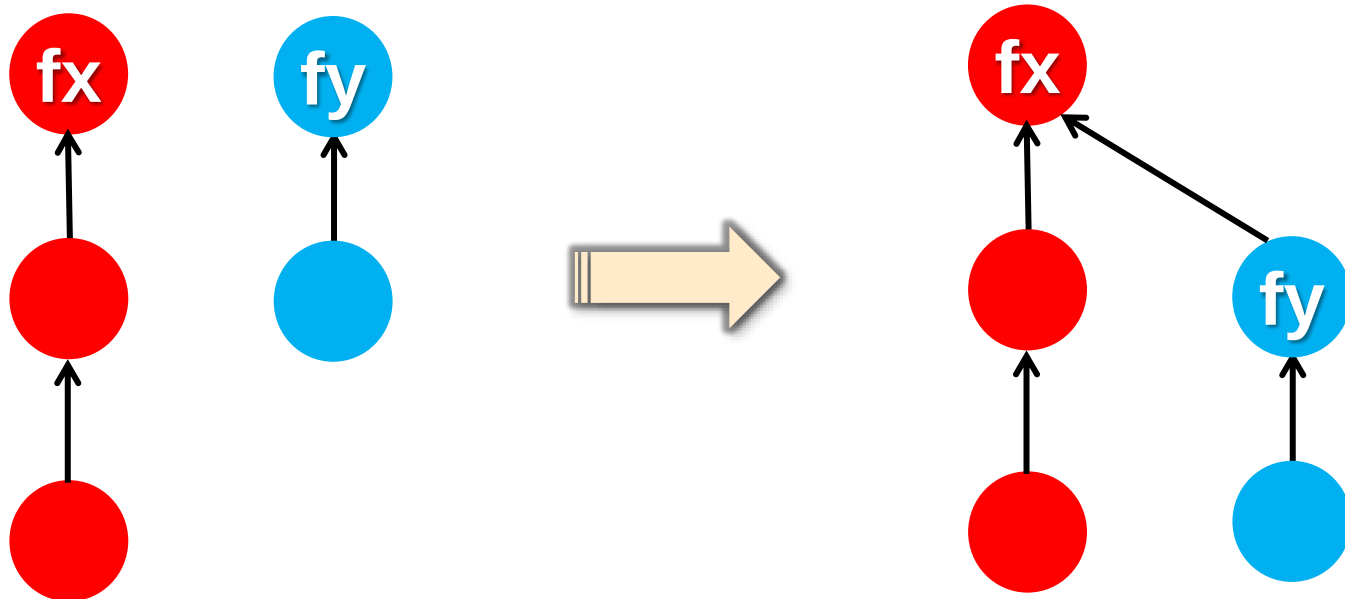
- 每个结点维护一个秩, 表示以该结点为根的子树的高度的上界。
- 合并操作：秩大的根作为秩小的根的父亲。



树高增加时（可能出现在Union操作时）更新秩、秩增加  
树高减小时（可能出现在Find操作路径压缩时）不更新秩、秩不变

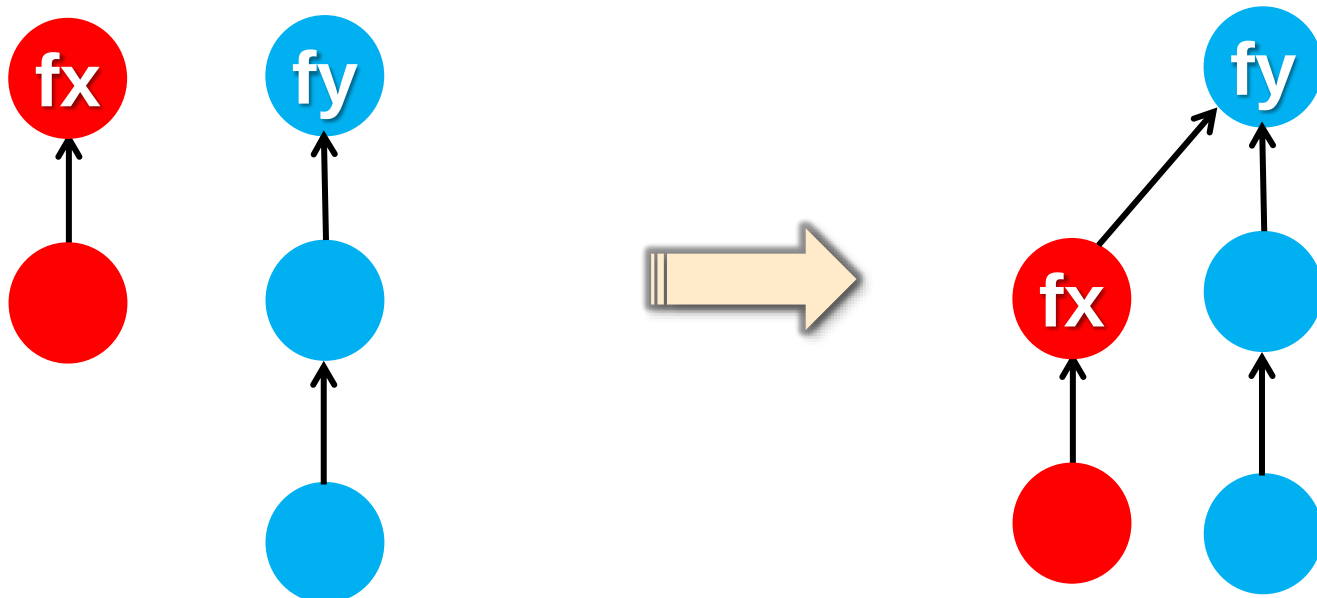
```
void UNION(int x, int y){  
    int fx=FIND(x), fy=FIND(y);  
    if(fx==fy) return;  
    if(rank[fx]>rank[fy])  
        Parent[fy]=fx;
```

//按秩合并  
//取x和y所在树的根  
//x和y在同一棵树  
//fy秩小  
//fx作为fy的父结点



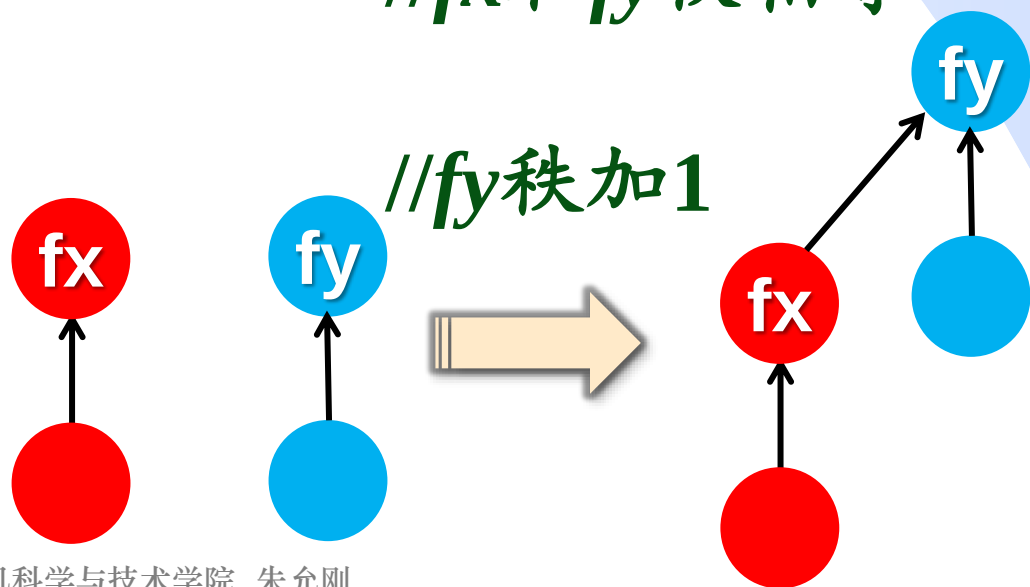
```
void UNION(int x, int y){  
    int fx=FIND(x), fy=FIND(y);  
    if(fx==fy) return;  
    if(rank[fx]>rank[fy])  
        Parent[fy]=fx;  
    else if(rank[fx]<rank[fy])  
        Parent[fx]=fy;  
}
```

//按秩合并  
//取 $x$ 和 $y$ 所在树的根  
// $x$ 和 $y$ 在同一棵树  
// $fy$ 秩小  
// $fx$ 作为 $fy$ 的父结点  
// $fx$ 秩小  
// $fy$ 作为 $fx$ 的父结点



```
void UNION(int x, int y){  
    int fx=FIND(x), fy=FIND(y);  
    if(fx==fy) return;  
    if(rank[fx]>rank[fy])  
        Parent[fy]=fx;  
    else if(rank[fx]<rank[fy])  
        Parent[fx]=fy;  
    else{  
        Parent[fx]=fy;  
        rank[fy]++;  
    }  
}
```

//按秩合并  
//取x和y所在树的根  
//x和y在同一棵树  
//fy秩小  
//fx作为fy的父结点  
//fx秩小  
//fy作为fx的父结点  
//fx和fy秩相等



```
void UNION(int x, int y){  
    int fx=FIND(x), fy=FIND(y);  
    if(fx==fy) return;  
    if(rank[fx]>rank[fy])  
        Parent[fy]=fx;  
    else if(rank[fx]<rank[fy])  
        Parent[fx]=fy;  
    else{  
        Parent[fx]=fy;  
        rank[fy]++;  
    }  
}
```

实现技巧:

只有根结点的秩在合并时起作用，根结点 $Parent$ 域空闲，故可以用 $Parent$ 域保存结点的秩。

//按秩合并  
//取 $x$ 和 $y$ 所在树的根  
// $x$ 和 $y$ 在同一棵树  
// $fy$ 秩小  
// $fx$ 作为 $fy$ 的父结点  
// $fx$ 秩小  
// $fy$ 作为 $fx$ 的父结点  
// $fx$ 和 $fy$ 秩相等

若 $x$ 不是根结点:

$Parent[x]$ :  $x$ 的父亲下标

若 $x$ 是根结点:

$Parent[x]$ :  $x$ 秩的相反数



$Parent[x] > 0$ : 表示 $x$ 父结点的下标;

$Parent[x] \leq 0$ : 表示 $x$ 是根,  $Parent[x]$ 值为 $x$ 秩的相反数

```
void Make_Set(int x){           //最终版本
    Parent[x] = 0;             //根结点的秩为0
}

int Find(int x){               //最终版本
    if(Parent[x] <= 0) return x; //x是根
    Parent[x] = Find(Parent[x]); //路径压缩
    return Parent[x];
}
```

```
void Union(int x, int y){  
    int fx=Find(x), fy=Find(y);  
    if(fx==fy) return;  
    if(Parent[fx]<Parent[fy])  
        //存的是秩的相反数，实际上相当  $rank[fx] > rank[fy]$   
        Parent[fy]=fx; //fx作为fy的父结点  
    else if(Parent[fx]>Parent[fy]) //fx秩小  
        Parent[fx]=fy; //fy作为fx的父结点  
    else{  
        //fx和fy秩相等  
        Parent[fx]=fy; //fy作为fx的父结点  
        Parent[fy]--; //fy秩加1  
    }  
}
```

# 时间复杂度

- 单独使用**按秩合并**或**路径压缩**，Union和Find操作的均摊时间复杂度为 $O(\log n)$ .
- 定理：一组 **$m$** 个Make\_Set、Union和Find操作的序列，其中Make\_Set操作的个数为 **$n$** ，在不相交集合上同时使用**按秩合并**与**路径压缩**，最坏情况时间复杂度为 $O(m \alpha(n))$ .
- 上述定理表明：同时使用**按秩合并**和**路径压缩**两种优化策略，每个操作的均摊时间复杂度接近 $O(1)$ .

## 并查集的应用——等价性问题

- **等价关系**：反身性、对称性、传递性。  
如：相等关系、亲戚关系
- **等价性问题**：给定集合 $S$ 及其上的若干等价元素对，询问 $S$ 上的两个元素是否等价。

例： $S = \{1, 2, 3, 4, 5, 6, 7, 8\}$

输入：1~2, 1~5, 5~6, 3~7, 6~8

询问：1和6是否等价？ 7和8是否等价？

等价类： $\{1, 2, 5, 6, 8\}, \{3, 7\}, \{4\}$

- 等价性问题的关键在于等价类的维护，这正是并查集的一个典型应用。

一个等价类  $\Leftrightarrow$  一个并查集

## 并查集的应用—等价性问题

- 初始每个元素生成一个集合，形成若干不相交集合并；

`for(int i=1;i<=n;i++) Make_Set(i);`

- 输入一对等价元素 $x$ 和 $y$ ，就合并 $x$ 和 $y$ 所在的集合；

`Union(x,y);`

- 判断两个元素 $x$ 和 $y$ 是否等价：  $x$ 和 $y$ 是否在同一个集合里。

`Find(x)==Find(y)`

## 并查集的应用—等价性问题

```
void EquivalenceClass(int n, int m, int q){  
    //等价性问题, n为元素个数, m为等价元素对数, q为查询数  
    int x,y;  
    for(int i=1; i<=n ;i++) Make_Set(i); //初始化并查集  
    while(m--){ //处理等价关系  
        scanf("%d %d", &x, &y); //读入一对等价元素  
        Union(x,y); //合并x、y所在的集合  
    }  
    while(q--){ //处理查询  
        scanf("%d %d", &x, &y); //读入一个查询  
        if(Find(x)==Find(y)) printf("YES\n");  
        else printf("NO\n");  
    }  
}
```

## 课下思考

有 $n$ 个城市，其中一些彼此相连，另一些没有相连。如果城市 $a$ 与城市 $b$ 直接相连，且城市 $b$ 与城市 $c$ 直接相连，那么城市 $a$ 与城市 $c$ 间接相连。“省份”是一组直接或间接相连的城市。给你一个 $n \times n$ 的矩阵 $C$ ，其中 $C[i][j]=1$ 表示第 $i$ 个城市和第 $j$ 个城市直接相连，而 $C[i][j]=0$ 表示二者不直接相连。返回矩阵中“省份”的数量。【华为、字节跳动、滴滴、小米、谷歌、苹果、亚马逊面试题】

省份 $\Leftrightarrow$ 等价类 $\Leftrightarrow$ 并查集

## 课下阅读

```
#define maxn 210
int findProvinceCount(int C[maxn][maxn], int n){
    //假设C从下标1开始存储数据
    for(int i=1; i<=n; i++) Make_Set(i);
    for(int i=1; i<=n; i++)
        for(int j=i+1; j<=n; j++)
            if(C[i][j] == 1) Union(i,j); //i和j相连
    int cnt = 0; //cnt为集合数目
    for(int i=1; i<=n; i++) //看有多少个集合
        if(Parent[i]<=0) //看有多少个根结点
            cnt++;
    return cnt;
}
```



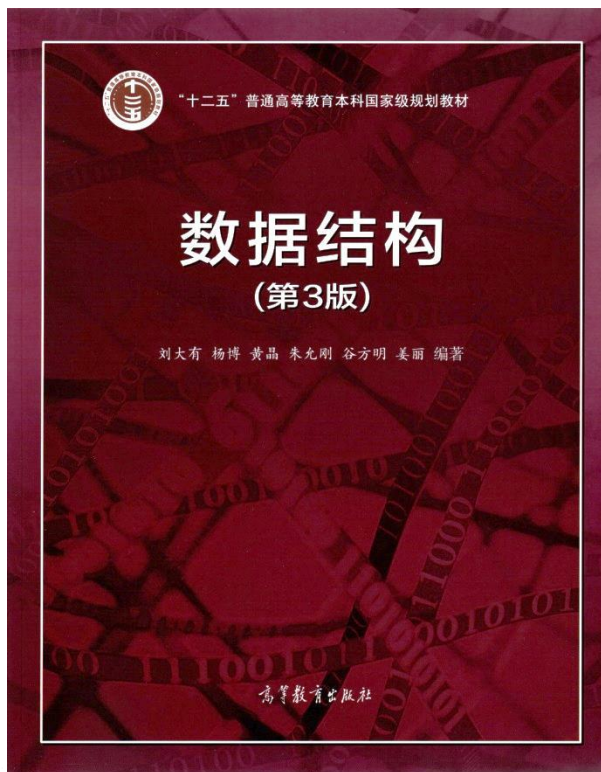
## 课下思考

操场上有很多同学在玩耍，体育老师要对他们排队。初始时操场上有 $n$ 位同学，自成一列。每次操作，老师的指令是“ $x\ y$ ”，表示学生 $x$ 所在的队列排到学生 $y$ 所在的队列的后面，即 $x$ 所在队列的队首排在 $y$ 所在队列的队尾的后面。（如果 $x$ 与 $y$ 已经在同一队列，则忽略该指令）。现给定若干指令，请输出执行所有指令后，每位同学所在队列的队首。【北京大学上机练习题，吉林大学上机考试题】

- 一个队列  $\Leftrightarrow$  一个集合
- 指令“ $x\ y$ ”， $x$ 所在集合与 $y$ 所在集合合并 $\text{Union}(x, y)$ ，注意题目规定了合并次序，故不能用按秩合并。
- 队首：根



# 树和二叉树的应用



- 压缩与哈夫曼树
- 表达式树
- 并查集
- **其他应用举例**

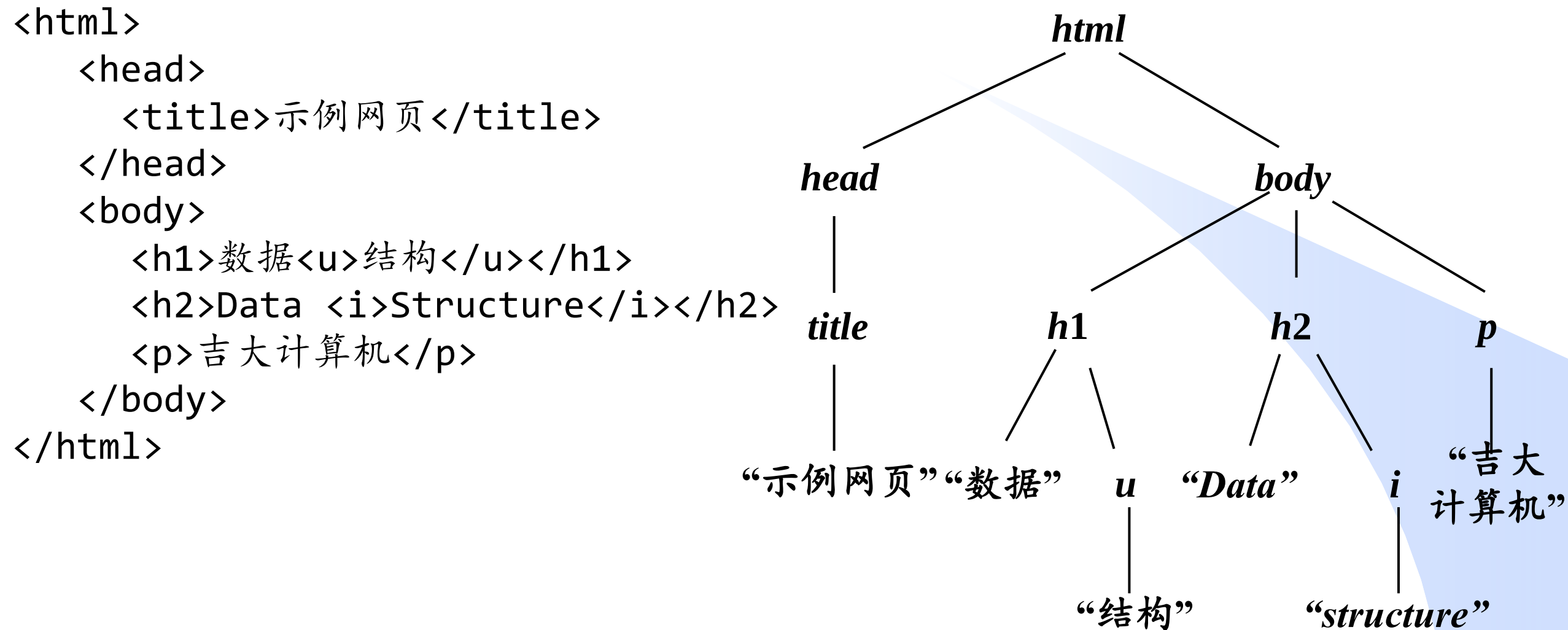
数据之法  
结构之美  
算法之道

zhuyungang@jlu.edu.cn

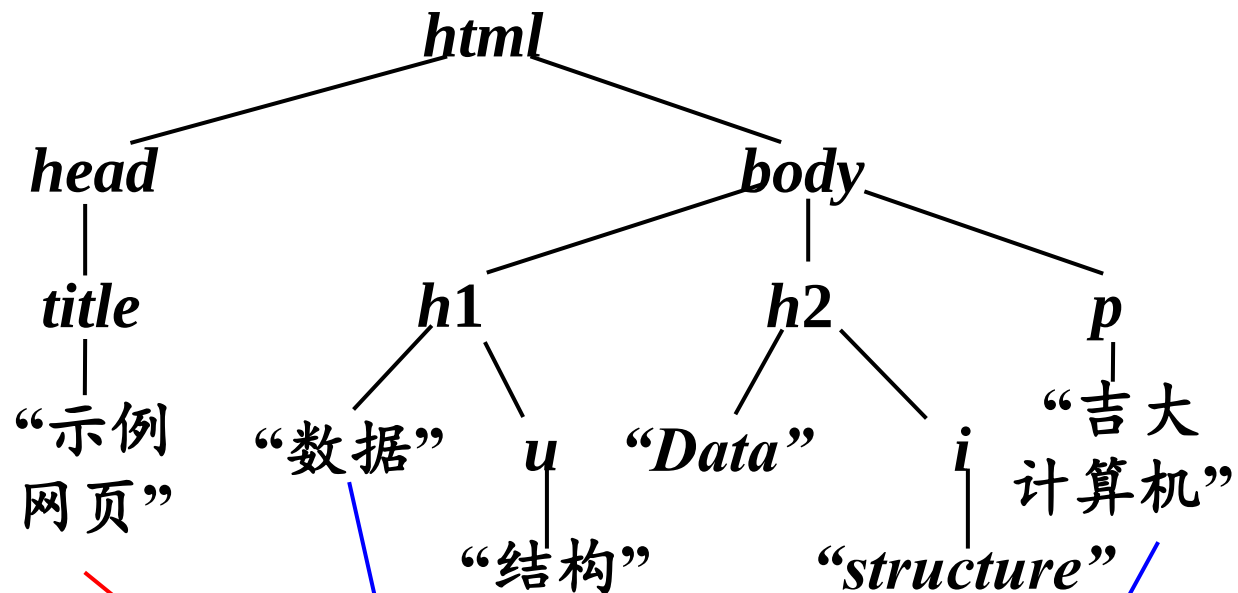
# 网页与DOM树

- **HTML: HyperText Markup Language (超文本标记语言)**。从语义的角度描述页面的结构。用于描述超文本中内容。
- 一个html网页的所有内容，在浏览器中以树形结构保存，称为**DOM (Document Object Model) 树**。其结点是网页中的每个标签和文本内容。
- 当打开网页时，浏览器会解析网页的html代码，生成**DOM树**。DOM树是网页的基础架构，为网页开发提供了强大的操作性和灵活性。

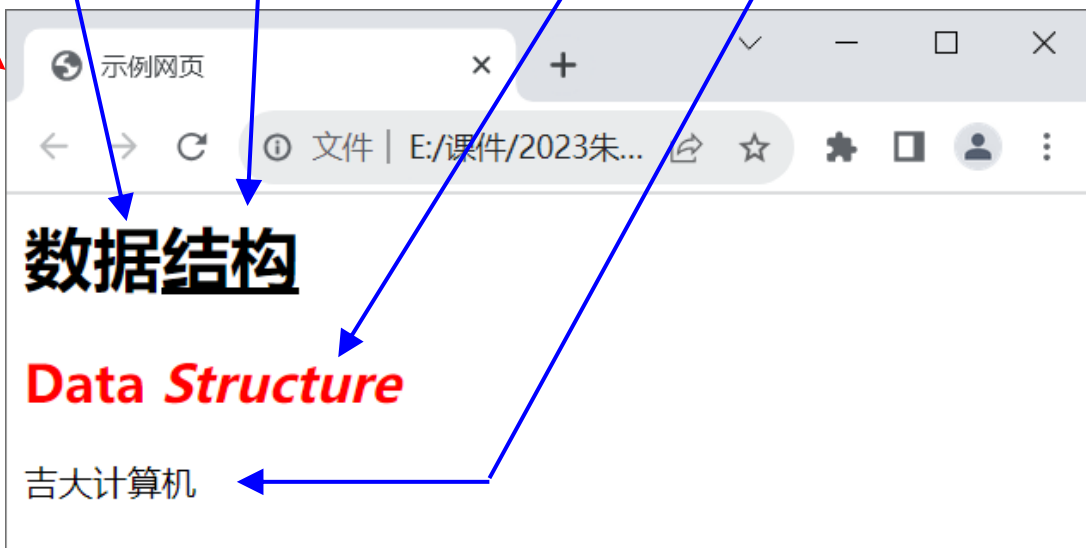
# 网页与DOM树



# 网页与DOM树



浏览器通过  
*DOM* 树将 *html*  
网页展示出来





# 计算机如何下棋？



设计一个评估函数，  
评估某一局面对本方  
的有利程度，分值越  
高表示对本方越有利

|   |   |  |
|---|---|--|
| O | X |  |
|   |   |  |
|   |   |  |

$f$  = 本方可能的三子  
连珠数目 - 对方可能  
的三子连珠数目

|   |   |  |
|---|---|--|
| O | X |  |
| O |   |  |
|   |   |  |

$$f = 6 - 4 = 2$$

|   |   |  |
|---|---|--|
| O | X |  |
|   | O |  |
|   |   |  |

$$f = 6 - 2 = 4$$

|   |   |   |
|---|---|---|
| O | X | O |
|   |   |   |
|   |   |   |

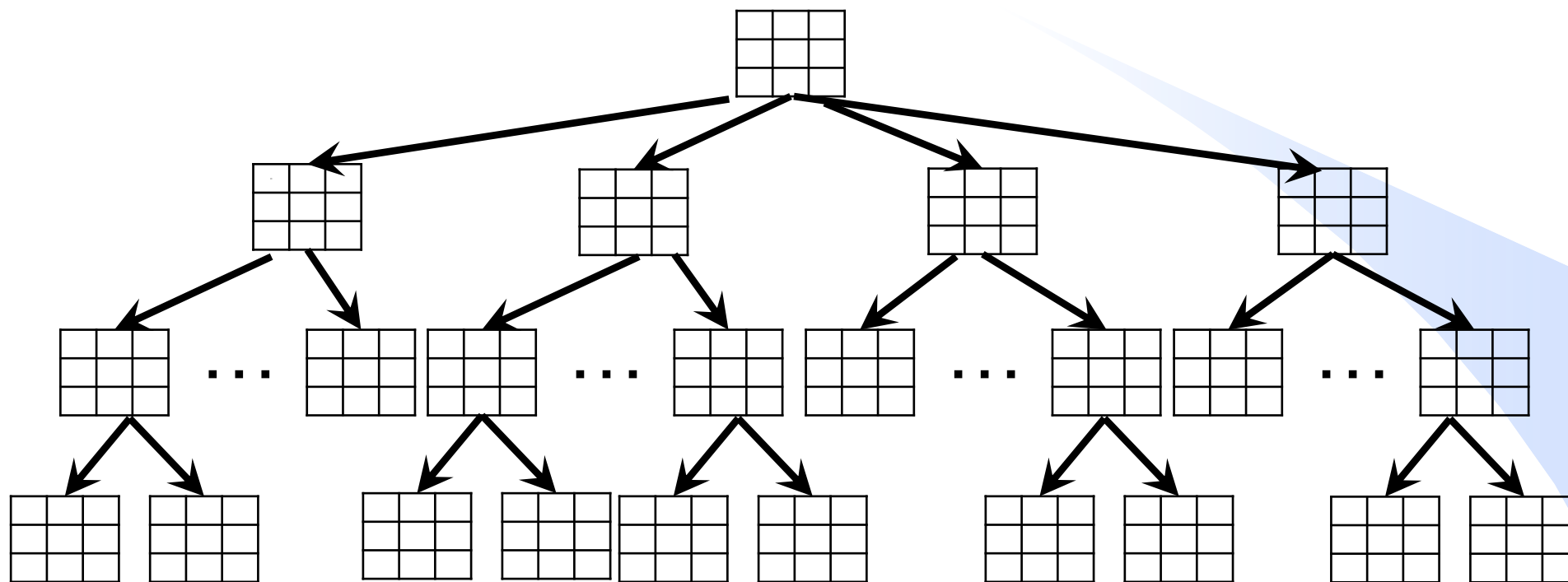
$$f = 6 - 3 = 3$$

.....

选择使下一个棋局状  
态对我方最有利（评  
估值最大）的动作。

# 博弈树

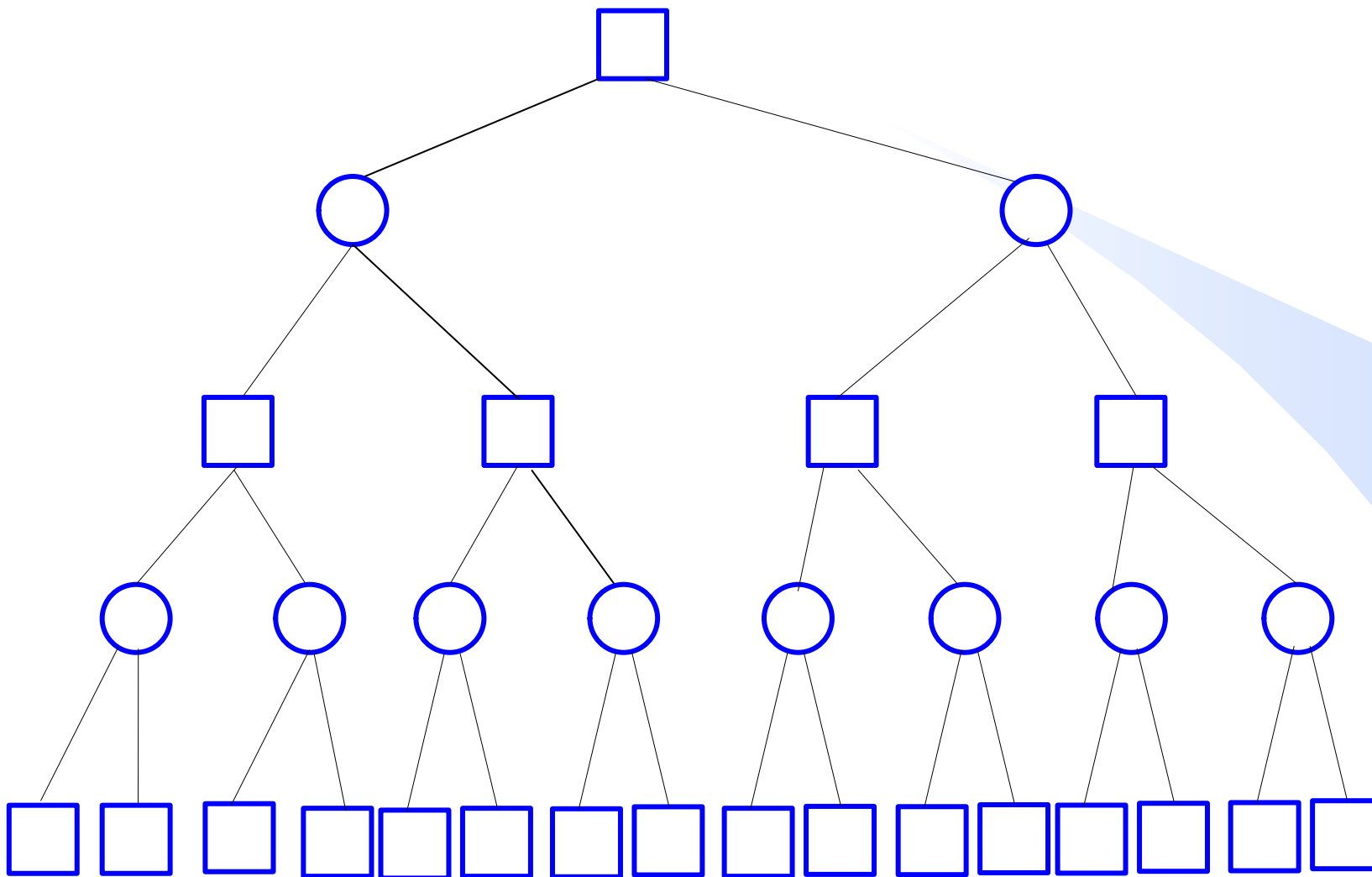
*D*



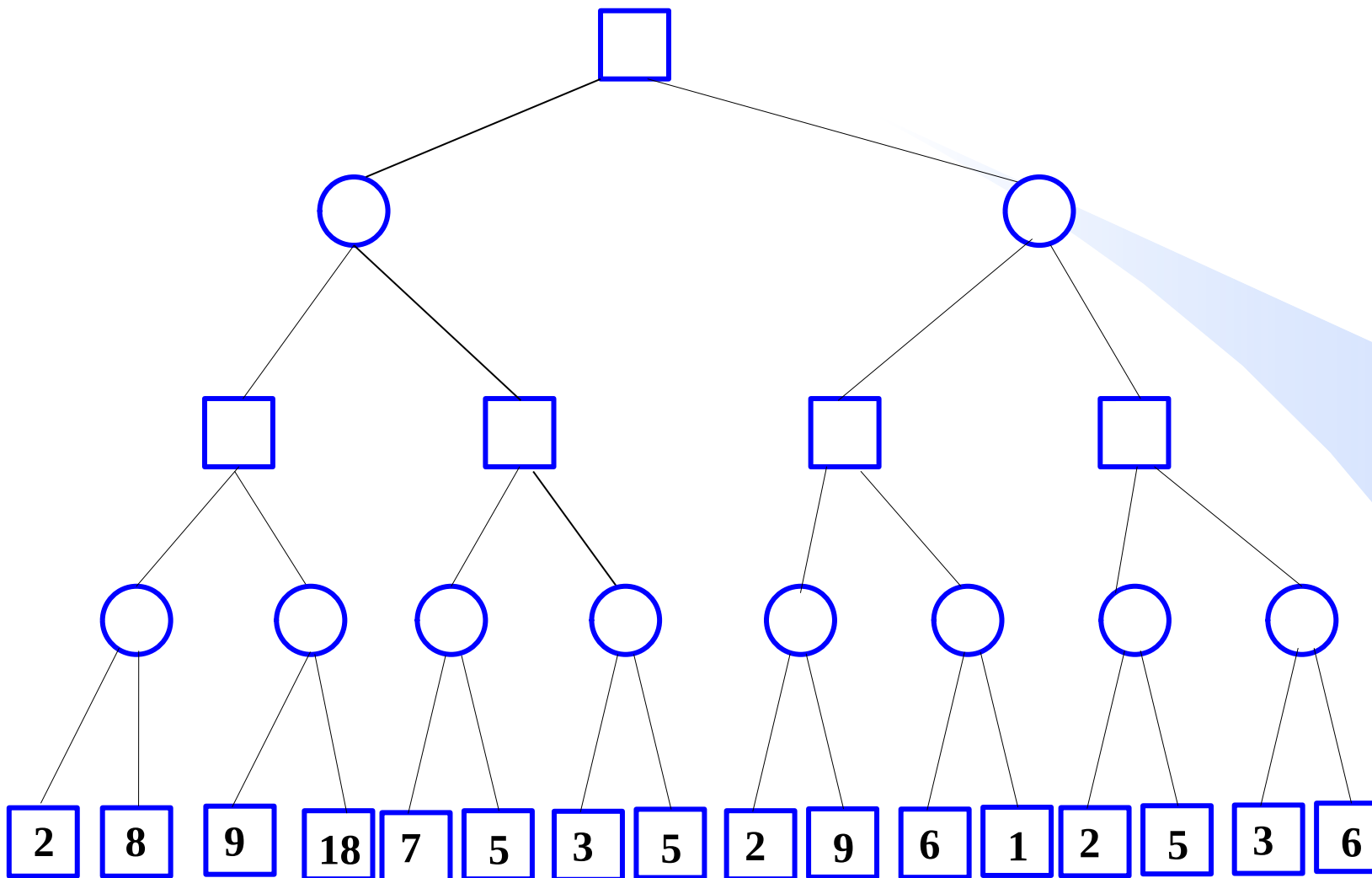


# 博弈树的极大极小搜索

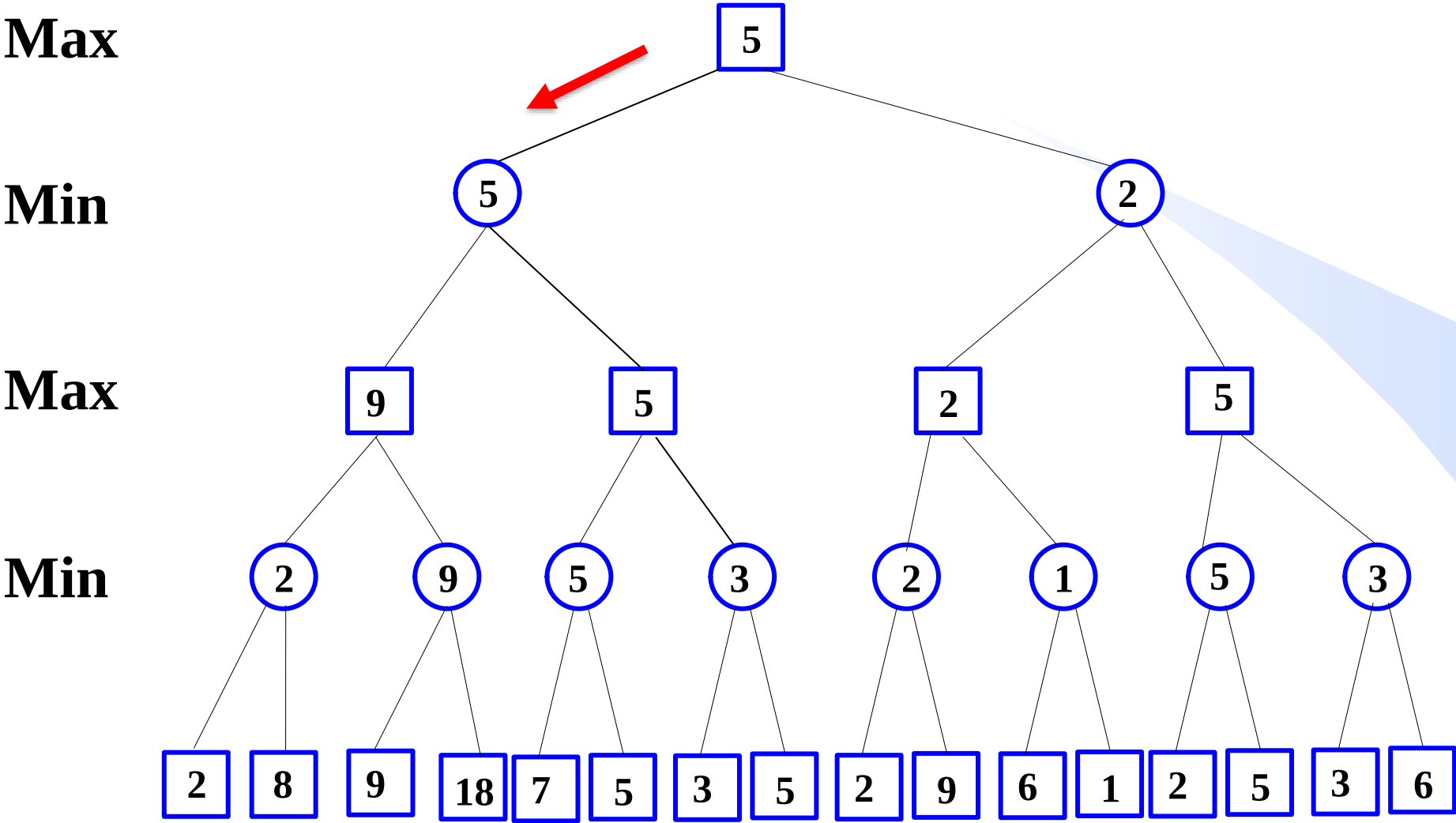
*D*



# 博弈树的极大极小搜索



# 博弈树的极大极小搜索



# 博弈树的极大极小搜索



**John von Neumann**

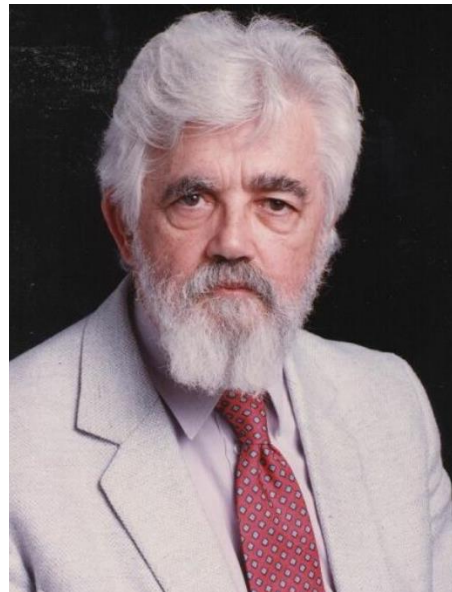
(冯·诺依曼)

提出博弈论的极大极小原理

美国科学院院士

普林斯顿大学教授

博弈论之父



**John McCarthy**

提出 $\alpha$ - $\beta$ 剪枝技术

斯坦福大学教授

图灵奖获得者

美国科学院/工程院院士

人工智能之父

# 1997年卡斯帕罗夫 2.5 : 3.5 Deep Blue



卡斯帕罗夫

国际象棋世界冠军，连续23年世界排名第一



Artificial Intelligence 134 (2002) 57–83

Artificial  
Intelligence

[www.elsevier.com/locate/artint](http://www.elsevier.com/locate/artint)

## Deep Blue

Murray Campbell<sup>a,\*</sup>, A. Joseph Hoane Jr.<sup>b</sup>, Feng-hsiung Hsu<sup>c</sup>

<sup>a</sup> IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, USA

<sup>b</sup> Sandbridge Technologies, 1 N. Lexington Avenue, White Plains, NY 10601, USA

<sup>c</sup> Compaq Computer Corporation, Western Research Laboratory, 250 University Avenue, Palo Alto, CA 94301, USA

### Abstract

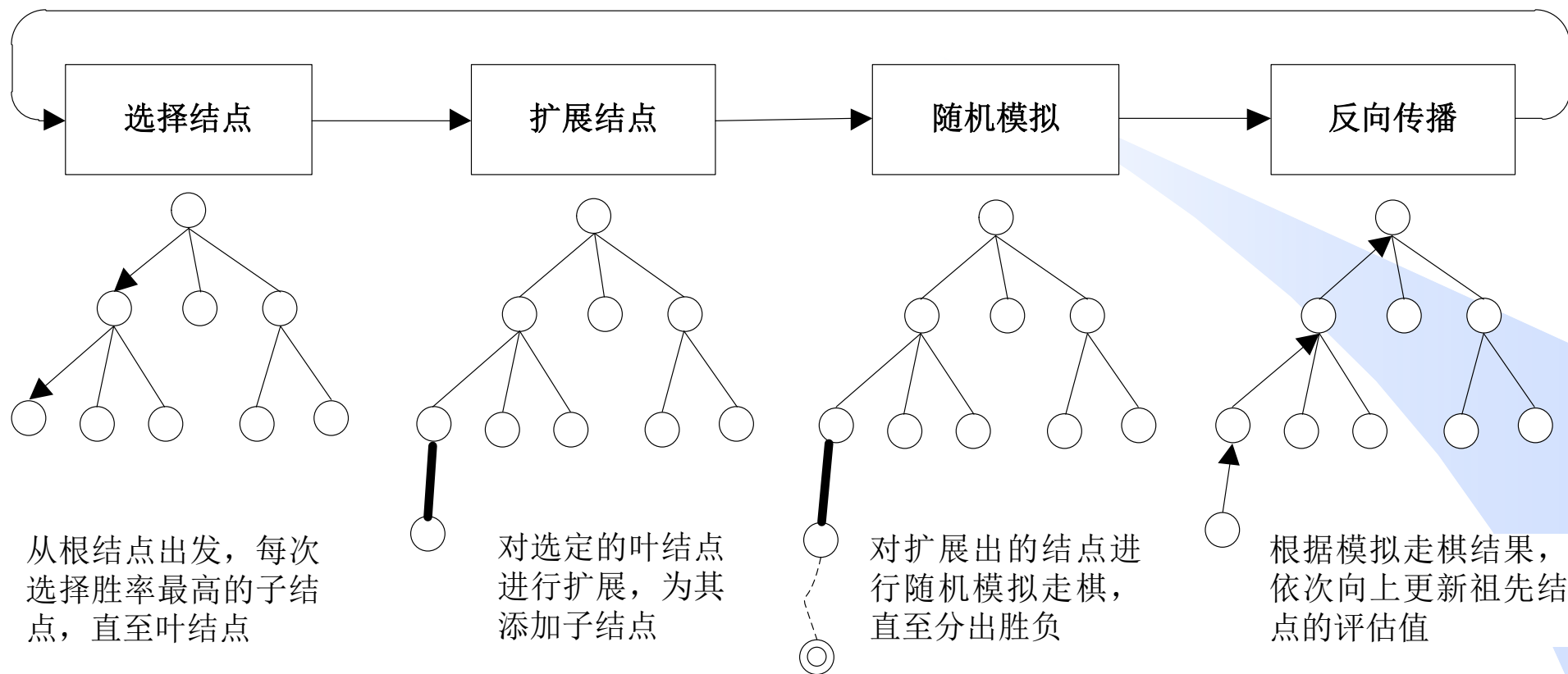
Deep Blue is the chess machine that defeated then-reigning World Chess Champion Garry Kasparov in a six-game match in 1997. There were a number of factors that contributed to this success, including:

- a single-chip chess search engine,
- a massively parallel system with multiple levels of parallelism,
- a strong emphasis on search extensions,
- a complex evaluation function, and
- effective use of a Grandmaster game database.



# 蒙特卡洛树搜索

## Monte Carlo Tree Search, MCTS





Google DeepMind  
Challenge Match

8 - 15 March 2016



AlphaGo

李世石 (韩国)

2006-2011年世界排名第一

获14次世界冠军

当时现役棋手中获世界冠军最多

# 2016年 AlphaGo 4:1 李世石



AlphaGo: 蒙特卡洛树搜索和深度学习技术相结合

David Silver, et. al. Mastering the game of Go with deep neural networks and tree search. *Nature*. 529:484-489, 2016.



# 2017年 AlphaGo Master 3:0 柯洁

柯洁（中国）

2014-2019年世界排名第一

获8次世界冠军







# AlphaGo Zero

无师自通：初始训练时不再使用人类棋谱做有监督学习，而是直接从随机下法开始自对弈强化学习

训练3天后

**AlphaGo Zero    100 : 0    AlphaGo**

训练40天后

**AlphaGo Zero    89 : 11    AlphaGo Master**

David Silver, et. al. Mastering the game of Go without human knowledge. *Nature*. 550: 354–359, 2017



# 自愿性质OJ练习题

- ✓ [HDU 1232](#) (并查集)
- ✓ [POJ 3253](#) (哈夫曼算法求WPL)
- ✓ [POJ 1308](#) (树基础)
- ✓ [HDU 1213](#) (并查集)
- ✓ [POJ 1521](#) (哈夫曼编码)
- ✓ [UVA 12676](#) (哈夫曼编码)